FINANCE BLUEPRINT

00 **How to read this**

01 **Standards**

02 **Development plan**

03 **The tabs**

THE TABS · 33 SCREENS

▾ **1 · General ledger**

1·1 Journal

1·2 Balance

1·3 Chart

1·4 Accruals

1·5a New entries

1·5b Reverse

▸ **2 · Revenue & AR**

▸ **3 · Deposits & Bank**

▸ **4 · Taxes & Remittance**

▸ **5 · Costs**

▾ **6 · Close & export**

6·1 Close

6·2 Export

6·1a Close the month

6·1b Reopen

6·2a Hand over

04 **Journeys**

05 **Cross-module**

06 **Register**

LEGEND

new update

keep remove

SMARTBILLHUB · FINANCE

Finance, as built and as planned

Six tabs, thirty-three screens, audited against the running code at `c9aab18` and driven in a real browser on a seeded utility. This page is what a build lane reads **instead of a conversation**: what exists, what a user can actually reach, what runs behind it, what the product refuses to do, and what we are committing to build.

VERSION	COMMIT	TABS	SCREENS	NAMING NO DOOR	WALKED	TASKS	ENDPOINTS, NO SCREEN
2.340.0	`c9aab18`	**6**	**33**	**1**	**15**	**92**	**22**
three literals agree	2026-09-23	the ratified strip, exactly	4 new · 21 update · 8 keep	the deliberate catch-all	in a browser, signed in	across six phases	incl. four accrual engines

SECTION 00

How to read this

Every statement here is one of six things, and the chip says which. A lane that cannot tell them apart will either build what nobody committed to or skip what was. The machine-readable copy is the JSON block at the top of this file — `#blueprint-manifest` — which carries every task, screen, dropdown source, ruling and open question as data, so nothing has to be parsed out of prose.

MARKER	MEANS	WHAT TO DO WITH IT
TASK	Committed work. Carries a plan id `Pn.m`	**Build it.** The plan gives it a reference to open and an outcome that says when it is done
TODAY	What is true in the code now, with a file and line	Do not re-derive it, and do not change it without a TASK. It was read, not assumed
GAP	A defect or a missing piece	**Information only.** Actionable only if it also carries a TASK — several gaps here are deliberately not scheduled
RULED	A Product Owner decision, with its date	Binding. Never re-ask — CLAUDE.md hard-won law 13: a re-ask manufactures inconsistency and then blames the human for it
OPEN	Nobody has answered, with the question's id	Never resolve it by choosing a value. Stub with the id, per the prime directive
SOURCE	Where a control's values come from	No drawing in this document authorises a literal array. The full set is in §01

GAP **The GAP / TASK split is the one that matters.** This audit found 92 rule breaches, seven refusal sentences that are no longer true, eleven foreign keys that are bare UUIDs and 22 endpoints no screen calls. Some are in the plan and some are deliberately not. Without the split a lane reads a finding as an instruction — or skips one that was scheduled — and either way the plan and the code diverge silently.

So: **a GAP with no TASK beside it is reported, not scheduled.** If you think it should be, that is a conversation with the Product Owner, not a decision to make while building.

TODAY **The manifest is derived from this document, not written beside it.** A hand-kept index is a second source of truth that drifts. This one is generated from the marked HTML, so a task with no id, a screen with no class or a dropdown with no source is absent from the manifest and its absence is visible.

It earned its place on the first run, and here is exactly how. Generating it against the assembled file caught **four stale counts** that reading could not: the masthead said **62 tasks** and there are **63**; Phase 0's mix said **DB 6** and there are **5**; Phase 1's said **Front end 7** and there are **8**; Phase 2's said **Front end 4** and there are **3**. Every one was written by hand beside the tables

rather than derived from them. **The previous version of this document asserted "92 rule violations" and "45 absences" in its masthead, computed from a JSON file I typed myself** — a second source of truth, which is the thing this replaces.

Three corrections to the previous version of this document

Per the standing rule: when we are wrong we say so in the place it was wrong, with the correct reading and how it was caught.

WAS	NOW	HOW IT WAS CAUGHT
The journal list drawing showed `JE-2609-0043`	The column is **empty**. 0 of 168 entries carry a number; `number_series.journal_entry.next_number` is still **1**	Querying the live table instead of trusting `BR-LED-021`. The drawing showed the rule, which **launders C1 into something that looks shipped**
Bank import drawn as two screens	**One screen.** Step 1 always renders; step 2 is a conditional panel *below* it. No rail, no fence	Reading the component. The filename says `Wizard`; the code is two stacked panels
"No fact in Finance has two homes"	**27 currency columns**, all free text, no lookup table, nothing linking them to the company setting	A reader asked why currency was not a foreign key. The cross-module pass had answered its own question wrong

> RULED **Two placement calls recorded here rather than re-asked.**
>
> **Refunds sits in Tab 2, not a tab of its own.** It answers at `/finance/refunds` and is **not** a Finance tab — the PO ruled the strip at six on 2026-09-03 ("*C25 says six tabs. - this is correct.*") and folded Refunds into Revenue & AR "*where the receivable it gives up already lives.*" All four of its doors are in Payments. An eighth tab here would contradict a ruling. **Considered and declined:** giving it its own tab because its page is full-size — declined, because the ruling is about where the act belongs, not how big the screen is.

What changed in this revision, and why

Fourteen changes, **every one of them raised by the Product Owner reading the previous revision**. None came from a gate. That is worth stating plainly: **no check in this repo reads a drawing**, so a document review is the only thing standing between a wrong drawing and a lane building from it.

CHANGE	WHY	LANDS AS
Tab 1 split into four	*Journal* · *Balance* · *Chart* · *Accruals*, one word each. "*Trial balance & journal*" was two screens wearing one name. `balance` **keeps its URL value** so bookmarks land; `journal` is new and becomes the landing view	P1.15
Full table UI on all four	Search · filter · columns · download · sort · pager · empty state, plus **a filter-criteria table and a components table under every drawing**. The criteria are read out of the handlers, not invented	P1.1–P1.9
Manual entries are untagged	`journal_lines` carries five dimensions; the manual modal captures none. **661 of 663 live lines are tagged and a hand-typed one is not**, so three reports drop it silently	P1.16 · `OQ-LED-1513`
2·1 re-laid out	Nine panels in build order → four bands: *The month* · *Does it agree?* · *Waiting for you* · *Go deeper*	P2.6
The refund queue cannot approve	It sits directly under a write-off queue that approves inline, and renders no action at all	P2.7
Batch lines have no actions	Three `PlainTable`s and one page-level button, on the screen where a wrong cheque number is found. **Two of the three missing acts already have endpoints**	P3.6 · P3.7
Resolve draws one line	The service takes a **tuple** of explanations and checks their sum; the modal builds one. A card shortfall is routinely a fee **and** a chargeback	P3.8
A batch code is not searchable	Only `f.status` and `f.method` are served. **The screen's problem, not the modal's**	P3.9
Tax payments have	`POST /finance/taxes/remittances` with **no GET and no model**. Owed and paid never appear together	P4.6

no list		
6·1 rebuilt as a close	Reported verbatim: "*as user i dont understand this screen at all*". Three system-side panels → three numbered steps, and **every not-done row becomes a door**	P5.4 · P5.5
No way to attach an invoice	21 columns, no document, and **no general attachment store anywhere in the platform**	P5.6 spike · `OQ-LED-1512`
Two screens had no drawing	2·4b and 5·3b carried a spec table and no wireframe. **B14 rule 42 makes the drawing the specification**, so those two were never actually specified. Now 29 of 29	drawn
On-screen text cut	**47 strings over the cap**, the longest 31 words. See below	throughout
Tab 7 removed	Reports are at `/reports/*` and were never in the Finance strip. **The document now matches the ratified six tabs exactly**	29 screens

THE TEXT CUT, AND WHY IT WAS A RULE BREACH RATHER THAN A STYLE NOTE

B14 rule 33 caps on-screen text by what it is for — a label ≤ 3 words · a fact ≤ 6 · an instruction ≤ 6 **per sentence** · a consequence before an irreversible act ≤ 6 per sentence and at most two · and **rationale is ZERO WORDS ON SCREEN**.

The previous revision drew **47 strings over the cap** inside its wireframes. The worst was 31 words on 3·4; the one that named the problem was on 1·5b: "*A posted entry is never edited. This writes a mirror entry that cancels it, then opens a corrected copy for you to check and post.*" **That is rationale, and rule 33 removes rationale rather than shortening it** — so it is deleted, not trimmed. A clerk who pressed ↻ already intends it.

⚠ **Why this mattered more than it reads. B14 rule 42 makes the drawing the specification.** A lane building 1·5b from the previous revision would have shipped those 25 words, passed `tsc`, passed every gate — **no check in this repo reads rendered copy** — and broken a ratified rule *because this document told it to*. **A drawing that breaches a rule is worse than no drawing**, and that is the reusable lesson here.

What was kept: data strings that only look long — a period band, a pager, a breadcrumb, an account pair like `DR 6900 / CR 1200`. Those are structured facts, not prose, and cutting them would lose meaning rather than words.

EVERY MODAL NOW STATES ITS DOOR, ON THE DRAWING

A page screen has always carried a **Doors** row in its spec table; a modal did not, so **twelve modals were drawn floating**, with nothing saying what opens them. Read as a set, the document looked as though those screens were unreachable — and **one of them genuinely is under-reachable** (2·2b, from the board). Each modal now carries an **Opens from** band above its drawing, naming the control, the screen and the state that gates it. **Verified in the code, not inferred from the flow**: 2·2a's approve is at `WriteOffQueuePage.tsx:273`, 3·3a's at `DepositsPage.tsx:262`, 4·2's at `RemittancePage.tsx:316`.

THE MANIFEST CAUGHT ONE STATED COUNT AGAIN, WHICH IS WHAT IT IS FOR

The masthead was written by hand as **2 new · 19 update · 8 keep**. The generated manifest reported **2 · 18 · 9** — 2·4b is a `keep`, not an update. **Corrected before publishing.**

That is the second revision running in which the only wrong numbers in this document were the ones a person typed. **Every count on the masthead is now derived**: 29 screens, 86 tasks across six phases, 33 filter criteria of which **21 are already served by a handler**, 16 dropdown sources, and 18 gaps against 32 tasks. **Read the manifest, not the prose.**

A DEFECT I INTRODUCED IN THIS REVISION, FOUND BY THE RE-CHECK AND RECORDED HERE

Asked to double-check the phases, the first thing the check found was **my own bug: eleven duplicate task ids.** `P2.6`, `P2.7`, `P2.8`, `P3.6 – P3.9`, `P4.6`, `P5.4 – P5.6` each existed **twice**.

The cause is exactly the shape CLAUDE.md law 11 describes. I inserted each new task after the last *Front end* row I could see in its phase — but every phase continues past that with Test, E2E, Settings and Other rows numbered higher. **I computed "the next free number" correctly against a world I had only partly read.** That is `DEF-CWP-3734`'s mechanism, where three lanes each computed the next free gate number correctly and all got the same answer.

Nothing downstream would have caught it. A duplicate id in an 85-row plan reads as a plan. The manifest counted 76 tasks and **was right** — it counts rows, and there were 76 rows. **The count was never the thing that was wrong.** Renumbered to the true maximum per phase; a duplicate-id assertion now runs with the manifest.

And two more honesty corrections from the same pass: I named `AbsentFigure`, `BandHeading` and `CountBadge` in the components tables as though they were part of the shared kit — **none of the three exists** (P2.14, P2.15). And `OQ-LED-1512` and `OQ-LED-1513` were written as if minted; they are **PROPOSED and not allocated**, and P0.15 allocates them against the remote. **Naming a component**

that does not exist is how a drawing becomes unbuildable, and it is this document's own version of hard-won law 1.

AND A RENDERING FAULT IN THREE DRAWINGS, WHICH EVERY CHECK PASSED OVER

1·2, 1·3 and 1·4 were drawn with no left rail. The frame is a fixed-width grid; without the rail the main column rendered at the wrong width and the screens were visibly broken.

⚠ **The verification said they were fine, and it was not lying.** It measured horizontal overflow, wireframe count and zero-height elements, across six widths and two themes — **and a missing rail produces none of those symptoms**. It reported clean over a scope it silently excluded, which is **hard-won law 7's fifth shape**, the one this repo fails in most often. **The Product Owner found it by looking at the page.**

The fix is the check, not the three screens. `verify3.mjs` now asserts, for every drawing: a page screen has a rail, a topbar, a page head and a tab strip; the main column starts after the rail and is at least 400px; and nothing overflows its own frame. **Proved RED first** — stripping the rail from 1·2 makes it print `1·2 PAGE WITH NO RAIL`. **29 of 29 clean.**

THE DRAWINGS WERE NOT BUILT TO THE TOKEN SCALE, AND NOW THEY ARE

Challenged on it, the audit found **157 off-scale values in the wireframe kit across 66 distinct numbers**, and 65 more in the screens' inline styles. **The kit had been tuned by eye** — `11.5px`, `12.5px`, `9px`, `13px`, `18px` — against a repo whose scale is **4 · 8 · 12 · 16 · 20 · 24 · 32** and whose type runs **11 · 12 · 13 · 14 · 15 · 16 · 20 · 25 · 32**, with `13.5px` reserved for table bodies and `12px` for table headers.

Every value is now on the scale — measured, and re-measured against the source. Four structural numbers were wrong and are corrected from the stylesheets rather than from memory: the topbar is **56px with a 2px divider** (`AppShell.css:261-262`, it was 54px and 1px), a control is **36px** high (it was 34), the tab strip is **13px text-base** (it was 13.5px, which is the *table* size), and the KPI strip keeps its **2px** rules (`KPIStrip.css:41-42`). The rail is 240px from `BR-SHL-001`, the page-head title 32px from B14 rule 40, radius 0 everywhere.

The guard is the deliverable, not the snap. `10-scripts/tokens_check.py` **reads the scale out of** `07-frontend/src/styles/tokens.css` **and never retypes it**, so when a token moves the check moves with it. **Proved RED first** — injecting a single `13.7px` makes it name the value and exit 1. **0 off-scale across 285,819 characters.**

⚠ **And the first attempt at this silently did nothing, which is worth recording.** The snapping script looked for a comment marker that was not in the file, so it selected an **empty** region, rewrote it, and the audit that followed reported **0 off-scale** — truthfully, of nothing. **That is hard-won law 7's fifth shape for the third time in this revision**: a check reporting zero over a scope it silently excluded. It was caught only by grepping the file for the values the script claimed to have removed and **finding them all still there**. The rewrite now asserts on its own slice before touching it.

AND THE SAME CHECK, TURNED ON THE PRODUCT

The audit above was of **this document's drawings**. Pointed at `07-frontend/src` the same gate finds **65 off-scale occurrences in 11 files** — so the blueprint is now stricter than the thing it describes. **P2.17** closes that, and **deliberately not by bulk-snapping**: `AppShell.css` holds 25 of the 65 and its own header says they are a prototype port under `BR-SHL-001`. **A blanket snap would break a ruled measurement while looking like housekeeping** — which is exactly what this revision's own snapper did when it flattened the KPI strip's 2px rules to 1px. Three values are verified and fixed first; the rest is triaged per file.

THE KPI STRIP WAS BROKEN IN EVERY REVISION, AND NO CHECK COULD SEE IT

Reported from the page: the tiles on 1·2 rendered **stacked in one column** with label and value run together — "*Debits$44,512.90*".

Two causes, and the second is the one that matters. (1) the token snap flattened `.kp .v.clip` from `146px` to `32px` — my own damage, restored. **(2)** `.kp` **never had** `grid-template-columns` **at all, and** `.kp>div>i` **and** `b` **never had rules** — not in this revision, not in the previous one. **The strip has been broken since it was first drawn**, on every screen that carries one.

⚠ **Four checks passed over it.** Overflow: none — a stacked strip fits fine. Structure: present — the element exists. Token scale: clean — every value was legal. The class audit: clean — **every class used had a rule**. **The fault was in child-element rules and a missing layout property, and nothing static can see either. Only rendering it can.**

Rebuilt from `KPIStrip.css`**, not from memory:** `repeat(auto-fit,minmax(180px,1fr))`, tile padding `16px 20px` with the first flush left, label **11px / 0.1em / uppercase / weight 800**, value **26px Archivo 800** — the ruled metric step, not the 30px it used to be.

And the check that closes it: `verify4.mjs` **renders every drawing and asserts the components.** KPI tiles side by side and label above value; table header 12px and body 13.5px; topbar 56px; rail 240px; page head 32px; and **every rendered font size on the scale** — measured through the fit-to-width transform, so the geometry is compared in design pixels. **Proved RED first**: removing the grid property makes it print "*KPI strip STACKED — 4 tiles in one column*" on 1·2 and 2·1. It also caught two faults nothing else had — a **10px** count badge and a **10.4px** inline `code` inheriting a relative size. **29 of 29 clean.**

THE PARAMS BAND AND THE VIEW SWITCHER WERE NEVER COMPONENTS EITHER

Reported from the page: *From 2026-09-01* 📅 *To 2026-09-30* 📅 *Property All* ⌄ *Apply* rendered as one line of loose text on 1·2.

Cause: `.pm` had exactly one rule — `.pm b{color:…}` — and no container at all. No background, no border, no layout, and the dates were **text rather than inputs**. In the product both this band **and the view switcher** are the same class, `.fin-params` (`finance.css:118`): flex, gap 12, wrap, align-end, `12px 16px` on the surface colour with a **2px top rule**, fields that zero their own margin, and the actions pushed right by `margin-left:auto`. **Both are rebuilt from it**, and each control is now a real field — label above, input below.

How many places had it — the question asked. Sweeping all 29 drawings for a control glyph outside a field: **4 screens, 8 places, and 2 of those were pager footers** (false positives — `Rows 50` ⌄ belongs to the footer component). **The real count is 1·2's params band and 1·5a's two GL-account pickers — all of them screens written in this revision**. The older drawings were using the kit correctly; the new ones were not.

And the KPI tiles now carry their caption line, which the kit had a rule for (`.kp .d`) and no drawing had ever used.

⚠ **The check for this was wrong on the first attempt, and the red proof is the only reason I know.** It walked ELEMENTS — `querySelectorAll('*')` — and a loose control is a **bare text node with no element to find**. Pointed at a deliberately broken copy it reported **clean**. It walks text nodes now, and prints *control drawn as raw text: "2026-09-01* 📅*"* on the broken copy and nothing on the real one. **A check that cannot be shown red is not a check** — that is hard-won law 7, and it has now caught three of my own faults in this revision alone.

ELEVEN OF TWENTY-NINE SCREENS ARE NOT FINANCE'S TO BUILD

Raised from the page, and it is the largest finding in this revision: *"we already have this screen in payments — use the same instead of recreating it … otherwise we will duplicate the work."*

Checked by file path, not by name. Eleven screens are `features/payments/` over `apps/payments/`: the five refund and write-off surfaces in tab 2, and **every one of the six screens in tab 3**. 2·1 is ledger code but **mounts three Payments panels**. Tabs 1, 4, 5 and 6 are clean.

Why it matters beyond wasted effort: a Finance lane that "builds" one of these rebuilds something already shipped and tested, **in a module it does not own** — and cross-module writes are forbidden, so it is a boundary violation rather than duplicated work. **Eight tasks are re-attributed** — P1.2 · P1.14 · P2.12 · P2.13 · P3.10 · P3.11 · P3.12 · P3.13 — and each now carries a **PAYMENTS** flag. **They stay in the plan**; they are raised, scheduled and reviewed with Payments.

The split is deliberate, not drift. `BR-PAY-020`: *"Payments owns the transaction record, ledger posts, FINANCE REPORTS."* `CR-060` C25 folded the queues into Revenue & AR *"where the receivable it gives up already lives"* — **the door moved, the code did not**. The full map is §05.

OPEN `OQ-PAY-1514` **PROPOSED** **— do the refund screens belong in Finance at all?** The PO is reviewing every refund surface in Payments. **Left open**; nothing in this document assumes an answer.

1·5A IS NOW A BATCH SCREEN — RULED 2026-09-23

RULED **PO: the screen takes "*1 or more entries*", each row its own journal entry, and the table is called Entries.** The composer lives in the **Entry** panel and `+ Add line` pushes what you typed into the table below.

⚠ **That ruling decided the composer's shape, not just its position.** `BR-LED-003` makes every entry balance on its own, so a row that IS an entry cannot be one account and one amount. The composer therefore takes a **debit account, a credit account and one figure** — balanced by construction. **Had the rows been lines of one entry, a single account per row would have been right**; the ruling is what rules it out.

OPEN `OQ-LED-1515` **PROPOSED** **— if entry 3 of 5 fails, what happens to 1 and 2?** The service posts one entry per call, so a batch is N calls. **Doing nothing chooses "each stands alone"** — 1 and 2 stay posted, and hard rule 4 means they never come back out. **That is why it is raised rather than defaulted**, and P1.22 is blocked on it.

Superseded within this revision: P1.20 drew a single-entry screen and is replaced by **P1.21** + **P1.22**. Recorded rather than quietly overwritten, because a task id that vanishes is a task somebody may still be holding.

THE EARLIER SINGLE-ENTRY RATIONALE, KEPT BECAUSE IT STILL HOLDS

The manual journal entry leaves its modal for `/finance/journal-entries/new` — lines table on the left, summary and post gates in a column of their own. **P1.16 forces it**: five dimensions per line take a line from three fields to eight, and no modal holds that. A draft reopens at `/{id}/edit`, which it needs because a draft already has a lifecycle and **a thing you come back to needs an address**. **1·5b stays a modal** — one field over the row it acts on. **The rule is not "screens are better"**: a form whose size is set by the data belongs on a page, and a confirmation belongs where it was invoked. **P1.20**, and no rule, gate or endpoint moves.

Revenue & AR becomes eight views — ruled 2026-09-24

WHAT CHANGED

RULED **The board is retired and each of its categories is a view with a table, search, filters and a column chooser** — Services · Aging · Tie-out · Write-offs · Refunds · Account ledger · By premises · Statements. **Tab 2 goes from 7 screens to 13, the document from 29 to 35.** The PO's own tables — which search, filter and page controls the server supports today, the screen count, and the tasks — sit at the top of tab 2.

Three views return from the Reports tab removed on 2026-09-23, placed under Revenue & AR by this ruling. **Only 2 of the 8 views are Finance code**; the other six are Payments or Reports, mounted rather than rebuilt.

Old number → new number

Earlier entries in this log use the old numbers and are left as written. Read them through this map.

WAS	NOW
2·1 The board	**retired** — its panels became 2·1 Services, 2·3 Tie-out and 2·4 Write-offs; its links became 2·6–2·8
2·2a Approve write-off	2·4a
2·2b Approve refund	2·5a
2·3 Refunds queue	2·5 Refunds — the same screen, as a view
2·4a Raise a refund	2·5b
2·4b Issue · clear · void	2·5c
2·5 Refund detail	2·5d
—	2·2 Aging **(new)** · 2·6 Account ledger · 2·7 By premises · 2·8 Statements **(returned)**

Tasks replaced, and one duplicate retired

TASK	WHAT HAPPENED
P2.11	The four bands — **replaced by P2.18**; there is no board to lay out
P2.12	Approve on the refund row — **resolved by P2.18**; the Refunds view already carries Approve
P2.13	Toolbars on both queues — **folded into P2.19**, which became full stack
P3.13	Search a batch by its code — **a duplicate of P1.7**, same search on the same deposits endpoint. P1.7 absorbs the half it lacked (the code column on 3·3b)

88 → 91 tasks. Seven added (P2.18–P2.24), four removed.

CORRECTIONS FOUND WHILE DOING IT — RECORDED, NOT QUIETLY FIXED

WHAT WAS WRONG	WHAT IT IS NOW
1·4 Accruals said two engines did not exist	All four have preview and post routes (`accruals_api.py:585-597`); no screen calls any of them. Today the view lists pending reversals and runs the sweep
1·4 promised every entry would reverse	**The most serious item here.** Depreciation and the allowance post as standard entries and **must never reverse** — a reversed depreciation entry leaves the plant at purchase cost forever, $40,000 on one truck. 1·4 now shows *Type* and *Reverses*; the tab-1 act table and P3.1 carry the rule
Two findings never landed	3·4's multi-line Resolve and 4·1's missing tax-payments list were in the plan and in chat, **never on the screens**. The insert script skipped them as duplicates because all three

Two findings never landed	5·4's multi-line Resolve and 4·1's missing-tax-payments list were in the plan and in chat, **never on the screens**. The insert script skipped them as duplicates because all three findings began with the same 40 characters. Restored; the guard now keys on a unique phrase
Renumbered task ids never reached the screens	When eleven duplicate ids were renumbered on 2026-09-23, the plan moved and the screens did not — 6·1 cited P5.4, which is now the partner feed's *Acknowledgement*. 3·2, 5·3a and 6·1 fixed; 1·4 cited P3.2 and P3.3, which are *Reopen a period* and *Refund detail*
Four Payments tasks unflagged	P1.3, P1.4, P1.5 and P1.7 touch Payments code and were missed by the ownership pass. Twelve tasks now carry an owner flag
The KPI strip	The real tiles are *Billed this month* · *Collected* · *Collection %* · *Still owed* — the board drew *Written off* in place of *Collection %*
Made-up references	The board used `CHQ-4471` and `ACC-10428`. The product's refund and account numbers are `RFD-2609-0004` and `AC-2026-00003`; tab 2 uses those formats throughout
6·1 still called "The board"	In the register, two days after it was renamed *Close the month*

Two more were mine within this pass and caught before publishing: the renumbering ran after the new door text and turned *2·5 Refunds* into *2·5d* on three pop-ups; and three new task rows held markup inside an attribute, which broke it at the first quote mark.

⚠ **The pattern across all of them is one this document keeps meeting:** a change landed in one place and its consequences did not reach the places that depend on it — a task renumbered in the plan but not in the screens, a screen renamed in its drawing but not in the register, a finding added to the plan but not to its screen. **A new check now fails any task id a screen cites that the plan does not contain.** It cannot catch a real id cited for the wrong purpose — that one still needs a reader.

Everything that was written beside a screen but never drawn — fixed 2026-09-24

WHY THIS PASS EXISTS

Asked to list every requested change and whether it reached the document, a check against the published page found **31 in, 6 partly in, 2 not in**. On four screens the finding and its task had been written **next to** the drawing while **the drawing still showed today's screen** — and the drawing is the specification (B14 rule 42), so a lane building from it would have built the old screen. This pass fixes all eight.

WHAT WAS MISSING	WHAT THE DOCUMENT HAS NOW
3·2 no way to act on a line	Row actions drawn on every payment — 👁 open · ✎ correct the reference · … with ↺ reverse — plus a *Live payments* tile that says when a locked batch stops summing
3·4 one cause per difference	A pop-up with one row per cause, *+ Add explanation*, and a residual that must reach $0.00 before *Record ✓*
4·1 tax payments not listed	A *Payments made* table under *By authority* — owed and paid on one screen
5·3a / 5·1 no upload	An upload in the capture pop-up, and — as the PO asked for the other end too — 👁 *View document* on every register row that has one
The text cut	Finished. 4·2's 27-word sentence is now "*Type what you actually paid.*"; 3·3a, 6·1 and 6·2 are inside the cap
Filter and component tables	Added to the seven list screens in tabs 3–5 that lacked them — every parameter read from its handler, and *to verify* where it was not
The capability map	The PO's own map, carried over with today's screen numbers and the task that closes each row — §06
The Accounts-sample styling	Its patterns are now kit components — kicker line, section rule, label/value rows, status chips, service badges, amber callout, drop zone — used on 2·5d, 3·2 and 2·1

TAB 2 IS FIVE VIEWS, NOT EIGHT — RULED THE SAME DAY

`RULED` The PO removed **Account ledger** ("*we dont need 2·6 … please remove it*") and then **By premises** and **Statements**: "*we can cover this and premises in report any way*". All three stay Reports screens at their own addresses; a Finance view of a Reports screen would only duplicate it. **Tab 2 is 2·1 Services · 2·2 Aging · 2·3 Tie-out · 2·4 Write-offs · 2·5 Refunds**, plus the four pop-ups and 2·5d. **Document 35 → 32 screens. P2.22 retired, so 91 → 90 tasks.** P2.19 was also narrowed: its paging duplicated P1.8, which already pages both queues.

TWO TRAPS CAUGHT ON THE WAY, ONE OF THEM BY A GUARD

A class-name collision. The new kicker style was first called `.kick` — a name the masthead already uses. The "already added?" check saw the old rule and skipped the whole insertion; had it not, the new rule would have restyled the masthead. The components now take unused names, and the insertion asserts every name is free before claiming it. **A guard that worked:** the tab-2 generator

refused to write when its screen count changed from the one it expected — exactly what it is for.

CLOSE & EXPORT BECOMES TWO VIEWS — RULED 2026-09-24

RULED PO: "*this tab is for close and export — there is no functionality to export … can we create tab*". **6·1 Close · 6·2 Export**, the same view pattern as tabs 1 and 2. Sending leaves the close screen for 6·2, which also gains what the server had and no screen showed: the four targets, Summary or Weekly, the hand-over history with drill-back, the drift read the PO made the condition of `OQ-203`, and three dimension extracts. **Old → new:** 6·2 Close the month → **6·1a** · 6·4 Reopen → **6·1b** · 6·3 Hand over → **6·2a**. **33 screens, 91 tasks** — P4.12 builds the Export view.

⚠ **Two values the previous 6·1 had wrong, found by reading the code for this:** there are **four** targets, not three (`EXPORT_TARGETS` includes **IIF**), and granularity is **Summary · Weekly** (`BR-LED-017`), not *Summary · Detail*. Two stale "*1·1 trial balance*" references — the trial balance has been 1·2 since tab 1 split — were fixed in the same sweep.

FOUR PAGES WERE DRAWN AS POP-UPS, AND THE CHECK LET THEM THROUGH

Looking at the redrawn screens found **3·3b, 3·5, 5·1 and 5·2** drawn without the side menu and top bar — framed as pop-ups, though each is a page with its own address. **The structural check passed all four**, because it treated any pop-up-style frame as a pop-up and skipped it. It now decides by **address**: a page drawn as a pop-up is itself a fault. **Proved red on the real page before the fix** — it named all four, and that 3·3b, 3·5 and 5·2 also lacked the tab strip.

And 3·3b's title was empty because of this revision's own text cut. The product's title is the ten-word sentence "*Import a bank statement and match it to your deposits*"; the cut deleted it where it should have shortened it. It reads *Import a statement* now. **A heading is capped at three words, not removed** — rule 33 removes *rationale*, never a label.

Wrong facts in the drawings, found by reading the product's code — fixed 2026-09-24

A TABLE SPILLED OUT OF ITS PANEL, AND NO CHECK COULD SEE IT

The 5·1 table ran **25px past its panel**. The layout check measures only the page, and the structure check compares a table with the whole 1440px frame — so a spill *inside* a drawing passed both. A **containment check** now compares every element with the box it sits in, across all 33 drawings, and prints what it skips. **Proved red on 5·1 before the fix.**

WHERE	WAS	IS NOW, AND THE SOURCE
5·1 status	*Settled* as a status	**Captured · Voided** only — a paid status is refused by rule (`BR-LED-028`). Payment has its own column, **Paid on**: *In the books* with the date, or *Recorded*
5·1 view document	a text button in its own column	👁 **in the row actions** — rule 15: icons, never text links. The column also caused the spill
5·1 filters	period = invoice date; volume filter missing	The period each invoice **lands in**; `f.volume_captured` added — **77 → 78 criteria**
Account numbers	5100 Purchased water · 5000 / 6100 power · 6400 / 6410 fees · 2200 Sales tax payable	The seeded chart: **5100 Purchased power · 5110 Purchased water · 6800 Bank fees** (pre-selected by designation). **No 2200 is seeded** — each utility mints one per tax authority (`chart.py:202`)
3·4 reasons	*Processor fee* · *Chargeback*	The seeded list: **Card processor fee · Returned item** — it has no *Chargeback*
5·3a form	no reference; Unit drawn absent; upload label outside the form	**Invoice reference** added; Unit is **free text by design**; the upload sits in the form. Buttons say *Capture invoice*, as the product does
5·1 callouts	two callouts numbered 5	1–6, one each

Export: one filter, one table, two ways out — ruled 2026-09-24

WHAT THE PO ASKED FOR ON 6·2

RULED "*this filter should be expanded — I need it by period start date and end date … show the table below it … then allow to send or download it in different file formats. We need a column selector — all selected columns go in the downloaded file.*" The extracts: "*can't you merge it with the filter itself*". The history: "*this is good*".

WAS	NOW

Month	**Period start · Period end** — any dates. The server already took them; only the screen offered one month
Three extract buttons	**Group by** Account · Service · Premises · Tax authority, in the filter
A three-column preview	A table with search, ↓ **CSV or PDF** and ▦ **Columns** — the download carries the ticked columns (P4.13)
Send ✓	↗ **Send**, the four checks shown before you press it; 6·2a names where the file goes
Hand-over history	Kept, as the PO said

WHY DOWNLOAD AND SEND STAY APART

A QuickBooks, Xero or IIF file taken without a record could be imported twice — the double booking `BR-LED-004` exists to stop. So ↓ gives the table (CSV or PDF, any dates, nothing recorded), and the four accounting formats leave only through ↗ Send — once — and come back from the history. **No Excel**: IS-11 rules CSV always, PDF where formatted, no XLSX.

FOUND ON THE WAY, AND FIXED

Every table toolbar had its last two controls reversed — 13 drew ▦ before download, and all 33 download marks were ↓. The product's order is search · filters · ↓ · ▦ (CR-045, pinned by a test) and its glyph is ↓ (B14 rule 4). A new check holds both. **6·2's filter table had two wrong rows:** the preview takes a start and end date, not a month, and the history's target filter *is* served. **6·2a still said it opened from 6·1. 1·2 and 1·3 drew 1100 as Cash and a seeded 2200** — the chart has Cash at 1000 and 1100 as Investments, and seeds no 2200; found when the account check learned to read a code and its name in separate cells. **91 → 92 tasks** — P4.13 carries the columns into the file.

SECTION 01

Standards

A violation is only a violation against a stated authority, so the authority comes first. Then the design system, measured rather than assumed. Then the register that prevents the most damage: where every control's values come from.

A · The rule set, in precedence order

#	AUTHORITY	STANDING HERE
1	`04-docs/02-decisions/14-ui-conventions.md` — **B14, 34 rules**, numbered 1–21 and 31–43	Wins over everything below. **Counted from B14's own status table**, not from a pointer — CLAUDE.md warns that number has been wrong six times
2	`05-prototype/r10-2026-08-16/UI-RULES.md` — 20 rules; Tables are 12–17	B14's *source*, not its authority
3	`NFR-3` · `NFR-5`	Universal server pagination · no hard caps. Load-bearing: they are what keeps T4/T3 reachable
4	CLAUDE.md hard rules 2 & 4	Money 4dp stored / 2dp displayed · financial immutability. **An edit control on an issued invoice, a payment or a posted entry is a finding, not a feature**

GREEN PROVES NOTHING HERE

No gate in this repository reads rendered output. Most of B14 is RATIFIED and UNENFORCED, so a green suite is evidence of nothing. Four rules were measurable on the running page and all four pass — they are marked MEASURED below. Everything else was read.

B · The design system, extracted rather than invented

FAMILY	PINNED LITERAL	SOURCE
Colour	bg `#f3f2f2` · surface `#eae9e9` · ink `#201e1d` · accent `#ec3013` · neutral ramp 100–900 · success `#1f9d57` / warning `#e08a1e` / danger `#e0341a` / info	`tokens.css`

Type	page-head **32px** · h4 20 · h5 16 · body 15 · md 14 · base 13 · sm 12 · xs 11 · KPI metric **26** · table body **13.5px**, header **12px** at `.08em`	`tokens.css` · B14 rule 40 · MEASURED
Font	Archivo, heading weight **800**	`tokens.css` · MEASURED
Space	4 · 8 · 12 · 16 · 20 · 24 · 32, and nothing off the scale	`tokens.css`
Radius	`0px` **at every size** — square on purpose, do not "fix"	`tokens.css` · MEASURED
Shell	240px inverted-ink left rail, **13 items**, 56px topbar, accent active item	`AppShell.tsx` SHELL_NAV, counted in the running app · MEASURED

TWO TABLE COMPONENTS, AND ONLY ONE IS THE SHARED ONE

`DataTable` is the server-driven list table — toolbar, filter chips, a column chooser with reorder and reset persisted per user, an empty state and a pager. `PlainTable` is "*the small plain table inside a detail tab or a wizard preview… 51 sites*" and supplies none of them. That distinction is sanctioned. **What is not sanctioned is a transaction list built from `PlainTable`** — and that one decision, repeated sixteen times, is the root of most of §02's front-end work.

C · The dropdown source register

No drawing in this document authorises a literal array in code. Where a control has no source, that is the finding, not the design.

CLASS	MEANS
SERVED	An endpoint field — the list is whatever the tenant configured
SIGNED	A closed enum with a rule id. A tenant cannot add a value **and a lane must not either**
DERIVED	Computed from data already on screen; no list to hardcode
NOT SERVED	No endpoint exists — **the named task creates one**, it does not seed a literal

CONTROL	SCREEN	CLASS	SOURCE	TASK
GL account picker	1·5a	SERVED	`GET /ledger/accounts → ledger_accounts`	—
Journal entry type	1·5a	SIGNED	`JE_TYPES`, CHECK `ck_journal_entries_status`	—
Journal state filter	1·1	SIGNED	`JE_STATUSES` — draft · posted	—
Trial-balance type chips	1·1	SIGNED	`LEDGER_ACCOUNT_TYPES`	—
Designation picker	1·2	SIGNED	CHECK constraint, 18 values. ⚠ **a deliberate hole:** extending it is a migration, not a setting	—
Chart category	1·2	SIGNED	`CHART_CATEGORIES`, validated in Python — "*a category is a GROUPING and not a posting pointer*"	—
Deposit status chips	3·1	NOT SERVED	**Literal array** — `DepositsPage.tsx:103-106`	TASK P1.4
Deposit method filter	3·1	SERVED	`f.method` accepted by the endpoint — **no control renders it**	TASK P1.4
Difference reason	3·4	SERVED	`bank_rec_difference_reasons` — **a real table**, the pattern to copy	—
Refund status chips	2·3	NOT SERVED	**Literal array** — `RefundsPage.tsx:198`	TASK P1.5
Tax remitting basis	4·2	SERVED	`tenants.tax_remitting_basis_by_authority`, set on Settings → Configure	—
Export target	6·2	NOT SERVED	**Typed out twice.** `endpoints.ts:17276` hand-copies `ledger.models.EXPORT_TARGETS`, with a comment saying "*verbatim*"	TASK P1.6
Cost service picker	5·3a	SERVED	the tenant's enabled services	—
Cost category picker	5·3a	SIGNED	`BR-LED-028` capital categories, refused options disabled	—

Purchase unit	5·3a	NOT SERVED	**No picker at all.** The vocabulary is OPEN `OQ-201` — correctly stubbed	—
Currency	everywhere	NOT SERVED	**No source at all.** 27 free-text `CharField(3)` columns across 7 modules; no lookup table; nothing links an entry to the company setting	TASK P0.6

TWO TRAPS THIS REGISTER IS BUILT TO CATCH

(1) A SIGNED enum with a deliberate hole. The designation list looks incomplete and is not — extending it is a migration because Django inlines the values into the CHECK constraint. "Completing" it in a picker breaks the rule.

(2) A literal today that should not be. Three of them, above. All three fail silently in the same way: the row exists, the API serves it, `tsc` passes, every test passes, and the chip simply never appears — the eleventh-service failure CLAUDE.md opens with.

TODAY I found those three **by accident** while grepping for something else. This register is what finds them on purpose.

SECTION 02

Development plan

Six increments, each shippable on its own. **No dates** — order is set by what depends on what, and by putting the cheapest value first. Every row in a task table is a TASK. Everything else in this document is a GAP and is **not scheduled**.

Phase 0 · Foundations

Ships	Stamps, foreign keys, a currency table. No screen.
A clerk can newly	**Nothing — and that is said out loud.** This is the only phase with no user value, so it is kept as short as it can be. The argument for going first is not tidiness: **a posted journal line is immutable**, so every day this waits, more rows post that can never be backfilled.
Task mix	DB 5 · Backend 2 · Settings 1 · Test 2 · E2E 1 · Other 1 = **12**
Done when	A newly posted entry carries a `JE-` number, a customer, and a tax authority where one applies; and a bad GL pointer is refused by the database rather than discovered on a report.

TASK	KIND	WHAT	WHAT IT INVOLVES	REFERENCE	OUTCOME
P0.1	DB	Customer on the journal line	Add `customer_id` to `journal_lines`, `PROTECT`, nullable for history. Partition-aware across `journal_lines_2026/27`	`US-LED-002` TC-4	A line posted today names the customer who held the account then, and a later transfer does not change it
P0.2	DB	Primary holder on the customer link	`is_primary` on `customer_accounts`; backfill where a link is the only live one	`customers/models.py:1152`	An account with two live customers names which one is billed; P0.1 has an answer to stamp
P0.3	Backend	**Three stamps, one dataclass**	`PostRequest` gains `number` and `customer_id`; `post()` calls the `BR-TEN-035` allocator and stamps `tax_authority_id` from the source line. ⚠ Settle first: does the `JE-` series cover system-generated entries or only manual? And "gapless" (decision log) vs "gaps ok, never reused" (`BR-TEN-035`) — **two documents disagree**	`ledger/posting.py` · `BR-LED-021`	168 of 168 entries numbered on a fresh seed, not 0 of 168; a tax line names its authority
P0.4	DB	Six GL pointers become foreign keys	`billing.gl_account_id` · `rating.gl_account_id` ×2 · `rating.liability_gl_account_id` · `tenancy.default_gl_account_id` · `tenancy.clearing_gl_account_id` → `ledger_accounts`. Prove no orphans first	the six `# DEFERRED FK` comments	A stale or typo'd GL id is refused at write, not found on a report
P0.5	DB	Four remaining deferred FKs	`customers.premises_id` → metering · `rating.account_id` → customers · `rating.account_type_id` → customers · `rating.verified_by_id` → identity	`BR-CUS-1010`	Ten deferred FKs become zero
P0.6	DB	**A currency table**	Create the lookup; FK from `tenants.currency`; **drop** `default="USD"` on `journal_entries.currency` and make it NOT NULL. ⚠ 27 free-text columns exist — this	`OQ-031` ruling: enums → lookup tables	The database refuses an unstamped currency, as `posting.py` already does in Python

			phase converts the two on the money path, not all 27		
P0.7	Backend	Currency validated at post	Entry currency must equal the tenant's configured currency. **Stamped, not joined** — hard rule 4 means a posted entry keeps its currency for ever, and `BR-TEN-021` makes a currency change forward-only	`posting.py:441`	A CAD tenant's books cannot quietly become USD
P0.8	Settings	Company currency reads the lookup	Settings → Company currency becomes a served picker, not free text	§05 Settings table	A tenant cannot type `US$`
P0.9	Test	**PostgreSQL only**	FK violation refused; currency mismatch refused; a failed statement aborts the transaction. ⚠ `config/settings/test.py` hardcodes in-memory SQLite, which **has no aborted-transaction state** — these cannot be proved on the default suite	hard-won law 14	Each constraint is shown refusing on a real database before it is trusted
P0.10	Test	Numbering behaviour	A draft has no number; a post allocates one; a rolled-back post leaves a gap and the gap is tolerated	`BR-TEN-035`	The contradiction in P0.3 is settled by an assertion, not a comment
P0.11	E2E	No new journey	`J19-month-end-close` gains one assertion: the trial balance's entries carry numbers	`08-qa/01-e2e/journeys/J19-month-end-close.md`	Stated honestly: no new journey is needed for a stamping change
P0.12	Other · billing · rating · tenancy	Their halves of P0.4	Three modules own the columns; each needs its own migration in the same release	B2 §2 reviewed FK set	No module ships a dangling pointer after this phase
P0.13	Settings	The JE series format the PO ruled	`JE-{MM}{DD}{YY}-{0000}` replaces the seeded `JE-{000000}` — 22 characters against a 24 limit, tokens all supported. Seed default **and** the S8 Numbering tab	`manual_je.py:75`	**PO 2026-09-23, verbatim *"you need je-MMDDYY- nnnn"*.** Free today — **0 of 168 entries carry a number**, so there is nothing to migrate. After P0.3 it becomes a migration
P0.14	Backend	One spelling for the premises filter	The journal reads `f.premises`, the trial balance reads `premises`. Same filter, same tab, two spellings. Accept both, serve `f.premises`, **deprecate the bare one rather than break a live caller**	`ledger/api.py:482` vs `:395`	Found while deriving the filter tables. Every other filter in Finance is `f.`-prefixed
P0.15	Other	Allocate the two question ids properly	`OQ-LED-1512` (attachment retention) and `OQ-LED-1513` (must a revenue line be tagged?) are **PROPOSED in this document and not allocated**. Run `allocate.py` against the remote and renumber if taken	`10-scripts/allocate.py`	**Law 11: an allocated id is a blob too.** A hand-picked number reads only this checkout and two lanes take the same one

Phase 1 · Lists that scale

Ships	The ten transaction lists move onto the shared `DataTable`.
A clerk can newly	**Find a deposit among a thousand**, and **send a colleague a filtered journal as a link**. Today neither is possible: one Finance list of seventeen has a pager, and every filter is browser state that dies on refresh.
Task mix	Front end 8 · Backend 2 · Test 1 · E2E 1 · Settings 1 · Other 1 = **14**
Done when	Every list that grows with transaction volume pages in SQL and carries its filters in the URL.

TASK	KIND	WHAT	WHAT IT INVOLVES	REFERENCE	OUTCOME
P1.1	Front end	Journal list onto DataTable	Search, five filter chips, column chooser, pager. **The server already accepts all of it** and the front end sends none of it	`ledger/api.py:478-485`	A clerk reaches entry 51. Today they cannot
P1.2	Front end **PAYMENTS**	Deposits list onto DataTable	Same kit; status and method chips	`DepositsPage.tsx`	UI-RULES 12·13·14·16 pass on 3·1
P1.3	Front end **PAYMENTS**	Refunds queue onto DataTable	⚠ The code argues against paging a queue — "*a page of an approval queue is a queue that lies about what is pending*". Real argument; NFR-3 carves out no exception. **Settle before**	`payments/selectors.py:1569`	Either a pager, or a recorded decision that this one list is exempt

			building		
P1.4	Front end **PAYMENTS**	Deposit chips from the server	Delete the literal array; add the method chip the endpoint already accepts	`DepositsPage.tsx:103-106`	A status added server-side appears without a front-end change
P1.5	Front end **PAYMENTS**	Refund chips from the server	Same shape	`RefundsPage.tsx:198`	As P1.4
P1.6	Front end	Export targets served once	Delete the hand-copy; serve the list or generate it from the schema	`endpoints.ts:17276`	One list, one fact
P1.7	Backend **PAYMENTS**	`q` on the deposits endpoint	Batch code, bank reference, payment confirmation. Copy the cost register's shape — minimum-length guard, term in the URL	`cost_capture.py:2090`	Search reaches the server on 3·1 · **and the batch code shows as a column on 3·3b's match table**, so a clerk holding a code off a deposit slip can find it on either screen — **this absorbs P3.13**, which was the same search
P1.8	Backend	**SQL pagination on eleven lists**	Journal, deposit batches, batch members, resolution trail, reconciling items, refunds, write-offs, cost register, drill-back, export history, and **6·2's by-premises grouping** — up to 40,000 rows at T5. ⚠ The journal pages **in Python** today — `paginate_queryset(list(...))` — which is bounded on the wire and unbounded in the query	`ledger/api.py:487` · NFR-3	A 200,000-row month does not materialise in memory
P1.9	Front end	Filter state into the URL	Including the GL sub-view switch, which today **wipes `q` , `sort` , `page` and every `f.*`**	`GeneralLedgerPage.tsx:859` · `CR-060` C14	The round-trip the change request already promises actually works
P1.10	Test	Proved at volume	Seed 20,000 journal lines; assert the query plan bounds and the page count	NFR-3 · NFR-5	Correct at T5, not just at T7
P1.11	E2E	`J2-read-to-cash`	Add a step: filter, copy the URL, open it in a new context, same rows	`J2-read-to-cash.md`	Shareability is asserted, not assumed
P1.12	Settings	**None — and that is a finding**	Page size is a per-user preference the shared `DataTable` already persists server-side. No tenant setting is needed. The full Settings picture is in §05 anyway	`DataTable.tsx:230`	Nobody adds a setting that already exists one layer down
P1.14	Front end **PAYMENTS**	**The bank import onto the shared wizard frame**	It is the **only genuine multi-step flow in Finance** and it is hand-rolled: two `<h3>` headings on one page, step 2 a conditional panel below step 1. **No Finance or Payments file imports `WizardFrame`** , while twenty files in tenancy, assets, metering, rating and customers do. ⚠ Moving it means a numbered rail, a forward fence, a working state and **one transactional commit** — not a cosmetic change	`BankImportWizard.tsx` · `BR-SHL-018` · `BR-SHL-019`	A clerk can step back to fix the account without losing the upload, and a half-finished import cannot be left half-committed
P1.15	Front end	Split the strip into four views	`GL_VIEWS` → `["journal","balance","chart","accruals"]` . **`balance` keeps its value** so bookmarks land; `journal` is new and becomes the landing view. Three catalogue keys change and one is added, across **eleven catalogues** through `i18n_fanin.py`	`GeneralLedgerPage.tsx:637`	One word per view. **A clerk cannot tell "Trial balance & journal" is two screens**
P1.16	Full stack	Five dimensions on a manual line	`ManualJournalLine` is "*an account, a side, an amount*". `journal_lines` carries **service · account · premises · unit · tax_authority** and a hand-typed line writes NULL on all five. **Optional per line**, collapsed behind `+ Tag this line` , so a pure reclass is untouched. Backend first	`manual_je.py:205-218`	**661 of 663 live lines are tagged; a manual one is not** — so the property filter, revenue-by-service and by-premises all drop it, with no error and no absent-figure note
P1.17	Front end	RowActions gains a real overflow	It is `<span className="inline-flex…">{children}</span>` and **nothing else** — no cap, no … menu. UI-RULES 15 wants **two visible plus overflow** and it is entirely the caller's job today	`components/RowActions.tsx:13-18`	**Blocks P1.1, P2.13 and P3.10**, all of which are drawn as 👁 ⟲ … . Build it once, not four times
P1.18	Test	Entry no. sorts by date, not by text	`MMDDYY` does not sort chronologically as a string — `JE-011526` precedes `JE-020325` by eleven months. Order by `entry_date` then sequence and **pin it with a test**	`reporting.py:2731` already orders correctly	A later lane "simplifying" this to a string sort would silently reorder the journal

P1.19	Other	Forty-seven strings, eleven catalogues	The rule-33 cut is not a drawing change — **every cut string is a key**. 47 strings shortened or deleted, plus the view renames, 6·1's three panel renames and the band headings. Retire the dead keys through `i18n_fanin.py`	`i18n/locales/*.json` ×11	**A deleted English string with ten live translations is a catalogue that lies.** The fan-in tool owns this file set — never hand-edit
P1.21	Front end	New entries — the batch screen	Composer in the **Entry** panel, `+ Add line` pushes a composed entry into the **Entries** table, summary reads the batch. **Debit account + credit account + one amount**, so every row balances by construction rather than by validation. `+ Another line` for a 3+ sided entry. Replaces the `FormModal` at `/finance/journal-entries/new` ; a draft reopens at `/{id}/edit`	`ledger/JournalEntryModal.tsx`	**PO 2026-09-23: the screen takes "*1 or more entries*"**, each row its own journal entry
P1.22	Backend	Posting a batch	**Blocked on `OQ-LED-1515` and must not start first** — the answer decides whether this is a new all-or-nothing endpoint or a loop in the client over the endpoint that exists. `ManualPostResult` carries ONE `entry_id` and ONE `number`	`ledger/manual_je.py:223-249`	Three entries is three calls. **If entry 3 of 5 fails, 1 and 2 are already in the books** and hard rule 4 means they never come back out
P1.13	Other · payments	Three of the ten lists are theirs	Deposits, refunds and the write-off queue are payments-owned; Finance draws them	module map	One team does not silently do another's migration

Phase 2 · Tell the truth

Ships	Seven refusal sentences that are no longer true, three dead catalogue strings, and one clipped figure.
A clerk can newly	**See the whole receivable.** Today the Finance headline reads `$23,766…` and loses its cents. And **filter the journal to one property** — a filter that has been built and applied in SQL since 2026-08-16 while the screen says it does not exist.
Task mix	Front end 3 · Copy 2 · Test 2 · E2E 1 · Settings 1 · Other 1 = **10**
Done when	No screen in Finance refuses over a gap that has closed, and no money figure is elided.

TASK	KIND	WHAT	WHAT IT INVOLVES	REFERENCE	OUTCOME
P2.1	Front end	**The KPI tile stops eating cents**	Remove `overflow:hidden; text-overflow:ellipsis` from `.kpi .v` ; widen the grid track from `minmax(180px,1fr)` to `240px` . Measured: the value needs **159px** and has **146px**	`KPIStrip.css:40,91` · hard rule 2	`$23,766.56` renders in full, and so does `$1,234,567.89`
P2.2	Test	**A guard worth more than the fix**	Assert no element carrying a money value has `scrollWidth > clientWidth` . ⚠ A string assertion **cannot** see this class of bug — the DOM text was always correct	hard-won law 6	Every KPI tile in the product is covered, not just this one
P2.3	Front end	Premises filter control	Add the chip. **No backend work** — `api.py:482` accepts `f.premises` and `reporting.py:730` filters on it in SQL	`CR-060` C24	A clerk filters the journal to one property
P2.4	Front end	**Delete** the "no report filters" panel	Removed, not reworded — B14 rule 33 puts rationale at zero words on screen. The honest build is the working control	`GeneralLedgerPage.tsx:925`	The screen stops stating something false
P2.5	Copy	Six stale refusal sentences	The S74 download ("*where this view lives has not been ruled*" — it was, 2026-09-03) · the period pack · the trial-balance download · the remittance report · the three report PDFs. ⚠ Four of the controls stay **disabled** and correctly so — it is the stated *cause* that is false, not the state	§04 absence rows	No Finance screen blames a service that is running
P2.14	Front end	AbsentFigure — a stated absence with its reason	Referenced on 1·4, 2·2, 2·3 and 6·1 and **it does not exist**. `BR-SHL-025` requires the reason to travel with the missing figure; today each site improvises	no `components/AbsentFigure.tsx`	**A blank cell and a zero say different things and neither says why.** This document named the component before checking it was real
P2.15	Front end	BandHeading and CountBadge	Both are new. **BandHeading** carries 6·1's three steps — the bands it was first drawn for on the old Revenue & AR board were retired by P2.18. **CountBadge** carries the pending counts in the	neither exists	Rule 43 was ratified after "use it" turned out to mean 120 lines

			Revenue & AR view labels and 1·5a's *Entries · 3*. **B14 rule 43**: a piece of UI used twice is a component, never inline markup in two pages		inline in a 4,186-line page
P2.16	Other	Two stale exception comments	`ledger/api.py:559` exception **(3)** says the trial-balance premises filter is *"NOT offered … premises_id is NULL on every row"* — **the param is passed at `:395` and 661 of 663 lines carry a premises**. Exception **(5)** says *"a journal entry has no number and KNOWN_SERIES has no series for one"* — **the series exists**	`ledger/api.py:559`	**Hard-won law 1.** Both were true when written; a stale caveat is read as current and sends the next lane the wrong way
P2.17	Front end	The product onto the token scale	**Three verified values first:** `DataTable.css:243` `padding: 6px 10px` — **the shared table, so every list in Finance inherits it** — and `finance.css:58` `padding: 5px 0` plus `:306` `gap: 14px`. **Then run `tokens_check.py` over `07-frontend/src` as an AUDIT and triage per file. ⚠ Not a bulk snap.** `AppShell.css` alone holds 25 of the 65 and its header says the widths are a prototype port under `BR-SHL-001` — **changing those is a rule change, not housekeeping**. The three `font-size: 0px` hits are **generated Tailwind utilities and not a defect**	`tokens_check.py` · `styles/tokens.css`	**65 off-scale occurrences in 11 files.** The blueprint now holds zero, so without this the drawings and the product disagree — and the drawings would be the ones that are right
P2.18	Front end	The Revenue & AR view strip	Five views — Services · Aging · Tie-out · Write-offs · Refunds — switched by `?view=`. The KPI strip sits above the strip on every view; pending counts ride in the labels. `/finance/revenue` opens on Services; `/finance/refunds` **redirects** to `?view=refunds`, never 404s. **Replaces the board and its bands (P2.11)**, and **resolves P2.12** — the Refunds view carries Approve itself	`ledger/RevenueArPage.tsx` · `financeShared.tsx`	**PO 2026-09-24** — a tab per category, each with table, search, filter and column chooser
P2.19	Full stack **PAYMENTS**	Paging, search and a pending count on both queues	`GET /write-offs` and `GET /refunds` each read **exactly one filter, `f.status`**, and are unpaginated on purpose. Add `q`, the missing filters (reason · date · requested by; method · date) and **a pending count returned beside the page**. **The server paging itself lands with P1.8**, which already covers both lists — this task does not repeat it — the count is what answers the code's own objection that *"a page of an approval queue lies about what is pending"*. **Includes the two views' toolbars**, which were P2.13	`payments/api.py:796` · `refunds_api.py:260`	At 40,000 accounts an unpaginated queue is the breach the project rules name — the same 3,000-connection excuse
P2.20	Front end	Services and Tie-out on the shared table	Both from `GET /finance/revenue-ar`. **Bounded** — at most eleven services and three subledgers — so paging and search stay in the browser	`ledger/RevenueArPage.tsx`	Today both are panels with no toolbar
P2.21	Full stack **REPORTS**	The Aging view	A view over the `ar_aging` report, **and the report's paging and filters** — as-of, band, service, property and search are **none of them supported today**. One row per debtor, so it must page	`reports/catalog.py` `KEY_AR_AGING` · `dashboard.py:673`	**There is no aging screen anywhere in the product** — only the report and a Dashboard panel
P2.23	Test	Every old door lands on the right view	The Dashboard aging panel, Outstandings' write-off result, the Payments bulk action's refund link and `/finance/refunds` bookmarks each assert the new view **by address and by screen id**	`RefundsQueuePanel.tsx:179` · `paymentsBulk.tsx:858`	**Hard-won law 2** — a restructure that moves doors is exactly how a door ends up pointing at nothing, with every test green
P2.24	Other	Five view names, eleven catalogues	`finance.revenue.view.*` × 5 through `i18n_fanin.py`; retire the board's band headings	`i18n/locales/*.json` ×11	A key added in English only is a catalogue that lies
P2.6	Copy	Three dead catalogue strings	Retire `payments.bulk.reapplyUnavailable` and `…receiptsUnavailable` through `i18n_fanin.py`; **add** the missing `finance.cost.recordOnAsset`, which renders from a default and shows English in ten locales	`bulkActions.test.tsx:342`	Eleven catalogues carry no orphan and no hole
P2.7	Test	Hold-expiry tests	For every remaining hold, assert the condition it names is still true. **Write the test in the same commit as the hold** — a hold with no such test is a promise nobody is keeping	hard-won law 2	The next rotting hold goes red instead of shipping
P2.8	E2E	No new journey	Stated honestly. Copy changes and a CSS fix need a rendered-output check, not a journey	—	Nobody writes a journey to prove a sentence
P2.9	Settings	**None**	Nothing here is tenant-configurable. Recorded so the absence is visible	§05	—
P2.10	Other · reports	Three of the six sentences	S59, S60 and the customer statement share one download component and one sentence	`reportsShared.tsx:143`	One change, three screens

Phase 3 · The missing doors

Ships	Four screens over backends that are already finished, tested and unreachable.
A clerk can newly	**Post the accruals the product already computes** — unbilled revenue, unpaid costs, depreciation, bad-debt allowance. **Reopen a month closed in error** without a shell. **See whose refund they are approving.**
Task mix	Front end 4 · Test 2 · E2E 1 · Settings 1 · Other 1 = **9**
Done when	The 22 orphan endpoints are 10, and none of the remainder is a Finance capability.

TASK	KIND	WHAT	WHAT IT INVOLVES	REFERENCE	OUTCOME
P3.1	Front end	**Accruals workbench** (screen 1·4)	Eight live routes over four engines — `unbilled_accrual` , `unpaid_cost_accrual` , `depreciation` , `allowance` , ~145 KB of tested Python with **zero front-end callers**. Preview then post, per engine. ⚠ **Two post `type=accrual` and reverse on day 1; depreciation and the allowance post `type=standard` and must never reverse** (`depreciation.py` , `allowance.py`)	`ledger/accruals_api.py`	The Accruals tab lists real accruals instead of an empty panel and a disabled button
P3.2	Front end	**Reopen a period** (screen 6·1b)	The endpoint and a typed client wrapper exist; no component calls either. System Admin only, reason required, audited	`endpoints.ts:17263`	A month closed by mistake is reopened from a screen
P3.3	Front end	**Refund detail** (screen 2·5d)	Four built reads with no caller: `GET /refunds/{id}` , `/refunds-and-credits` , `/accounts/{id}/credit-sources` , `/credit-path` . This is why the queue apologises for not naming the account	`payments/refunds_api.py`	An approver sees whose money they are giving back
P3.4	Front end	GL map save	`PUT /ledger/gl-mappings` shipped under `CR-060` C10/T11 and no screen puts to it; the panel sends the clerk to Settings → Services for half the map	`chart_api.py:443`	The whole map is saved on one screen
P3.5	Test	**Doors asserted by address**	For each new screen, assert the inbound link resolves to the address *and* the screen id it opens. ⚠ A hold-expiry test once asserted a screen was registered at one address while the router mounted it at another, and both opened onto "not built yet"	hard-won law 7	A route without a door cannot ship again
P3.10	Front end PAYMENTS	Row actions on the batch lines	👁 open the payment · ✎ correct the reference · … carrying ↺ reverse. **Two of the three already have endpoints**; `correct-reference` permits `confirmation` and `memo` on a banked payment, and only the tender is refused	`DepositBatchPage.tsx:163,250,343`	Three `PlainTable` s, **no row control anywhere**, on the screen where a wrong cheque number is found
P3.11	Front end PAYMENTS	Say when a locked batch no longer sums	A reversal inside a locked batch is lawful, so the frozen `recorded_total` can drift from the live payments. **Show the difference; change no rule**	`_refuse_if_batch_locked`	Today nothing surfaces it
P3.12	Front end PAYMENTS	Multi-line resolve	A repeating row (reason · GL · amount · note), `+ Add explanation` , and **a running residual that must reach $0.00** before Reconcile enables. **The service already takes a tuple and already checks the sum**	`reconciliation.py:2301` · `api.py:1135`	A $172.50 card shortfall is a $147.50 fee **and** a $25.00 chargeback. One forced reason files the chargeback as a fee and defeats CS-18
P3.6	Test	Accrual posting on PostgreSQL	The day-1 reversal pair, linked both ways, on a real database	`BR-LED-005` · LE-4	1210 nets 0.00, as the worked example says
P3.7	E2E	`J19-month-end-close`	Extend with reopen: close July, reopen it, confirm the trial balance is byte-identical before and after	`J19` step 7 invariance	Reopen is proved not to move a figure
P3.8	Settings	**None**	Who may reopen is a role gate, not a setting — and it is the same unruled question as P4.9	§05	—
P3.9	Other · payments	Refund detail is theirs	Finance draws it at a `/finance` address; payments owns the reads and the table	module map	The seam is declared rather than assumed

Phase 4 · Produce a file

Ships	The five refused downloads, a real export file, a deposit slip, and the filing receipt a confirmed rule already requires.
A clerk can newly	**Hand the accountant an actual file.** Today the handover is *recorded* and nothing is written or emailed. And **keep the filing receipt an auditor asks for** — `BR-LED-019` requires the attachment and the form refuses it by name.
Task mix	Backend 2 · DB 2 · Front end 2 · Test 1 · E2E 1 · Settings 1 · Other 1 = **10**
Done when	Every download control in Finance either works or is disabled for a reason that is true.

TASK	KIND	WHAT	WHAT IT INVOLVES	REFERENCE	OUTCOME
P4.1	Backend	Seven reports become export resources	Trial balance, remittance, account ledger, by-premises, customer statement — **plus the export preview and revenue by service**, so each of 6·2's four groupings downloads. The kernel exists and already registers four others at startup	`apps/reports/exports.py` · `apps/tenancy/exports.py`	The IS-11 pipeline serves Finance the way it already serves billing
P4.2	Backend	**The export batch writes a file**	CSV · IIF · QBO · Xero from the **stored payload**, so the file is hash-stable and a later ref edit cannot rewrite a sent batch. ⚠ Direct OAuth sync is explicitly v2 under `BR-LED-020` — **not this phase**	`ledger/exports.py:462`	A closed month leaves as a file a person can open
P4.3	DB	Receipt attachment on a remittance	A confirmed rule requires it and the form says *"a receipt file cannot be kept yet, so it is refused rather than accepted and lost"*	`BR-LED-019`	The auditor artefact is kept, not refused
P4.4	DB	`tax_remittances` — the filing register	Verified absent from the live schema. Past filings are visible only in the journal and the audit trail	B2 r13 #3	Screen 4·1 grows a filing-history table instead of an apology
P4.5	Front end	Enable the five downloads	And delete the sentences P2.5 corrected, now that they are not merely false but unnecessary	`reportsShared.tsx:143`	Five disabled buttons become five working ones
P4.11	Full stack	Payments made, on the taxes screen	A second panel: authority · period · paid on · amount · confirmation · the entry it posted. **Servable now from the journal by** `source_type=remittance` ; a table is only needed if a filing reference or document must be kept	`finance_api.py:1110` — **POST with no GET**	**You can record a tax payment and never see it again.** Owed and paid never appear together
P4.12	Front end	The Export view (6·2)	Its own view on Close & export: **Period start and end** — any dates, which the server already takes; **Group by** Account · Service · Premises · Tax authority, replacing the three extract buttons; the preview with search, ↓ **CSV or PDF** and ▦ **Columns**; ↗ **Send** with the four checks shown; the hand-over history with drill-back; the drift read. `RULED` PO 2026-09-24	`finance_api.py:771` · `:1230-1233` · `cost_api.py:808`	One filter, one table, two ways out
P4.13	Full stack · tenancy kernel	A download carries the chosen columns	IS-9 already says a download is *"current filters and visible columns"*; the kernel's body carries no column state, so every file holds the default set. Add `columns` to `POST /exports` , honour it in order, and have ▦ send its ticks. `RULED` PO: *"all selected columns will go in downloaded file"*. The kernel is tenancy's — P4.10	`apps/tenancy/exports.py` · `DownloadButton.tsx:11`	The file holds exactly the columns on screen
P4.6	Front end	Deposit slip	No deposits export resource is registered anywhere	`DepositsPage.tsx:353`	A bookkeeper gets the slip the screen has been promising
P4.7	Test	Hash-stable regeneration	Regenerate a sent batch's file and assert byte-identity; then edit a GL ref and assert the sent file **does not move**	`J19` step 7	Immutability is proved on the artefact, not just the row
P4.8	E2E	`J19` step 4 downloads	The journey already names the control and the endpoint; it gains the file	`J19-month-end-close.md`	The month-end walk ends with an artefact
P4.9	Settings	Export destination and retention	Where files land and how long they are kept. ⚠ Related but **separate**: who may close a period is `OPEN` and is a ruling, not a setting	§05	A tenant chooses a destination once
P4.10	Other · reports · tenancy	The kernel is theirs	Finance registers resources against a kernel tenancy owns and reports extends	`apps/reports/apps.py:26`	No second export mechanism is invented

Phase 5 · The partner contract

Ships	The partner ledger feed stops handing out UUIDs.
A clerk can newly	**Answer a partner's "which entry is this?"** — which today is unanswerable, because the feed identifies entries only by UUID and the Reference column carries an internal id rather than a printed document number. This phase is the weakest of the six on direct clerk value and that is said plainly; its user is the finance officer who currently fields reconciliation emails they cannot resolve.
Task mix	Backend 4 · Test 1 · E2E 1 · Settings 1 · Other 1 = **8**
Done when	A partner can tie every row we send back to a document the utility can find.

TASK	KIND	WHAT	WHAT IT INVOLVES	REFERENCE	OUTCOME
P5.1	Backend	`JE-` number in the feed	**Depends on P0.3.** The seam already cites the gap by defect id	`integrations/seams/ledger.py:20 · DEFLED-5540`	A partner quotes an entry number back and we can find it
P5.2	Backend	Tax authority in the feed	**Depends on P0.3.** Today a partner cannot split tax by authority from the ledger feed at all	`partner_api.partner_ledger`	The per-authority split survives the boundary
P5.3	Backend	Document number, not internal id	The Reference column carries the source document's **internal identifier**. Turning one into the other needs a read billing and payments do not publish	`finance.gap.reference_is_the_source_id…`	A partner matches a journal line to a bill number they hold
P5.9	Front end	Make 6·1 a close, not a board	Page head says the month and what is left. **Two numbered steps** — finish, then close — and every not-done checklist row gets a door to the screen that clears it. **Sending moved to its own view, 6·2 Export (P4.12).** *Handed over so far* becomes the Export view's *Handover history*	`CloseExportPage.tsx`	**Reported verbatim: "*as user i dont understand this screen at all*"**
P5.10	Backend	Amounts on the checklist	The checklist builder already computes each figure to decide done vs not done, **and throws it away**. Carry it to the row	`ledger/close.py`	*Accruals posted* ○ does not say whether that is five minutes or a day
P5.11	Spike	Attach a document to a purchase	**Spike first, not a column.** The PO asked for both ends: an **upload on 5·3a** and **👁 View document on each 5·1 row that has one**. `supplier_invoices` has 21 columns and no document; the platform has **no general attachment store at all**. Brings storage, tenant-scoped access, size and type limits, virus posture and retention	`ledger/models.py:896`	5·2 argues about a supplier's figure and **cannot open the document that settles it**
P5.4	Backend	Acknowledgement	*"Nothing reports back whether the accounting package accepted it."* A recorded ack per batch	`finance.export.noAckStatus`	A handover has an answer, not just a send
P5.5	Test	The feed reconciles	Pull a window; assert `total_records` matches the rows and `total_entries` matches the paged unit, and that **no page boundary falls inside an entry**	`seams/ledger.collectable`	A balanced pair is never split across a page
P5.6	E2E	A partner-pull journey	**New journey file** — none exists. Pull a month, reconcile every row to a document	—	The contract is walked, not assumed
P5.7	Settings	Partner key scopes	Which addresses a key may read. ⚠ `OPEN` **the sync contract**: the feed is window-only, so a reversal posted in October against a September entry lands in October's window — a partner who already pulled September never sees it. **Settle before a second partner is onboarded**	`partner_urls.py`	A partner's scope is configured, not coded
P5.8	Other · billing · payments	They publish the document numbers	P5.3 cannot be done inside Finance. This is the dependency that gates the phase	§05 closing line	The read exists, so the feed can carry it

Risks

WHAT WILL BITE	WHERE
A green suite proves nothing	No gate in this repo reads rendered output. The clipped `$23,766…` passed every string assertion because the DOM text was correct. P2.2 is the only task that changes this, and it covers one class of bug
The test database hides a whole class of failure	`config/settings/test.py` hardcodes in-memory SQLite, which has **no aborted-transaction state**. Every constraint in P0 must be proved on PostgreSQL or it is not proved (hard-won law 14)
A hold outlives its reason	Seven found in this audit, one of them refusing over a gap that closed five weeks earlier. P2.7 writes the expiry test; without it the eighth is already forming
Two answers to one question	The `JE-` series is "gapless" in the decision log and "gaps ok, never reused" in `BR-TEN-035`. P0.3 cannot be built until one wins
Immutability is not a convention	P0.1 and P0.3 add stamps to a table where **nothing can be backfilled**. Every day of delay is permanent unstamped history — this is the whole argument for Phase 0 going first
A migration with no way back	P0.4 adds six FK constraints across three modules. Prove no orphans *before*, and state the rollback *before* shipping — the change process names step 7 as the one usually skipped
A literal outlives its list	Three hardcoded arrays found. Each fails silently: the row exists, the API serves it, `tsc` passes, and the chip never appears. §01's register is the standing defence

SECTION 03

The tabs

Six tabs, thirty-five screens. Every flow is drawn in one order — ① **the list** · ② **one row opened** · ③ **what you can do to it** · ④ **what each of those changes, and where.** A screen appears **once**, in the step that owns it, and the drawing shows the **end state** with every difference from today marked. Where a step is empty for a tab, it says so.

> **THE RAIL IS DRAWN ONCE, AND EVERY LIST OBEYS ONE TOOLBAR**
>
> All drawings share one geometry from a single kit. **Every list-bearing screen carries the same six controls in the same order** — search · filter chips · `+ Filter ▾` · `▦ Columns` · `↓` download · sortable headers, with a pagination footer and an empty state (UI-RULES 12·13·14·16·17). Act drawings show the modal alone, because **a `FormModal` is what the code renders**.

Tab 1 · General ledger

Four views, one word each: Journal · Balance · Chart · Accruals. Switched by `?view=`.

> TASK **The strip was three views and is now four.** `GL_VIEWS` becomes `["journal","balance","chart","accruals"]`. Today `?view=balance` renders the trial balance **and** the journal on one page; they split. `balance` **keeps its meaning** so existing bookmarks and tickets still land on the trial balance, `journal` is a new value, and the unknown-view fallback already covers anything else without a 404.
>
> **Nothing is being reversed.** The three-view shape was never ruled — `CR-060` W6 merely describes "*Trial balance & journal*" as one screen. One-word names are a **deliberate constraint tighter than B14 rule 33**, which already permits three. *Balance* is the one awkward name: *trial balance* has no natural single word, and it is accepted on the grounds that it reads correctly standing next to *Journal*.
>
> ⚠ **ONE THING GOES TO THE PRODUCT OWNER.** The landing view moves to **Journal**. `CR-060` T10's recorded reason for `/finance` opening on the General ledger is "*opening on the trial balance means the first thing anyone sees is whether the month balances.*" Journal-first contradicts that reasoning, though not its letter. **Put in plain English:** "*Opening Finance today shows whether September balances. Should it instead show the last entries posted?*"

> GAP **Two prefixes for one fact, found while gathering the filter list.** The journal reads `f.premises` (`api.py:482`); the trial balance reads `premises`, **with no `f.`** (`api.py:395`). One filter, two spellings, on the same tab.
>
> And the same file's exception (3) says the trial-balance premises filter "*is NOT offered: `journal_lines.premises_id` is NULL on every row.*" **Both halves are stale** — the param is passed at `:395`, and **661 of 663 live lines carry a premises**. Measured, not assumed.

1·1 Journal `/:tenant/finance?view=journal` UPDATE ① THE LIST

SMARTBillHub

Finance › **Journal**

⌕ Search ? ◔

BB Brazos Bend Utilities

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

v2.340.0

DW Dana Whitfield
System Admin

Finance

FY2026 · P12 · Sep 2026 · Open

+ Journal entry

General ledger | Revenue & AR | Deposits & Bank | Taxes & Remittance | Costs | Close & export

1 Journal **Balance** **Chart** **Accruals**

Journal entries

2 Search memo, entry number… | All | Posted × | 3 + Filter ▾ | 5 ↓ | 4 ▦ Columns

6 ENTRY NO. ⇅	DATE ⇅	TYPE	MEMO	SOURCE	DEBIT	CREDIT	STATE	7 ACTIONS
JE-080226-0043	2026-08-02	Standard	Bill run 2608-A	Invoice	$15,066.00	$15,066.00	Posted	👁 ⟲ …
JE-080226-0042	2026-08-02	Standard	Payment PY-2608-0041	Payment	$292.87	$292.87	Posted	👁 ⟲ …
JE-073126-0041	2026-07-31	Accrual	Unbilled revenue, July	Manual	$4,812.40	$4,812.40	Posted	👁 ⟲ …
Draft — none	2026-09-23	Standard	Reclass 6900→6910	Manual	$400.00	$400.00	Draft	👁 ✎ …

8 1–50 of 168 · Rows 50 ⌄ ⇤ ← Page 1 of 4 → ⇥

FIN-02

END STATE · DASHED = NEW · WALKED 2026-09-23

1 **Journal** · a view of its own, and the landing view · NEW · P1.15

2 **Search** · the server has accepted `q` since `api.py:483` and the front end has never sent it. Searches the memo — and the entry number once P0.3 lands · UI-RULES 12 · NEW CONTROL, EXISTING READ

3 **+ Filter** ▾ · five criteria, all four already accepted by the server · UI-RULES 12 · NEW

4 ▦ **Columns** · the shared `ColumnChooser` — show/hide, reorder, Reset, persisted per user server-side · UI-RULES 14 · NEW

5 ↓ **Download** · the filtered journal as CSV through the IS-11 pipeline · NEW · P4.1

6 **Sortable headers** · ⇅ cycling asc → desc → clear; the SQL ordering already exists at `reporting.py:2731` · UI-RULES 13 · NEW

7 **Row actions as icons** · 👁 view · ⟲ reverse, then … . **Two visible plus overflow** — the cap `RowActions` does not enforce · UI-RULES 15 · CHANGED

8 **Pagination footer** · without it a clerk reaches 50 entries and no more · UI-RULES 16 · NEW · P1.1 · P1.8

THE ENTRY NUMBER — FORMAT RULED BY THE PRODUCT OWNER, 2026-09-23

RULED **PO, verbatim: "*you need je-MMDDYY- nnnn like this otherwise it will not work*".** So the format is `JE-{MM}{DD}{YY}-{0000}` , rendering **JE-080226-0043** for an entry dated 2 August 2026.

⚠ **THIS SUPERSEDES THE SEEDED DEFAULT AND THE SUPERSESSION IS STATED RATHER THAN ASSUMED.** `manual_je.py:75,79` seeds `JE-{000000}` for every tenant, on the strength of `BR-LED-021` (PO 2026-08-12, `OQ-176` , "*Approve a JE- series*") — which approved the **series** and never fixed its **shape**. The design's worked examples **JE-104** (LE-8) and **JE-201/JE-202** (LE-4) are plain running

numbers and are **also superseded**.

The change is free, and this is the fact that makes it so. 0 of 168 posted entries carry a number today and `number_series.journal_entry.next_number` is still **1**. **There is no existing number to migrate, in any tenant.** Change the format now and nothing is reprinted, re-issued or made inconsistent. **After P0.3 ships this becomes a migration.**

It is mechanically valid — checked, not assumed. `{MM}`, `{DD}` and `{YY}` are supported tokens (`numbering.py:52`), and the format string is **22 characters against a `MAX_FORMAT_LENGTH` of 24**. The date comes from the **entry date**, never a clock — `BR-TEN-043` .

And it is the shape the platform already uses: the payment series is `PY-{YY}{MM}{DD}-{0000}` , date plus a four-digit counter, which is why `PY-2608-0041` appears in the memo column two rows down. **The counter does not reset daily** — `RESTART_PERIODS` offers only `never` , `yearly` and `monthly` , so `nnnn` keeps running and the date identifies the day rather than counting within it. **That is exactly how payments already behave**, so this needs no new mechanism.

GAP **One consequence to build around rather than to ask about: `MMDDYY` does not sort chronologically as text.** `JE-011526-0001` sorts before `JE-020325-0001` though it is eleven months later. Payments avoid this by leading with `{YY}` . **So the Journal's *Entry no.* ⇅ column must sort by `entry_date` then sequence, never by the rendered string** — which is what `reporting.py:2731` already orders by. **P1.17** pins it with a test so nobody "simplifies" it to a string sort.

A draft has no number, and that is the rule rather than a gap. The number is stamped **at POST**, so the fourth row is drawn as a draft showing *Draft — none*. **An absent number on a draft is correct; an absent number on a posted entry is `DEF-LED-5540` .**

⚠ **Two recorded statements still conflict and P0.3 must not choose between them silently:** the decision log calls the series **"gapless"**, and `BR-TEN-035` says **"gaps ok, never reused"**. A rolled-back post consumes its number under the second and cannot under the first. **`BR-LED-021` flags it and nothing has ruled.**

Filter criteria

CRITERION	PARAMETER	VALUES	SOURCE	SERVED?
Search	`q`	free text — memo, and entry number after P0.3	`api.py:483` → `reporting.py:2726`	SERVED
Type	`f.type`	Standard · Accrual · Reversal	`JE_TYPES` , CHECK-constrained	SIGNED
State	`f.status`	Draft · Posted. ⚠ **not** *reversed* — that is a link, never a status	`JE_STATUSES`	SIGNED
Source	`f.source_type`	Invoice · Payment · Opening balance · Credit note · Counter receipt · Refund (the six live on the fixture)	`journal_entries.source_type`	SERVED
Property	`f.premises`	the tenant's premises	`api.py:482` → `reporting.py:730` , in SQL	SERVED
GL account	`f.ledger_account`	the chart	`api.py:485` · `GET /ledger/accounts`	SERVED
Period	`from` · `to`	the accounting period, defaulted from fiscal settings	`GET /shell/display`	SERVED

SIX CRITERIA, ZERO NEW ENDPOINTS

Every one of these is already accepted by the handler. This screen is a pure front-end change — the server has been ready since `api.py:478-485` and nothing has ever sent it a filter.

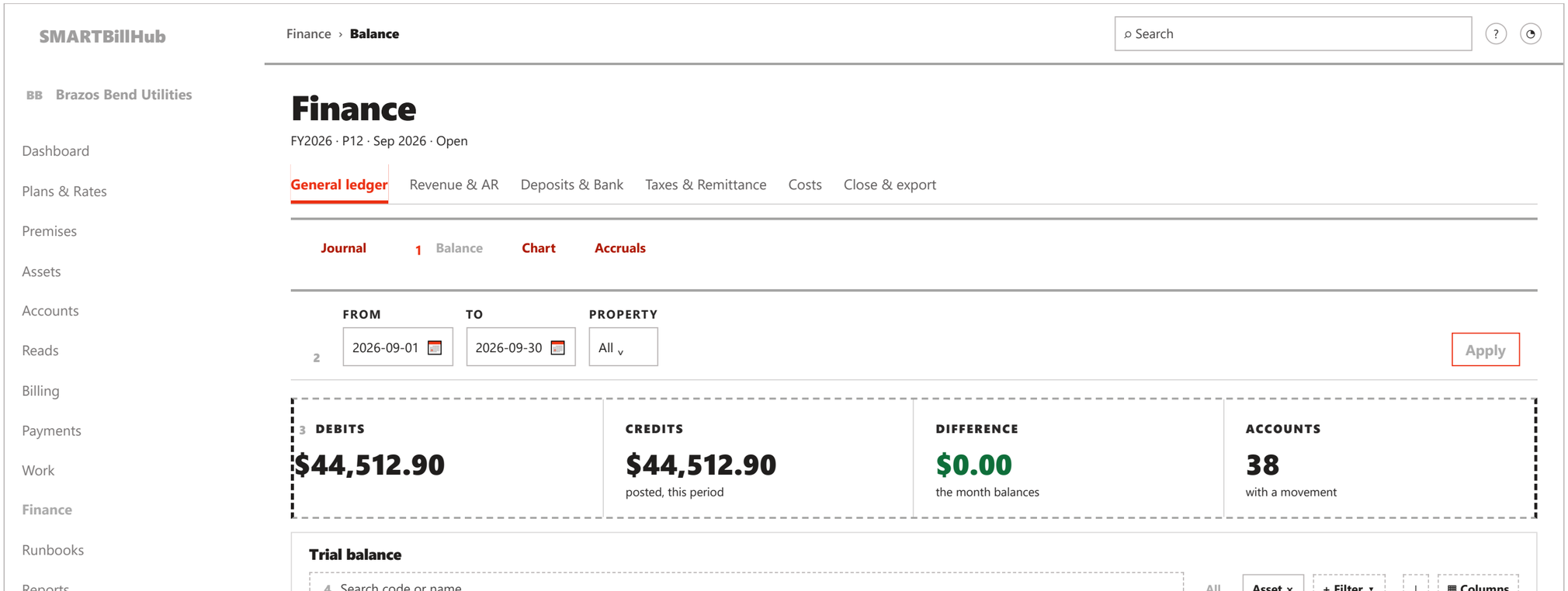
Components to use

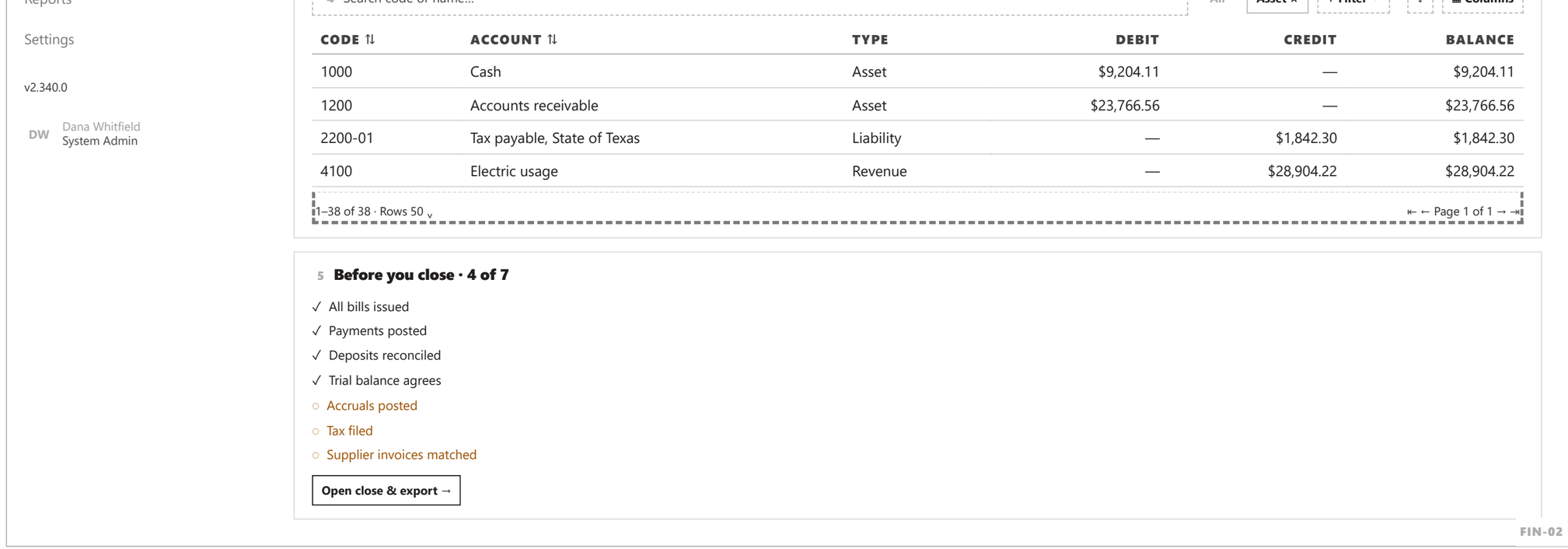
COMPONENT	WHERE	WHY THIS ONE
`DataTable`	the whole list	Brings the toolbar, chooser, pager and empty state together. **The trial balance already uses it** — this is reuse, not a new variant
`FilterBar`	chips `+ + Filter ▾`	Removable chips, *All* as default — UI-RULES 12
`ChipMultiSelect`	Source, Type	Multi-select is chips, never a wall of checkboxes — B14 rule 9
`ColumnChooser`	▦	Reorder and Reset, persisted server-side per user
`ListPager`	footer	The ratified footer shape
`DownloadButton`	↓	Carries its own disabled reason until P4.1 registers the resource
`MoneyCell`	Debit · Credit	**The one money formatting path** — BR-SHL-023. Never `${amount} {currency}`
`DateCell`	Date	Regional format from the tenant

`StatusPill`	State	11px, square, the ratified pill
`RowActions`	Actions	Icons with tooltips. ⚠ It is a bare `<span>` and **builds no overflow** — the cap has to be honoured by the caller
`Panel` · `PageHeader` · `FinanceHeader`	frame	One page header, one tab strip — B14 rules 2 and 43

FOR	A finance officer looking for a specific entry, or reviewing what posted this period.
DOORS	**Nine** — the rail (`AppShell.tsx:270`) · the tab strip twice · the Dashboard aging panel and to-do row · the write-off result on Outstandings · back links from all three report viewers. **It is the landing view, so every door to `/finance` is a door to this screen**
PRIMARY ACTION	`+ Journal entry` — `+ Noun` opens a creator, B14 rule 4
FIELDS	Entry no. · Date · Type · Memo · Source · Debit · Credit · State · Actions
TABLE CONTROLS	**All six, none of which exists today.** Search ✗ · chips ✗ (one status select only) · `+ Filter` ✗ · Columns ✗ · Download ✗ · sortable headers ✗ · **pager ✗**. The list pages **in Python** at `api.py:487` and the front end never sends `page`
WRITE SURFACES	**Detail** NOTHING — `GET /ledger/journal-entries/{id}` is mounted DELETE-only · **Add** 1·5a · **Edit** NOTHING, correctly — hard rule 4 · **Update** 1·5b reverse + repost
TODAY	`TODAY` Renders inside `?view=balance` as a panel below the trial balance. No search, no chips, no chooser, no pager, no sortable headers
TASK	`TASK` P1.15 the split · P1.1 onto DataTable · P1.8 SQL paging · P1.9 URL state · P0.3 the number · P4.1 the download
WALKED	**YES** — as a panel on `/brazosbend/finance`
RULES	Obeys B14 2 · 4 · 20 · 31 · 40 · hard rule 4. Breaks ✗ UI-RULES 12 · 13 · 14 · **15** · 16 · ✗ NFR-3

1·2 Balance `/:tenant/finance?view=balance` UPDATE ① THE LIST





END STATE · DASHED = NEW · WALKED 2026-09-23

1 **Balance** · was "*Trial balance & journal*"; the journal moved out to 1·1 · CHANGED · P1.15

2 **Params band** · from · to · property. **Stays here and not on Journal** — it describes the period's state, not an entry list · EXISTS

3 **Totals** · debits, credits, difference, account count. `KPIStrip` · NEW

4 **Toolbar** · search, chips, Columns, download. **All four are client-side today** — the endpoint takes only `from` , `to` , `premises` · NEW

5 **Close checklist** · stays on Balance for the same reason as ②; it is read-only here and acted on in tab 6 · EXISTS

Filter criteria

CRITERION	PARAMETER	VALUES	SOURCE	SERVED?
Period	`from · to`	date pair, defaulted from fiscal settings	`api.py:393`	SERVED
Property	**`premises`**	the tenant's premises	`api.py:395 → reporting.py:730`	SERVED ⚠ **no `f.` prefix**
Search	`q`	code or account name	—	NOT SERVED client-side
Type	`f.type`	Asset · Liability · Equity · Revenue · Expense	`LEDGER_ACCOUNT_TYPES`	NOT SERVED client-side
Hide zero rows	`f.nonzero`	on · off	—	NOT SERVED

> **CLIENT-SIDE IS LAWFUL HERE, AND ONLY HERE**
>
> A trial balance is **one row per account** — 38 today, and **bounded by the chart, not by the tenant's size**. It is the one Finance list NFR-3 does not bite on, so filtering in the browser is a deliberate choice rather than the debt `DEF-PAY-4221` describes. **Stated so nobody "fixes" it into a server round-trip.**

Components to use

COMPONENT	WHERE	WHY THIS ONE
`DataTable`	the balance	Already used here — `manualPagination={false}` , since the rows are bounded
`KPIStrip`	totals	⚠ `.kpi .v` ellipsises at `minmax(180px,1fr)` ; a 10-digit total truncates

`FilterBar · ChipMultiSelect`	Type	Five values — chips, not a select
`DateField ×2`	from · to	B14 rule 7
`MoneyCell`	Debit · Credit · Balance	BR-SHL-023
`Panel · ColumnChooser · DownloadButton`	frame	The shared kit

FOR	A finance officer checking the month balances before close.
DOORS	**Four** — the **Balance** view button on 1·1, 1·3 and 1·4 · a saved `?view=balance` bookmark, **which is why the value did not change** · the Dashboard "*trial balance*" to-do · 6·1's back link
PRIMARY ACTION	None — it is a read. `Open close & export →` is the onward door
FIELDS	Code · Account · Type · Debit · Credit · Balance
TABLE CONTROLS	Sortable ✓ · pager ✓ (bounded) · search ✗ · chips ✗ · Columns ✗ · download ✗
WRITE SURFACES	**None.** A trial balance is derived; nothing on it is editable
TODAY	`TODAY` Renders the trial balance **and** the journal on one page. The journal half moves to 1·1
TASK	`TASK` P1.15 the split · P1.3 the toolbar · P4.1 the download
WALKED	**YES** · 38 accounts, difference $0.00
RULES	Obeys B14 2 · 20 · 31 · 40 · NFR-3 (bounded — stated above). Breaks ✗ UI-RULES 12 · 14

1·3 Chart `/:tenant/finance?view=chart` UPDATE ① THE LIST

SMARTBillHub

Finance › **Chart**

⌕ Search ? ◔

BB Brazos Bend Utilities

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports

Finance

FY2026 · P12 · Sep 2026 · Open

+ Account

General ledger | Revenue & AR | Deposits & Bank | Taxes & Remittance | Costs | Close & export

Journal | Balance | 1 Chart | Accruals

Chart of accounts

2 Search code or name… | All | Active × | 3 + Filter ▾ | Show package codes | ↓ | ▦ Columns

CODE ⇅	NAME ⇅	TYPE	DESIGNATION	CATEGORY	4 QUICKBOOKS	XERO	STATE	ACTIONS
1000	Cash	Asset	Cash	Current	QBO·101	Not mapped	Active	✎ ⊘ …
1200	Accounts receivable	Asset	Receivable	Current	QBO·102	XR·A12	Active	✎ ⊘ …
2200-01	Tax payable, State of Texas	Liability	Tax payable	Current	Not mapped	XR·L20	Active	✎ ⊘ …
4100	Electric usage	Revenue	Service revenue	Operating	QBO·401	XR·R41	Active	✎ ⊘ …

1–50 of 38 · Rows 50 ⌄ ⇤ ← Page 1 of 1 → ⇥

Settings

v2.340.0

DW Dana Whitfield
System Admin

END STATE · DASHED = NEW · WALKED 2026-09-23

1 **Chart** · was "*Chart of accounts & mapping*"; the mapping is columns ④, not a second name · CHANGED · P1.15
2 **Search** · **already served** at `chart_api.py:260` and already sent · EXISTS
3 **+ Filter** ▾ · type, category, status all served; **designation is not** · NEW
4 **Package codes** · `qbo_ref` · `xero_ref` . Behind `with_references=1` , so the columns are only fetched when shown · EXISTS

Filter criteria

CRITERION	PARAMETER	VALUES	SOURCE	SERVED?
Search	`q`	code or name	`chart_api.py:260`	SERVED
Type	`f.type`	Asset · Liability · Equity · Revenue · Expense	`LEDGER_ACCOUNT_TYPES` , `chart_api.py:261`	SIGNED
Category	`f.category`	tenant's own categories	`chart_api.py:265`	SERVED
State	`f.status`	Active · Retired (`deleted_at`)	`chart_api.py:266`	SERVED
Show package codes	`with_references`	`1` · `true` · `yes`	`chart_api.py:267`	SERVED
Designation	`f.designation`	Cash · Receivable · Tax payable · Service revenue …	`ledger_accounts.designation`	NOT SERVED — **the gap**
Unmapped only	`f.unmapped`	on · off	—	NOT SERVED — **P4.4 needs it**

TWO MISSING FILTERS, AND THE SECOND BLOCKS A CLOSE

GAP **Designation** is how 4·1 finds the tax accounts and how the exporter finds cash — it is the load-bearing column and you cannot filter by it. **Unmapped only** matters more: **export gate 4 refuses the month if any account it needs has no package code**, and with 38 accounts and two targets there is no way to list the ones that will fail. Today the clerk hits the refusal and hunts by eye.

Components to use

COMPONENT	WHERE	WHY THIS ONE
`DataTable`	the chart	Bounded like the balance, so paging stays client-side
`FilterBar` · `ChipMultiSelect`	type · category · state	Three served criteria
`ColumnChooser`	▦	**Drives** `with_references` — showing the package columns is what fetches them
`RowActions`	Actions	✎ edit · ⊘ retire, then … . **Retire is danger tier** — B14 rule 6 names it
`StatusPill` · `AbsentFigure`	State · Not mapped	An unmapped code is a **stated absence**, never blank — BR-SHL-025

FOR	An admin maintaining the chart and its accounting-package mapping.
DOORS	**Three** — the **Chart** view button on 1·1, 1·2 and 1·4 · `?view=chart` bookmarks · 6·2a's "*fix the mapping*" link from a refused export
PRIMARY ACTION	`+ Account`
FIELDS	Code · Name · Type · Designation · Category · QuickBooks · Xero · State · Actions
TABLE CONTROLS	Search ✓ · chips ✓ · sortable ✓ · **Columns ✗ · download ✗ · pager ✗**

WRITE SURFACES	**Add** `POST /ledger/accounts` · **Edit** name, category, refs · **Retire** sets `deleted_at` ; **history is never deleted**
TODAY	TODAY Search and three filters work. No chooser, no download, no designation filter
TASK	TASK P1.15 rename · P1.4 chooser + download · P4.4 the unmapped filter
WALKED	**YES** · 38 accounts, 6 unmapped on one target or both
RULES	Obeys B14 2 · 4 · 6 · 20 · 31 · UI-RULES 12 · 13. Breaks *X* UI-RULES 14 · 15

1·4 Accruals `/:tenant/finance?view=accruals` UPDATE ① THE LIST

CORRECTION · ALL FOUR ENGINES EXIST, AND TWO OF THEM MUST NEVER REVERSE

TODAY **The previous version of this drawing was wrong twice.** It said depreciation and the bad-debt allowance "*have no engine at all*" — **both do**: all four engines have a preview and a post route (`accruals_api.py:585-597`), about 145 KB of tested Python, and **no screen calls any of them**. And it promised "*four balanced entries, each with its reversal linked*" — **which would have built a money error**.

Two of the four are accruals; two are not, on purpose. Unbilled revenue and unpaid costs post `type=accrual` and **reverse on the 1st** (`BR-LED-005`). Depreciation posts `type=standard` and **stands** — `depreciation.py` : "*IT IS NOT AN ACCRUAL AND IT MUST NEVER BE ONE … a depreciation entry that reversed itself on day 1 would leave the plant at its purchase cost forever*", the error `CR-060` §5 measures at **$40,000** on a truck. The allowance **stands** too — `allowance.py` : "*IT IS A BALANCE, NOT AN ACCRUAL … carried from month to month.*"

So the drawing now carries a *Type* and a *Reverses* column, and the difference is on screen rather than in a docstring. **The view keeps the PO's one-word name, *Accruals***, even though two of its four rows are not accruals — the columns carry the distinction.

SMARTBillHub

Finance › **Accruals**

⌕ Search ? ◔

BB Brazos Bend Utilities

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

Finance

FY2026 · P12 · Sep 2026 · Open

6 Post all ✓

General ledger | Revenue & AR | Deposits & Bank | Taxes & Remittance | Costs | Close & export

Journal | Balance | Chart | 1 Accruals

Period-end entries

2 Search… | All | Not posted × | 3 + Filter ▾ | ↓ | ▦ Columns

ENTRY ⇅	4 TYPE	SERVICE	BASIS	AMOUNT	REVERSES	STATE	ACTIONS
Unbilled revenue	Accrual	Electric	Reads after cycle end	$4,812.40	2026-10-01	Not posted	👁 ✓ …
Unbilled revenue	Accrual	Water	Reads after cycle end	$1,106.85	2026-10-01	Not posted	👁 ✓ …
Unpaid costs	Accrual	—	Received, not invoiced	$2,400.00	2026-10-01	Not posted	👁 ✓ …
Depreciation	**Standard**	—	Straight line, asset register	$3,150.00	**Never**	Not posted	👁 ✓ …
Bad-debt allowance	**Standard**	—	Aging bands, to target	$1,184.30	**Never**	Not posted	👁 ✓ …

1–5 of 5 · Rows 50 ⌄ ⇤ ← Page 1 of 1 → ⇥

5 **Waiting to reverse**

v2.340.0

DW Dana Whitfield
System Admin

ENTRY	POSTED	AMOUNT	REVERSES
JE-083126-0031 · Unbilled revenue	2026-08-31	$4,390.12	2026-09-01

Run reversals

FIN-02

END STATE · DASHED = NEW · FIGURES ILLUSTRATIVE (B14 RULE 37)

1. **Accruals** · the PO's one-word name, kept · EXISTS
2. **Search** · over entry and service · NEW
3. **+ Filter** ▾ · entry, type, service, state — all in the browser; the list is bounded · NEW
4. **Type and Reverses** · **the two columns that stop the $40,000 error**. Accrual → reverses on the 1st; Standard → never · NEW
5. **Waiting to reverse** · **what the view does today** — the accruals due to reverse, and the day-1 sweep that mirrors them. Kept · EXISTS
6. **Post all ✓** · four independent posts on four routes, each keyed to its period, so a re-run posts only what is missing. Per-row ✓ posts one · NEW · P3.1

Filter criteria

CRITERION	PARAMETER	VALUES	SOURCE	SERVED?
Period	`from · to`	the accounting period	each engine's preview route	SERVED
Entry	`f.engine`	Unbilled revenue · Unpaid costs · Depreciation · Bad-debt allowance	DERIVED — the four routes	NOT SERVED in the browser
Type	`f.type`	Accrual · Standard	`JE_TYPES`	NOT SERVED in the browser
Service	`f.service`	**eleven**, incl. **Fire**	`GET /reference/services` — **never a literal array**	NOT SERVED in the browser
State	`f.status`	Not posted · Posted	DERIVED from the posted entries	NOT SERVED in the browser

> **FIVE ROWS — SO THE TOOLBAR IS FOR CONSISTENCY, NOT VOLUME**
>
> Four engines, with unbilled revenue split by service. **It will not grow with the tenant.** The toolbar is here because every list carries the same six controls; server paging is not proposed and NFR-3 does not bite.

Components to use

COMPONENT	WHERE	WHY THIS ONE
`DataTable`	both lists	The same table as the other three views
`StatusPill`	Type	**Accrual and Standard in different tones** — the difference has to be seen, not read
`RowActions`	Actions	👁 the lines it would post · ✓ post this one, then …
`ConfirmDialog`	`Post all ✓`	Irreversible. Two short sentences — "*Posts four entries. Two reverse on the 1st.*" — B14 rule 33
`MoneyCell · DateCell`	cells	The shared kit

FOR	A finance officer making the month-end postings before closing the period.
DOORS	**Three** — the **Accruals** view button on 1·1, 1·2 and 1·3 · `?view=accruals` bookmarks · **6·1's *Accruals posted* checklist row**, which is the door that matters
PRIMARY ACTION	`Post all ✓`
FIELDS	Entry · Type · Service · Basis · Amount · Reverses · State · Actions
TABLE CONTROLS	All six, in the browser — bounded

WRITE SURFACES	**Post** per engine — preview and post, four times (`accruals_api.py:585-597`). **Two post `type=accrual` and reverse on day 1; two post `type=standard` and stand. Run reversals** — the day-1 sweep, which exists. **No edit, ever**
TODAY	`TODAY` Lists the accruals waiting to reverse and runs the sweep (`getPendingAccruals` , `runAccrualAutoReversals`). **None of the eight engine routes is called from any screen**
TASK	`TASK` P3.1 the four engines, preview then post · P1.15 the toolbar
WALKED	**YES** — the pending-reversal list and the sweep
RULES	Obeys B14 20 · 31 · 33 · `BR-LED-005` . Breaks *X* UI-RULES 12 · 13 · 14 · 15 today

1·5a New entries

`/:tenant/finance/journal-entries/new` `UPDATE` ③ WHAT YOU CAN DO

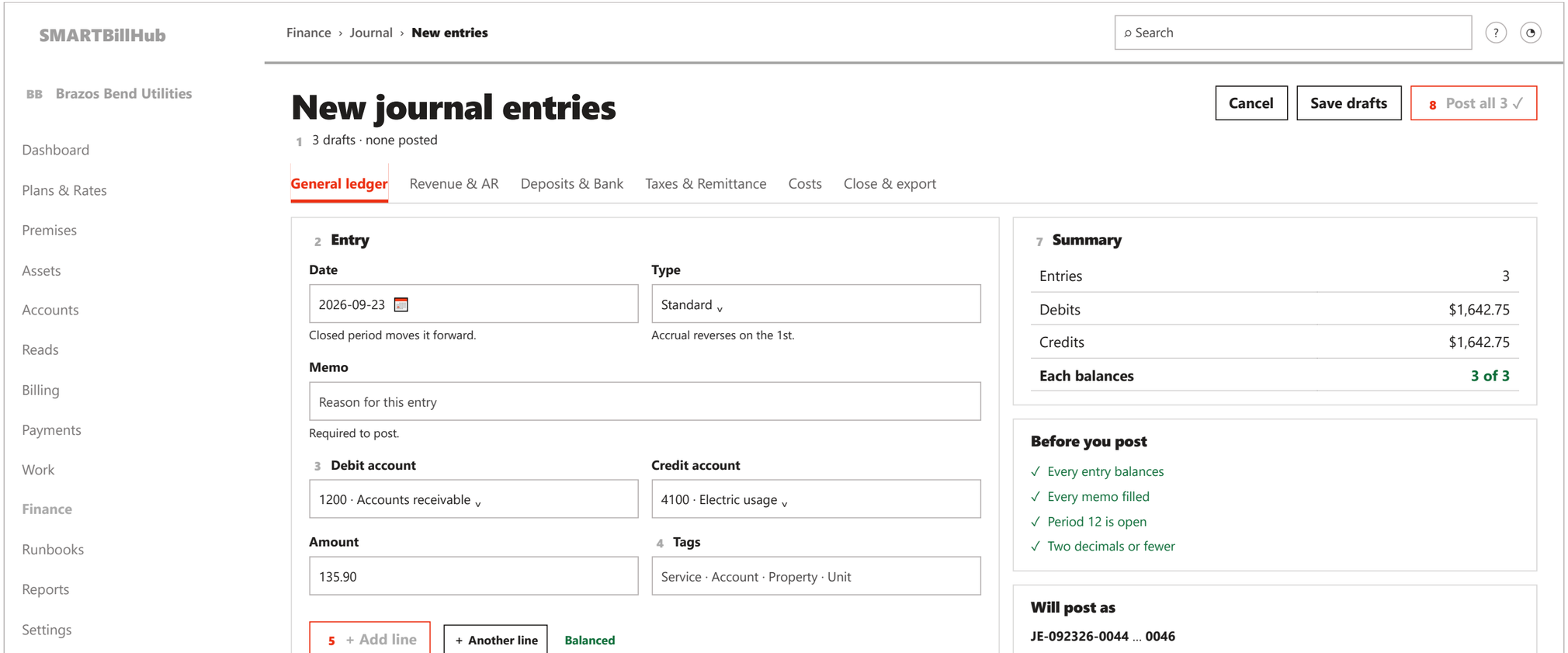
RULED 2026-09-23 — SEVERAL ENTRIES, COMPOSED ONE AT A TIME

`RULED` **PO: the screen takes "*1 or more entries*", and each row of the table is its own journal entry, not a line of one.** The composer sits in the **Entry** panel, `+ Add line` pushes what you typed into the **Entries** table below, and the summary reads the whole batch.

⚠ **That forces one thing about the composer, and it is the whole design.** `BR-LED-003` makes **every journal entry balance on its own** — so a row that is an entry cannot be a single account and a single amount. **The composer therefore captures a DEBIT account, a CREDIT account and ONE amount**, and each press produces a balanced two-line entry that cannot be unbalanced. `+ Another line` handles the rare entry needing three or more sides; it opens inside the composer and the running total must reach zero before `+ Add line` enables.

"Entries" is the PO's word and is used throughout. ⚠ It collides with `journal_entries` , where an *entry* is the same thing — which here is **correct rather than confusing**, because a row really is one entry. **The table head says *Entries · 3* and the button says *Post all 3* ✓**, so the count on screen is always a count of journal entries.

Opens from · `+ Journal entry` in the page head of **1·1 Journal** — **now a navigation, not a modal** · the ✎ row action on a **draft**, which reopens that one at `/journal-entries/{id}/edit`



v2.340.0

DW Dana Whitfield
System Admin

6 **Entries** 3

#	DATE	MEMO	DEBIT	CREDIT	AMOUNT	STATE	
1	2026-09-23	Reclass power cost	6910	6900	$400.00	Draft	✎ ✕
2	2026-09-23	Accrue unbilled water	1200	4200	$1,106.85	Draft	✎ ✕
3	2026-09-23	Correct meter rebill	1200	4100	$135.90	Draft	✎ ✕

One number each, at post.

FIN-02

PROPOSED · **WAS A SINGLE-ENTRY MODAL** · COMPOSER ABOVE, ENTRIES BELOW

1. **3 drafts · none posted** · the batch's state in four words · NEW
2. **Entry** · the composer. **Every field for one entry, and the add control lives here** — as ruled · NEW
3. **Debit and credit account, one amount** · **this is what makes a row an entry**. Two accounts and one figure cannot be unbalanced, so `BR-LED-003` holds by construction rather than by validation · NEW
4. **Tags** · the five dimensions, optional · P1.16 · NEW
5. **+ Add line** · the PO's name for it. Pushes the composed entry into the table and clears the form, keeping the date · NEW
6. **Entries** · one row per journal entry, each editable and removable before posting · NEW
7. **Summary** · reads the batch, not one entry. ***3 of 3* is the figure that matters** — a total that balances while an individual entry does not is still unpostable · NEW
8. **Post all 3 ✓** · the count is in the button, so nobody posts three thinking it is one · NEW

WHAT THIS NEEDS FROM THE BACKEND, AND IT IS NOT NOTHING

GAP **The service posts exactly ONE entry per call.** `ManualDraftResult` and `ManualPostResult` each carry one `entry_id`, one `entry_date`, one `number` (`manual_je.py:223-249`). **Posting three entries is three calls**, and three calls raise a question one call never had.

OPEN `OQ-LED-1515` PROPOSED **— if entry 3 of 5 fails, what happens to 1 and 2?** Two honest answers and **neither is chosen here**: **(a) all-or-nothing** — one transaction, nothing posts unless everything does, which is safest and means a single bad memo blocks four good entries; **(b) each stands alone** — 1 and 2 post, 3 is reported and stays a draft, which matches **hard rule 4** (a posted entry is never unwound) and **hard rule 6** (each entry carries its own `posting_key`, so a retry is idempotent). **(b) is what the existing machinery already does** if the screen simply calls three times — so doing nothing IS choosing (b), which is exactly why it has to be ruled rather than defaulted.

P1.21 builds the batch screen; **P1.22** is the backend once the question is answered. **P1.22 must not start first** — the answer decides whether it is a new endpoint or a loop in the client.

Fields

FIELD	CONTROL	SOURCE	REQUIRED
Date	Date field	Defaults to today, **kept after Add** — a batch is usually one day. Never read from a clock server-side (`BR-TEN-043`)	Yes
Type	Select	`JE_TYPES` — Standard · Accrual · Reversal	Yes
Memo	Text	Cleared after Add — **each entry needs its own reason**	**To post**
Debit account	Select	`GET /ledger/accounts` — **never a literal array**	Yes
Credit account	Select	Same source. **Refused if equal to the debit account**	Yes
Amount	Money	One figure, applied to both sides. **4 decimals stored, 2 displayed**; more than two is refused (`BR-LED-025`)	Yes
Tags	Selects	**P1.16.** Applied to both lines. Unit walks to its premises for an address — hard rule 8	No

Components to use

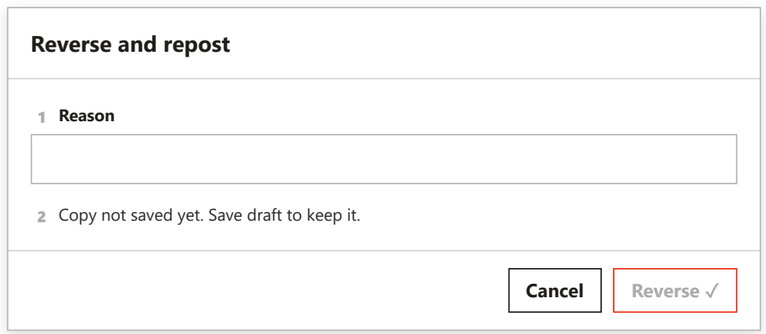
COMPONENT	WHERE	WHY THIS ONE
`PageHeader`	title + actions	Replaces the modal's header and footer. **Not** `WizardFrame` — composing is a loop, not a sequence of steps
`DateField · SelectField · TextField · MoneyField`	the composer	The `FormModal` field set, reused outside a modal

`DataTable`		Entries	Rows with ✎ × . **No toolbar or pager** — a batch you are composing is not a list; UI-RULES 16 exempts it
`CountBadge`		*Entries · 3*	The same badge the Revenue & AR view labels use — P2.15
`MoneyCell`		Amount	BR-SHL-023, and it enforces two decimals at the edge
`ConfirmDialog`		`Post all 3 ✓`	**The one thing that stays a modal.** Names the count and that it cannot be undone — rule 33, two short sentences
`Panel` ×5 · `StatusPill`		both columns	One panel per idea

FOR	An admin or billing clerk putting a batch of corrections into the books in one sitting.
DOORS	**Two** — `+ Journal entry` on 1·1 · the ✎ row action on a **draft**. **A posted entry has no** ✎ — hard rule 4
PRIMARY ACTION	`Post all 3 ✓` , page head right, after `Cancel` and `Save drafts`
FIELDS	Date · Type · Memo · Debit account · Credit account · Amount · Tags — **per entry**
TABLE CONTROLS	None, and that is correct. **A batch being composed is not a list**
WRITE SURFACES	**Add** `POST /ledger/journal-entries` **per entry** · **Update** `post_manual_je` **per entry** · after posting, **nothing**
TODAY	TODAY A `FormModal` over 1·1 that composes **one** entry with a free-form line list. No address, no batch, no draft to return to
TASK	TASK **P1.21 the batch screen** · TASK P1.22 the posting path, **after** `OQ-LED-1515` · P1.16 the tags · P0.3 the number on `post()`
WALKED	**NO** — read from the render tree
RULES	Obeys B14 4 · 6 · 7 · 32 · 33 · 38 · 40 · 43 · hard rules 4 · 6 · 7 · 8 · `BR-LED-003` · `BR-LED-025`

1·5b Reverse `modal` KEEP ③ WHAT YOU CAN DO

Opens from · the ↺ row action on **1·1 Journal**, on a `posted` entry only. A draft has no ↺ ; an already-reversed entry renders it disabled with its reason



AS BUILT · COPY CUT TO B14 RULE 33

CORRECTION TO THE PREVIOUS VERSION OF THIS DOCUMENT

Two strings were over the cap. The lead paragraph — "*A posted entry is never edited. This writes a mirror entry that cancels it, then opens a corrected copy for you to check and post*" — is **rationale, 25 words, and is deleted outright**; a clerk who pressed ↺ already intends this. The warning ran 22 words in two long sentences and is now **two sentences of four and five**. Rule 33 permits at most two sentences of six before an irreversible act.

1 **Reason** · rides onto the repost memo as "*Correction of a reversed entry: {{reason}}*" · EXISTS

2 **Unsaved warning** · the corrected copy is a draft until saved · COPY CUT

FOR	An admin or billing clerk correcting a posted entry without breaking immutability.
DOORS	**One** — the ↺ row action on 1·1
PRIMARY ACTION	`Reverse` ✓
WRITE SURFACES	**Update** `POST /ledger/journal-entries/{id}/reverse` — writes a mirror, opens a copy. A draft is refused ("*not in the books yet*"); an already-reversed one returns "*the first mirror stands*"
TODAY	TODAY Idempotent — `journal_entries.reversal_of` carries a UNIQUE, so one mirror per entry
TASK	Copy cut only
WALKED	**NO** — no posted entry in the fixture had been reversed
RULES	Obeys hard rule 4 · `BR-LED-003` · **33 after the cut above** · `OQ-175` RULED System Admin and Billing Clerk; Executive not

④ What each act changes, and where — Tab 1

ACT	WRITES	THE LEDGER	ELSEWHERE
Post an entry 1·5a	`journal_entries` (`draft` → `posted` , `posted_at` , **and after P0.3 a `JE-` number**) · `journal_lines` · `audit_log`	**Is the ledger.** Balanced pair enforced by constraint; refused if debits ≠ credits or any amount carries more than two decimals (`BR-LED-025`)	**1·2** trial balance moves · 2·3 Tie-out recomputes · 6·1 checklist may change · **a closed period dates it forward and says so**
Reverse + repost 1·5b	A **mirror** entry with `reversal_of` set · `audit_log` on both · a new draft copy	Two balanced pairs, net zero. **The original is never touched**	**1·1** shows both and marks the original *reversed later* · an export batch already sent **does not move** — 6·2's drift panel reports it
Add / retire an account 1·3	`ledger_accounts` · `deleted_at` on retire · `audit_log`	Nothing posts. A retired account keeps its history	6·2a export gate 4 refuses the month if an account it needs has no package code · 4·1 reads the 2200-series by designation
Post accruals 1·4, after P3.1	**Two** entries of type `accrual` (unbilled revenue, unpaid costs) with `auto_reverse_entry` linked, and **two of type `standard`** (depreciation, the allowance) **that stand**	Recognises this period's revenue, cost, wear and bad-debt risk. **The day-1 sweep reverses the two accruals only** — a reversed depreciation entry would leave the plant at purchase cost forever	6·1 "*accruals posted*" flips to done · 5·2 bought-vs-billed reconciles against the accrued side

Tab 2 · Revenue & AR

RULED **Five views, one per category the board used to stack** — Services · Aging · Tie-out · Write-offs · Refunds. **Each has a table, search, filters and a column chooser.** The KPI strip sits above the view strip on every view, and the pending counts ride in the labels. **The board and its four bands are retired.**

> **READ BEFORE BUILDING ANYTHING ON THIS TAB**
>
> **Only 2 of the 5 views are Finance code.** Services and Tie-out are `features/ledger/` ; Write-offs and Refunds are **Payments**; Aging is built over a **Reports** report. **Mount their components — never rebuild them.** The full map is §05.
>
> **The board's three report links stay in Reports.** Account ledger, By premises and Statements were placed here on 2026-09-24 and removed the same day by the PO: "*we can cover this and premises in report any way*". Each keeps its own address under `/reports/` ; a Finance view of a Reports screen would only duplicate it.

Search, filters and columns — which ones the server supports today

TAB	SEARCH	FILTERS	COLUMNS	PAGES
Services	in the browser (≤11 rows)	Period ✓ · Service (in the browser)	✓	not needed
Aging	✗	As-of · Band · Service · Property — **none supported**	✓	✗
Tie-out	in the browser (3 rows)	Period ✓ · State (in the browser)	✓	not needed
Write-offs	✗	State ✓ · Reason · Date · Requested by — **3 of 4 not supported**	✓	✗ **no paging, on purpose**
Refunds	✗	State ✓ · Method · Date — **2 of 3 not supported**	✓	✗ **no paging, on purpose**

The write-off list reads exactly one filter (`f.status`). So does the refund list.

Screen count

	TODAY	PROPOSED
Views	2 (the board, the refunds queue)	**5**
Pop-ups: approve ×2, raise a refund, issue / clear / void	4	4
Refund detail	1	1
Tab 2	**7**	**10**
Whole document	**29**	**32 (+3)**

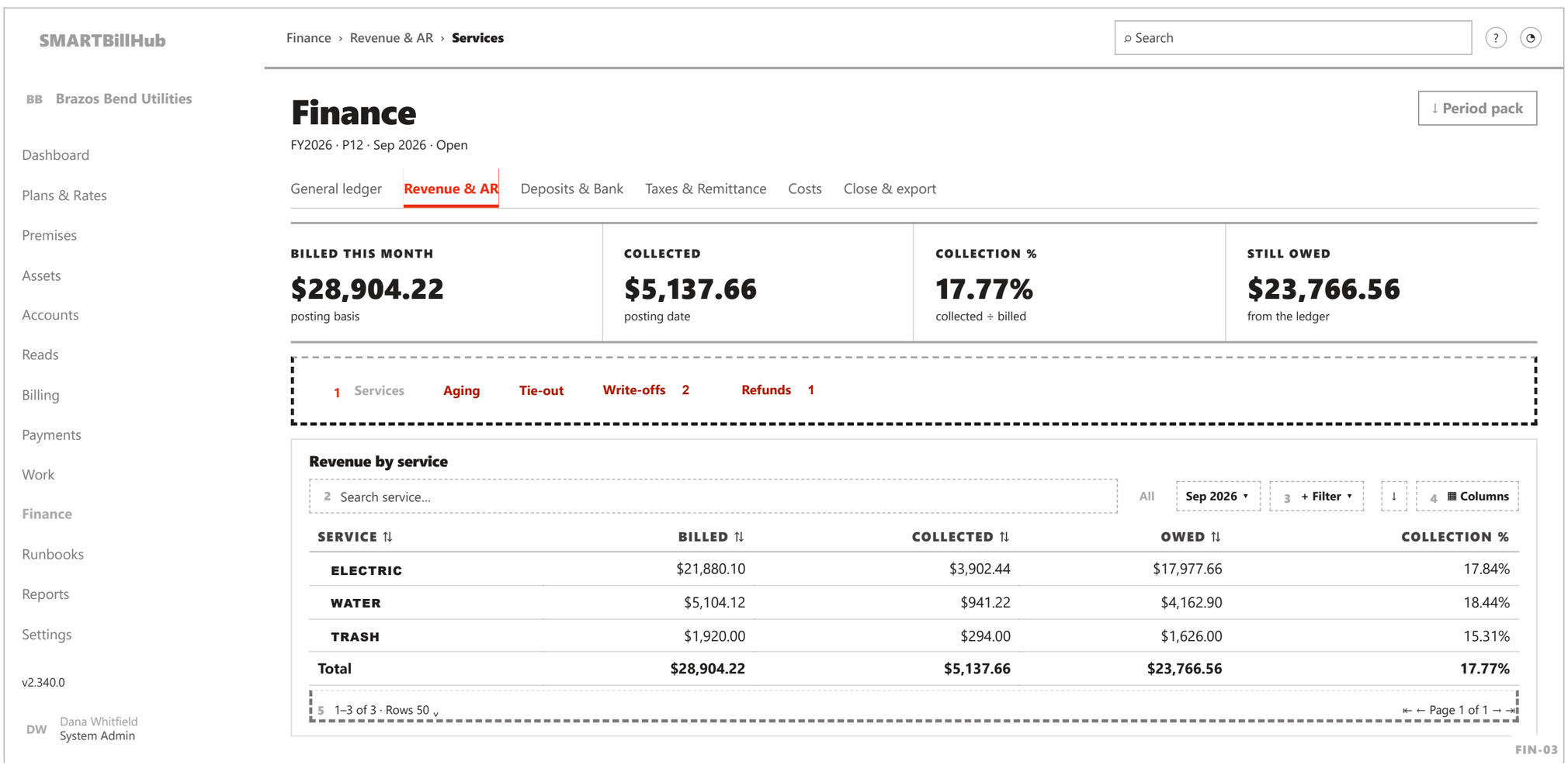
Where the +3 comes from: the board goes (−1). Services, Aging, Tie-out and Write-offs are new (+4). Refunds reuses today's screen (0). The three report screens stay in Reports (0).

Tasks it adds

TASK	KIND	WHAT
TASK P2.18	Front end	The view strip: 5 views, `?view=` switching, KPIs above, counts in the labels. `/finance/revenue` opens on Services; `/finance/refunds` redirects rather than breaking old links
TASK P2.19	Full stack · PAYMENTS	Search, the missing filters and the pending count on the write-off and refund lists, and the two views' toolbars. **Server paging comes from P1.8**
TASK P2.20	Front end	Services and Tie-out on the shared table
TASK P2.21	Full stack · REPORTS	The Aging view over the `ar_aging` report, and the report's paging and filters
TASK P2.23	Test	Every old link to the board and to `/finance/refunds` lands on the right view
TASK P2.24	Other	5 view names across all eleven languages

88 → 90 tasks: 6 added, 3 replaced and 1 duplicate retired — each named in §00, with the old-to-new screen map.

2·1 Services `/:tenant/finance/revenue?view=services` UPDATE ① THE LIST



END STATE · DASHED = NEW · **THE LANDING VIEW**

1. **The view strip** · five views under Revenue & AR, switched by `?view=` . **The KPI strip sits above it on every view**, and the pending counts ride in the labels · NEW · P2.18
2. **Search** · UI-RULES 12 · NEW
3. **+ Filter** ▾ · the criteria below · UI-RULES 12 · NEW
4. ▦ **Columns** · the shared `ColumnChooser` , persisted per user · UI-RULES 14 · NEW
5. Bounded — every row fits one page, so paging stays in the browser · UI-RULES 16

Filter criteria

CRITERION	PARAMETER	VALUES	SOURCE	SERVED?
Period	`from · to`	the accounting period	`GET /finance/revenue-ar`	SERVED
Service	`f.service`	**eleven**, incl. **Fire**	`GET /reference/services` — never a literal array	NOT SERVED in the browser
Search	`q`	service name	—	NOT SERVED in the browser

Components to use

COMPONENT	WHERE	WHY THIS ONE
`DataTable`	the list	Brings toolbar, chooser, pager and empty state together — the same table as tab 1
`FilterBar` `ChipMultiSelect`	chips · `+ Filter`	Removable chips, *All* as default — P14 rule 9

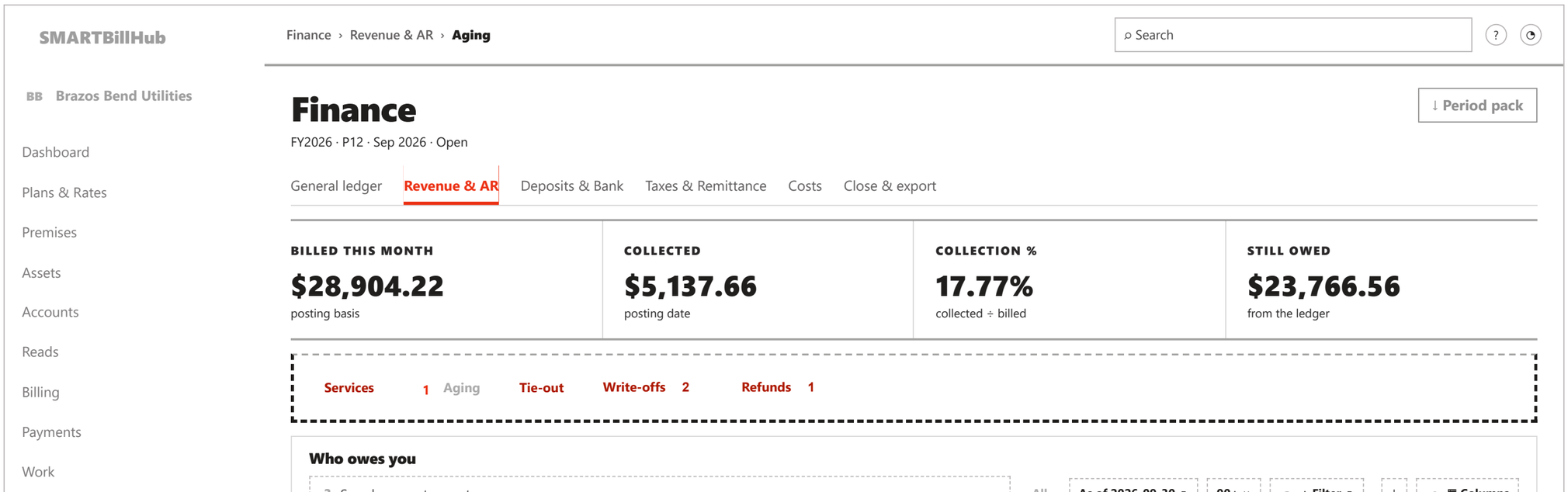
`FilterBar · ChipMultiSelect`	chips + `+ Filter ▾`	Removable chips, *All* as default — B14 rule 9
`ColumnChooser · DownloadButton`	▦ · ↓	The shared kit
`KPIStrip`	above the view strip	⚠ P2.1 — `.kpi .v` ellipsises a ten-digit total at 146px
`MoneyCell · DateCell · StatusPill`	cells	The one money path — BR-SHL-023

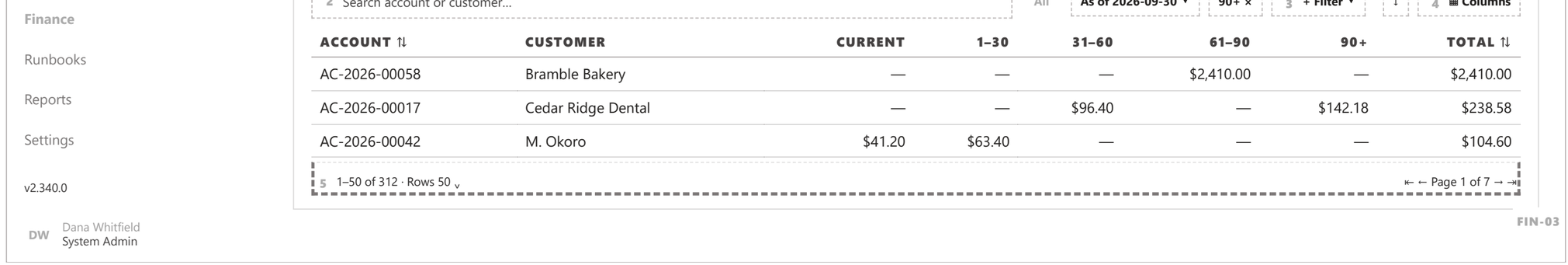
FOR	A finance officer answering *what did we bill, and how much came in?*
DOORS	**Three** — the **Revenue & AR** tab, **which lands here** · a `/finance/revenue` bookmark · the Dashboard revenue panel
PRIMARY ACTION	None — a read. `↓ Period pack` renders disabled with its reason
FIELDS	Service · Billed · Collected · Owed · Collection %
TABLE CONTROLS	All six, **in the browser** — at most eleven rows, bounded by the service catalogue and not by tenant size
WRITE SURFACES	**None**
TODAY	`TODAY` A panel on the board with no toolbar
TASK	`TASK` P2.18 the strip · `TASK` P2.20 onto the shared table
WALKED	**YES** — as a panel on the board
RULES	Obeys B14 2 · 20 · 31 · 40 · NFR-3 (bounded)

2·2 Aging `/:tenant/finance/revenue?view=aging` NEW ① THE LIST

OWNED BY REPORTS · THE REPORT EXISTS, THE SCREEN DOES NOT

`TODAY` The data is the `ar_aging` report in `06-backend/apps/reports/` (`catalog.KEY_AR_AGING`). **There is no aging screen anywhere in the product** — only this report and a Dashboard panel (`dashboard.py:673`). **Build the view over the report; do not copy the report into Finance.**





PROPOSED · THE WHOLE VIEW IS NEW · FIGURES ILLUSTRATIVE (B14 RULE 37)

1 **The view strip** · five views under Revenue & AR, switched by `?view=` . **The KPI strip sits above it on every view**, and the pending counts ride in the labels · NEW · P2.18
2 **Search** · UI-RULES 12 · NEW
3 **+ Filter** ▾ · the criteria below · UI-RULES 12 · NEW
4 ▦ **Columns** · the shared `ColumnChooser` , persisted per user · UI-RULES 14 · NEW
5 **Server paging** · without it a clerk sees 50 rows and no more · UI-RULES 16 · NFR-3

Filter criteria

CRITERION	PARAMETER	VALUES	SOURCE	SERVED?
As of	`as_of`	a date, defaulted to the period end	`ar_aging` report	NOT SERVED
Band	`f.band`	Current · 1–30 · 31–60 · 61–90 · 90+	DERIVED from the report	NOT SERVED
Service	`f.service`	**eleven**, incl. **Fire**	`GET /reference/services`	NOT SERVED
Property	`f.premises`	the tenant's premises	—	NOT SERVED
Search	`q`	account number or customer name	—	NOT SERVED

ONE ROW PER DEBTOR, SO IT MUST PAGE

GAP An aging list grows with the tenant — **at 40,000 accounts it cannot load whole**. None of the five criteria is supported by the report today, and neither is paging. **P2.21 is full stack for that reason**: mounting the report is the small half.

Components to use

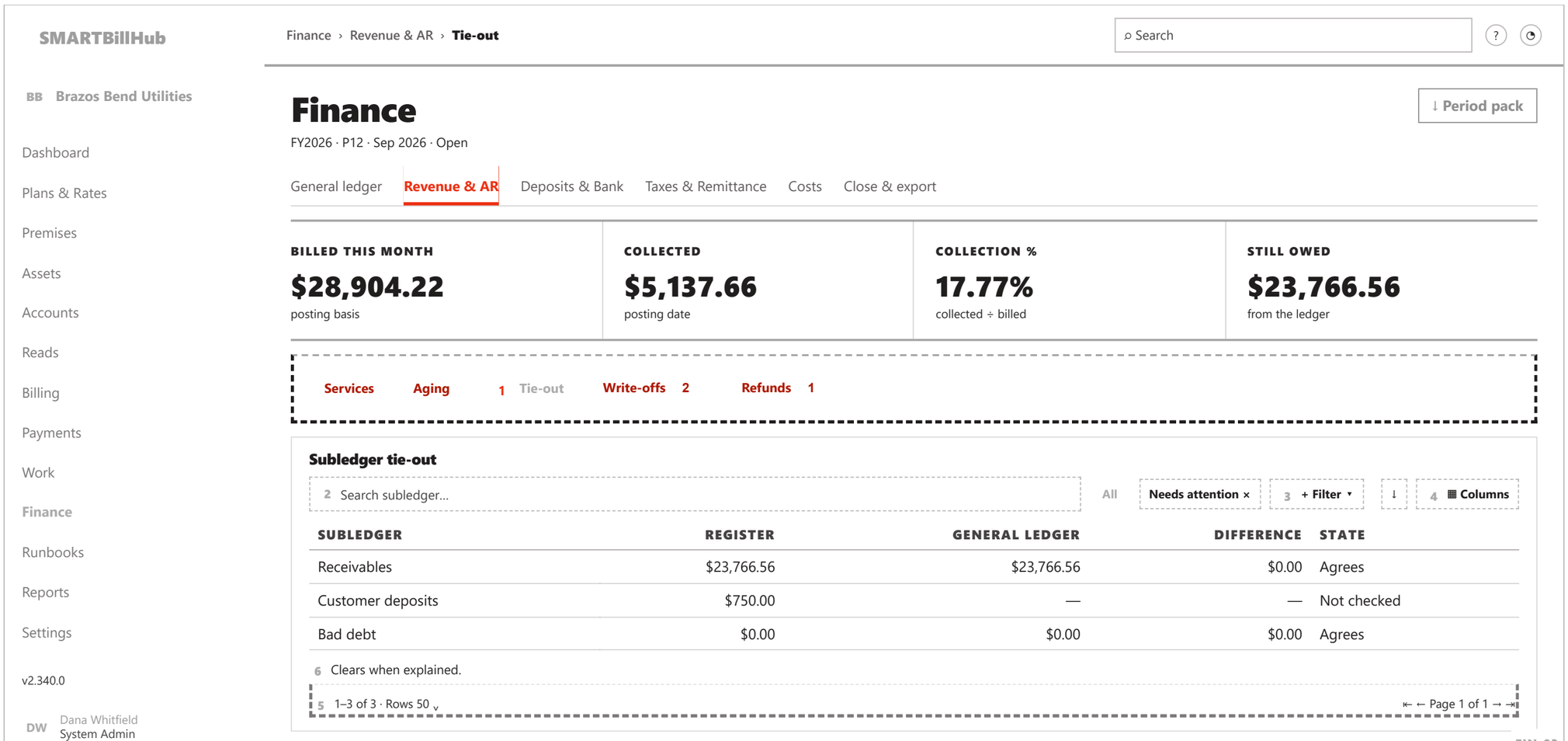
COMPONENT	WHERE	WHY THIS ONE
`DataTable`	the list	Brings toolbar, chooser, pager and empty state together — the same table as tab 1
`FilterBar · ChipMultiSelect`	chips `+ + Filter ▾`	Removable chips, *All* as default — B14 rule 9
`ColumnChooser · DownloadButton`	▦ · ↓	The shared kit
`KPIStrip`	above the view strip	⚠ P2.1 — `.kpi .v` ellipsises a ten-digit total at 146px
`MoneyCell · DateCell · StatusPill`	cells	The one money path — BR-SHL-023
`AbsentFigure`	empty buckets	A dash means *nothing owed in this band* — never an unknown. P2.14

FOR	A finance officer deciding who to chase, and in what order.
DOORS	**Two** — the **Aging** view · the Dashboard aging panel, **which today leads nowhere in Finance**
PRIMARY ACTION	None — a read

FIELDS	Account · Customer · Current · 1–30 · 31–60 · 61–90 · 90+ · Total
TABLE CONTROLS	All six — **none supported by the report today**
WRITE SURFACES	None here. A write-off is raised in Payments and approved on 2·4
TODAY	TODAY **No Finance screen.** The `ar_aging` report and a Dashboard panel only
TASK	TASK P2.21 the view and the report's paging and filters REPORTS
WALKED	**NO** — there is nothing to walk
RULES	Obeys B14 20 · 31 · 37. Breaks ✗ NFR-3 until P2.21

2·3 Tie-out

`/:tenant/finance/revenue?view=tieout` UPDATE ① THE LIST



FIN-03

END STATE · DASHED = NEW

1 **The view strip** · five views under Revenue & AR, switched by `?view=` . **The KPI strip sits above it on every view**, and the pending counts ride in the labels · NEW · P2.18

2 **Search** · UI-RULES 12 · NEW

3 **+ Filter** ▾ · the criteria below · UI-RULES 12 · NEW

4 ▦ **Columns** · the shared `ColumnChooser` , persisted per user · UI-RULES 14 · NEW

5 Bounded — every row fits one page, so paging stays in the browser · UI-RULES 16

6 **The undismissable note** · three words; it clears when the entry that explains the difference posts · EXISTS

Filter criteria

CRITERION	PARAMETER	VALUES	SOURCE	SERVED?
Period	`from · to`	the accounting period	`GET /finance/revenue-ar`	SERVED
State	`f.state`	Agrees · Differs · Not checked	DERIVED	NOT SERVED in the browser
Search	`q`	subledger name	—	NOT SERVED in the browser

> **THREE ROWS GET THE FULL TOOLBAR, FOR THE REASON ACCRUALS DOES**
>
> **Every list in the product carries the same six controls**, so a clerk who learns them once never finds them missing. **Not because three rows need filtering.** Server paging is not proposed and NFR-3 does not bite.

> **ONE ROW CAN NEVER AGREE**
>
> GAP **Customer deposits reads *Not checked* for good.** Nothing in the product writes the deposit register, so the General-ledger side has nothing to tie to (§05, cross-module). **The row is honest** — an `AbsentFigure` with its reason, never a zero.

Components to use

COMPONENT	WHERE	WHY THIS ONE
`DataTable`	the list	Brings toolbar, chooser, pager and empty state together — the same table as tab 1
`FilterBar · ChipMultiSelect`	chips `+` `+ Filter ▾`	Removable chips, *All* as default — B14 rule 9
`ColumnChooser · DownloadButton`	▦ · ↓	The shared kit
`KPIStrip`	above the view strip	⚠ P2.1 — `.kpi .v` ellipsises a ten-digit total at 146px
`MoneyCell · DateCell · StatusPill`	cells	The one money path — BR-SHL-023
`AbsentFigure`	Not checked	The reason travels with the missing figure — P2.14

FOR	A finance officer checking the books agree with the registers they summarise.
DOORS	**Two** — the **Tie-out** view · 6·1's *Trial balance agrees* checklist row
PRIMARY ACTION	None — a check
FIELDS	Subledger · Register · General ledger · Difference · State
TABLE CONTROLS	All six, in the browser — bounded
WRITE SURFACES	**None**. A difference clears by posting on 1·5a
TODAY	TODAY A panel on the board, with bad debt a separate panel
TASK	TASK P2.20 onto the shared table · P2.14 the absent figure
WALKED	**YES** — as a panel on the board
RULES	Obeys B14 20 · 31 · 33 · BR-SHL-025

2·4 Write-offs `/:tenant/finance/revenue?view=writeoffs` UPDATE ① THE LIST

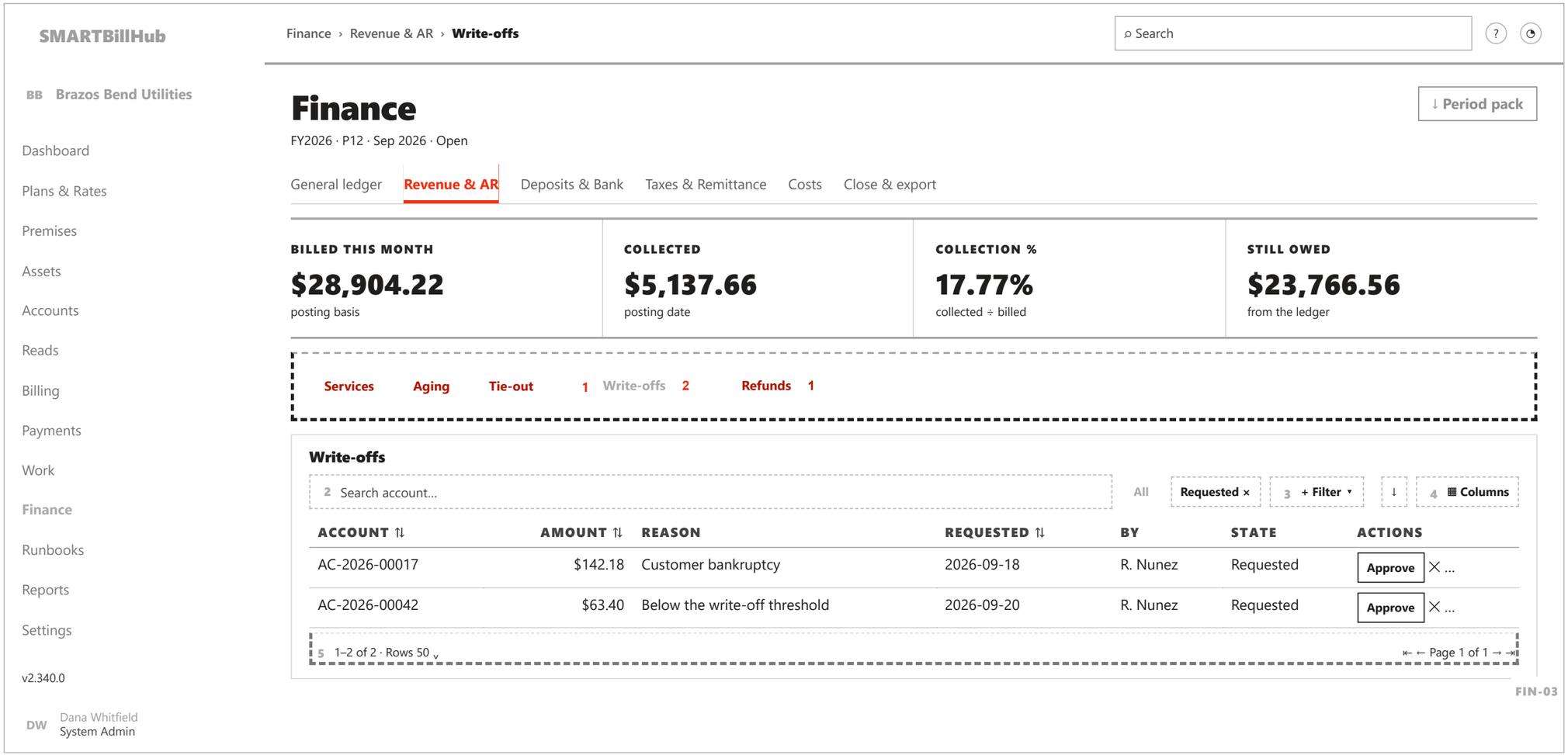
OWNED BY PAYMENTS · REUSED HERE, NOT REBUILT

TODAY **This is not Finance code.** It is `07-frontend/src/features/payments/WriteOffQueuePage.tsx` over `06-backend/apps/payments/` , shown as a view under Revenue & AR. **Mount the component; do not rebuild it.** Any change is a Payments change.

NO PAGING ON PURPOSE — AND WHY THAT HAS TO CHANGE

GAP The list reads **exactly one filter,** `f.status` , and its own code says: "*Unpaginated, deliberately … a utility under 3,000 connections has a queue a person reads in one screen, and a page of an approval queue is a queue that lies about what is pending.*"

The objection is real and the ceiling is not. The product now serves utilities up to **40,000 accounts**, and the project rules forbid an unpaginated list — the same excuse the `/outstandings` defect names. **The answer to "a page lies about what is pending" is the count in the view label**: *Write-offs · 2* states the true total on every page, so paging can no longer hide anything. **P2.19** serves that count beside the page.



END STATE · DASHED = NEW · THE QUEUE THAT WAS A PANEL ON THE BOARD

1 **The view strip** · five views under Revenue & AR, switched by `?view=` . **The KPI strip sits above it on every view**, and the pending counts ride in the labels · NEW · P2.18

2 **Search** · UI-RULES 12 · NEW

3 **+ Filter** ▾ · the criteria below · UI-RULES 12 · NEW

4 ▦ **Columns** · the shared `ColumnChooser` , persisted per user · UI-RULES 14 · NEW

5 **Server paging** · without it a clerk sees 50 rows and no more · UI-RULES 16 · NFR-3 · **P1.8**

Filter criteria

CRITERION	PARAMETER	VALUES	SOURCE	SERVED?
State	`f.status`	Requested · Approved · Rejected · Reversed	chip values are a literal array — `WriteOffQueuePage.tsx:135` , the P1.5 shape	SERVED
Reason	`f.reason`	Below the write-off threshold · Customer bankruptcy · Deceased, no estate · …	the reasons US-PAY-012 ships	NOT SERVED
Requested	`from · to`	a date range	—	NOT SERVED
Requested by	`f.requested_by`	the tenant's users	—	NOT SERVED
Search	`q`	account number	—	NOT SERVED

Components to use

COMPONENT	WHERE	WHY THIS ONE
`DataTable`	the list	Brings toolbar, chooser, pager and empty state together — the same table as tab 1
`FilterBar · ChipMultiSelect`	chips + `+ Filter ▾`	Removable chips, *All* as default — B14 rule 9
`ColumnChooser · DownloadButton`	▦ · ↓	The shared kit
`KPIStrip`	above the view strip	⚠ P2.1 — `.kpi .v` ellipsises a ten-digit total at 146px
`MoneyCell · DateCell · StatusPill`	cells	The one money path — BR-SHL-023
`RowActions`	Actions	`Approve` inline, then `…` . ⚠ builds no overflow — P1.17
`CountBadge`	the view label	The pending count — P2.15

FOR	The second person in a two-person write-off: the approver.
DOORS	**Three** — the **Write-offs · 2** view · the Outstandings write-off result · the Dashboard to-do
PRIMARY ACTION	None at page level — **Approve** is on each requested row
FIELDS	Account · Amount · Reason · Requested · By · State · Actions
TABLE CONTROLS	All six — **only the state filter is supported today**
WRITE SURFACES	**Approve** 2·4a · **Reject** · **Reverse** an approved one. **Raised in Payments, never here** — BR-PAY-020
TODAY	TODAY A panel on the board with Approve inline, no toolbar, no paging
TASK	TASK P2.19 search, filters and the count PAYMENTS · P1.8 paging
WALKED	**YES** — as a panel on the board
RULES	Obeys B14 4 · 6 · 31 · BR-PAY-020 · BR-PAY-037. Breaks ✗ NFR-3 until P2.19

2·5 Refunds `/:tenant/finance/revenue?view=refunds` UPDATE ① THE LIST

OWNED BY PAYMENTS · REUSED HERE, NOT REBUILT

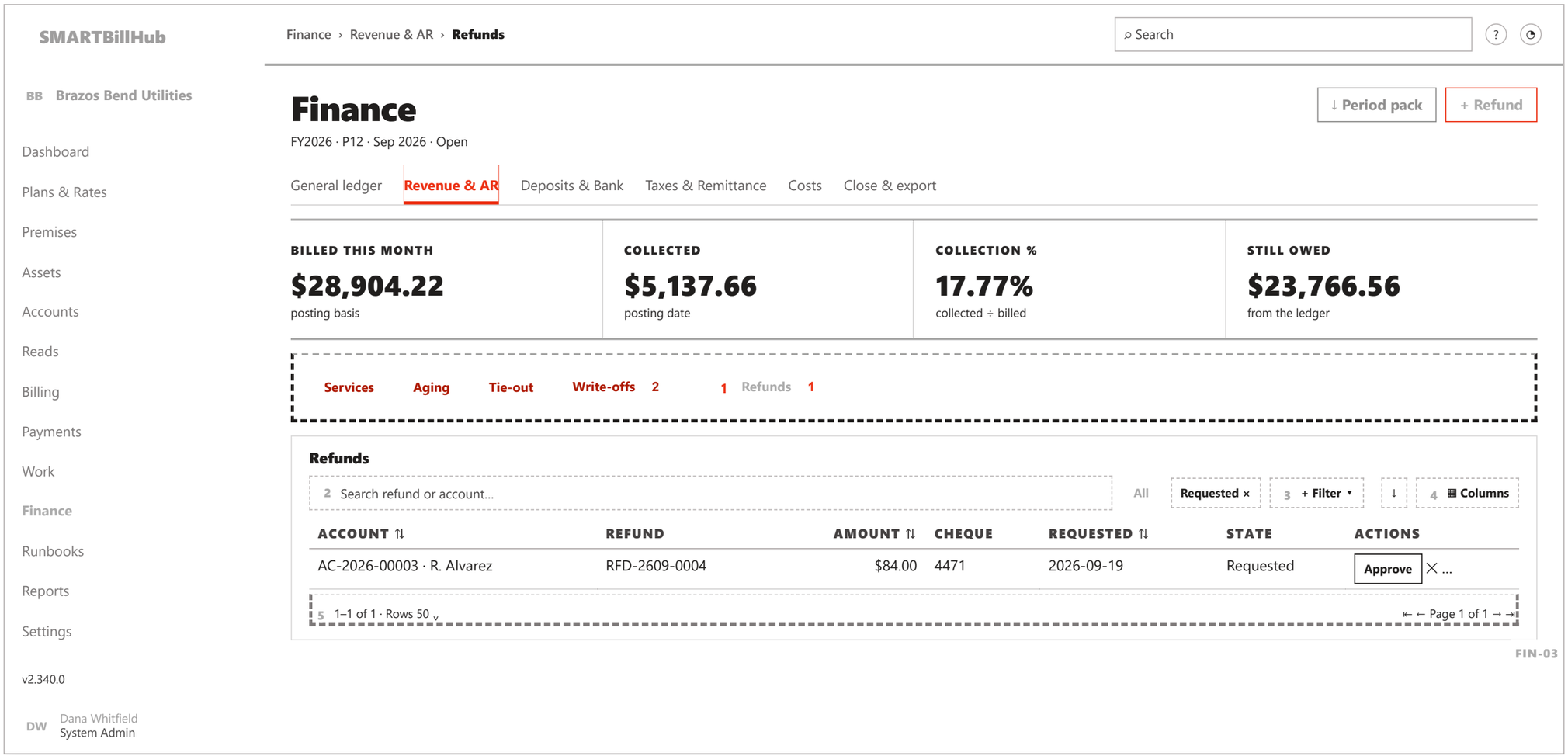
TODAY **This is not Finance code.** It is `07-frontend/src/features/payments/RefundsPage.tsx` over `06-backend/apps/payments/` , shown as a view under Revenue & AR. **Mount the component; do not rebuild it.** Any change is a Payments change. **It is today's refunds screen**, routed at `/finance/refunds` since the strip went to six tabs (PO 2026-09-03, *"C25 says six tabs. - this is correct."*) — **reused as a view, not redrawn**. `/finance/refunds` **redirects here**; it is never deleted, because the links are in circulation.

OPEN `OQ-PAY-1514` PROPOSED **— do the refund screens belong in Finance at all?** The PO is reviewing every refund surface in Payments. **If not**, this view and its four acts leave Finance, and the redirect points back to Payments.

NO PAGING ON PURPOSE — AND WHY THAT HAS TO CHANGE

GAP The list reads **exactly one filter,** `f.status` , and its own code says: "*Unpaginated, deliberately … a utility under 3,000 connections has a queue a person reads in one screen, and a page of an approval queue is a queue that lies about what is pending.*"

The objection is real and the ceiling is not. The product now serves utilities up to **40,000 accounts**, and the project rules forbid an unpaginated list — the same excuse the `/outstandings` defect names. **The answer to "a page lies about what is pending" is the count in the view label**: *Refunds · 1* states the true total on every page, so paging can no longer hide anything. **P2.19** serves that count beside the page.



END STATE · DASHED = NEW · TODAY'S REFUNDS SCREEN AS A VIEW

1 **The view strip** · five views under Revenue & AR, switched by `?view=` . **The KPI strip sits above it on every view**, and the pending counts ride in the labels · NEW · P2.18

2 **Search** · UI-RULES 12 · NEW

3 **+ Filter** ▾ · the criteria below · UI-RULES 12 · NEW

4 ▦ **Columns** · the shared `ColumnChooser` , persisted per user · UI-RULES 14 · NEW

5 **Server paging** · without it a clerk sees 50 rows and no more · UI-RULES 16 · NFR-3 · **P1.8**

Filter criteria

CRITERION	PARAMETER	VALUES	SOURCE	SERVED?
State	`f.status`	Requested · Approved · Issued · Cleared · Void	chip values are a literal array — `RefundsPage.tsx:198` — P1.5	SERVED
Method	`f.method`	Cheque · ACH	`REFUND_METHODS` — a literal too	NOT SERVED
Requested	`from · to`	a date range	—	NOT SERVED
Search	`q`	refund number, account or cheque	—	NOT SERVED

Components to use

COMPONENT	WHERE	WHY THIS ONE
`DataTable`	the list	Brings toolbar, chooser, pager and empty state together — the same table as tab 1
`FilterBar · ChipMultiSelect`	chips `+ + Filter ▾`	Removable chips, *All* as default — B14 rule 9
`ColumnChooser · DownloadButton`	▦ · ↓	The shared kit
`KPIStrip`	above the view strip	⚠ P2.1 — `.kpi .v` ellipsises a ten-digit total at 146px
`MoneyCell · DateCell · StatusPill`	cells	The one money path — BR-SHL-023
`RowActions`	Actions	Approve · Issue · Clear · Void, each gated by status and rendered disabled with its reason
`CountBadge`	the view label	The pending count — P2.15

FOR	A finance officer returning a customer's credit.
DOORS	**Four** — the **Refunds · 1** view · a `/finance/refunds` bookmark, **redirected** · the Payments bulk action's *open refund queue* link (`paymentsBulk.tsx:858`) · 2·5d's back link
PRIMARY ACTION	`+ Refund` — opens 2·5b
FIELDS	Account · Refund · Amount · Cheque · Requested · State · Actions
TABLE CONTROLS	All six — **only the state filter is supported today**
WRITE SURFACES	**Raise** 2·5b · **Approve** 2·5a · **Issue · clear · void** 2·5c · **Detail** 2·5d — **NEW**
TODAY	TODAY A full screen at `/finance/refunds` , reached through a link on the board
TASK	TASK P2.19 search, filters and the count PAYMENTS · P1.8 paging · P1.3 onto the shared table · P1.5 chips from the server
WALKED	**YES**
RULES	Obeys B14 4 · 6 · 31 · BR-PAY-020. Breaks ✗ NFR-3 until P2.19

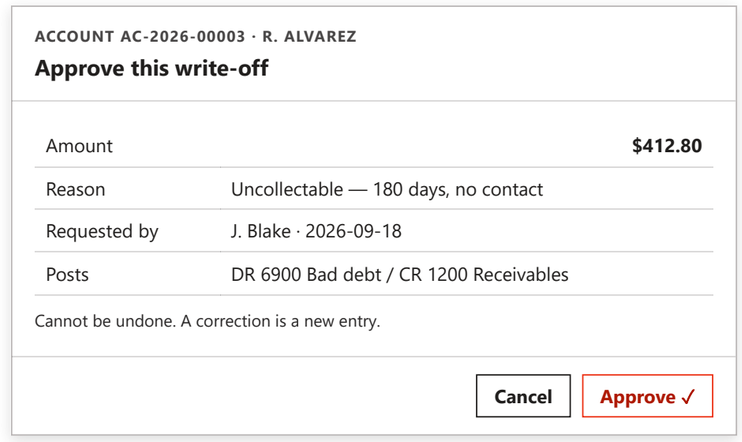
2·4a Approve write-off

`modal` KEEP ③ WHAT YOU CAN DO

Opens from · the `Approve` control on a **requested** row of **2·4 Write-offs**. An approved row offers `Reverse` instead, and a rejected one offers nothing

OWNED BY PAYMENTS · REUSED HERE, NOT REBUILT

TODAY **This is not Finance code.** It is `07-frontend/src/features/payments/WriteOffQueuePage.tsx` over `06-backend/apps/payments/` , shown as a view under Revenue & AR. **Mount the component; do not rebuild it.** Any change is a Payments change.



AS BUILT · FIGURES ILLUSTRATIVE · DANGER TIER, AND THE CODE RENDERS DANGER — WRITE-OFF IS NAMED IN B14 RULE 6

FOR	An admin or executive giving up a receivable.
DOORS	The `Approve` control on **2·4 Write-offs**
PRIMARY ACTION	`Approve ✓` — **danger tier**, per B14 rule 6, which names write-off explicitly
FIELDS	Account · Amount · Reason · Requested by · the pair it will post
TABLE CONTROLS	Not a list
WRITE SURFACES	**Update** `POST /write-offs/{id}/approve → payments/writeoffs.py`
TODAY	TODAY Bad debt is a **direct** write-off at approval (`BR-LED-008` , narrowed to uncollectable debt). A **concession** credit debits a separate account — `BR-LED-022` , OPEN **awaiting a finance re-sign**
TASK	None
WALKED	**NO** — the queue was empty on the fixture
RULES	Obeys B14 6 · hard rules 4 · 7 · `BR-PAY-020`

2·5a Approve refund

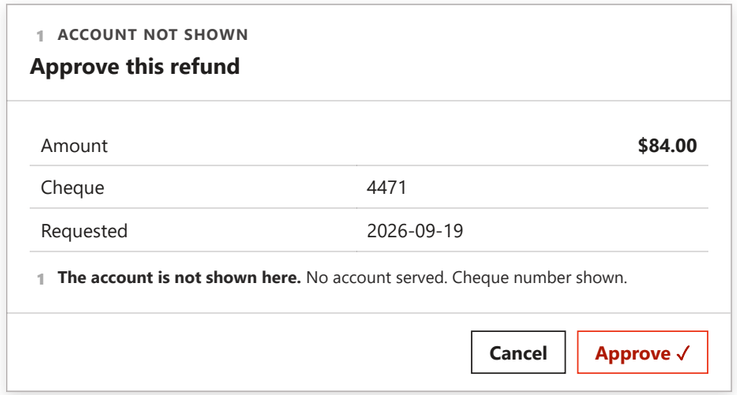
`modal` KEEP ③ WHAT YOU CAN DO

Opens from · the `Approve` row action on a **requested** refund in **2·5 Refunds**. **The board panel that could not approve is gone** — the Refunds view carries the action itself

OWNED BY PAYMENTS · REUSED HERE, NOT REBUILT

TODAY **This is not a Finance screen.** It is `07-frontend/src/features/payments/RefundsPage.tsx` , rendered at a Finance address. The file's own header records why: "*THIS WAS AN EIGHTH FINANCE TAB AND IT IS NOT ONE NOW*" — PO ruling 2026-09-03, "*C25 says six tabs. - this is correct.*" **The page is RE-HOMED, not withdrawn**; `/finance/refunds` stays routed for links already in circulation, and the door is `RefundsQueuePanel` inside Revenue & AR.

Nothing here is to be built. It exists, it is tested, and it belongs to Payments. This document draws it so the Finance flow reads end to end — **not as work**. Any change to it is a **Payments** change.



AS BUILT · THE APOLOGY AT ① IS THE PRODUCT'S OWN WORDS

1 **No account** · four reads exist and none is called — `GET /refunds/{id}` , `/refunds-and-credits` , `/accounts/{id}/credit-sources` , `/credit-path` . **An approver decides without seeing whose money it is** · CHANGED · P3.3

FOR	A System Admin approving money going back out.
DOORS	The `Approve` row action on **2·5 Refunds**
PRIMARY ACTION	`Approve ✓` — danger tier
FIELDS	Amount · Cheque · Requested. **Not** the account
TABLE CONTROLS	Not a list
WRITE SURFACES	**Update** `POST /refunds/{id}/approve` — `SystemAdminOnly`
TODAY	TODAY The panel hardcodes `status='requested'` at `RefundsQueuePanel.tsx:120`
TASK	TASK P3.3
WALKED	**NO** — queue empty
RULES	Obeys B14 6 · hard rule 4. Breaks *X* B14 32 — an absence stated on a surface that should simply carry the fact

2·5b Raise a refund

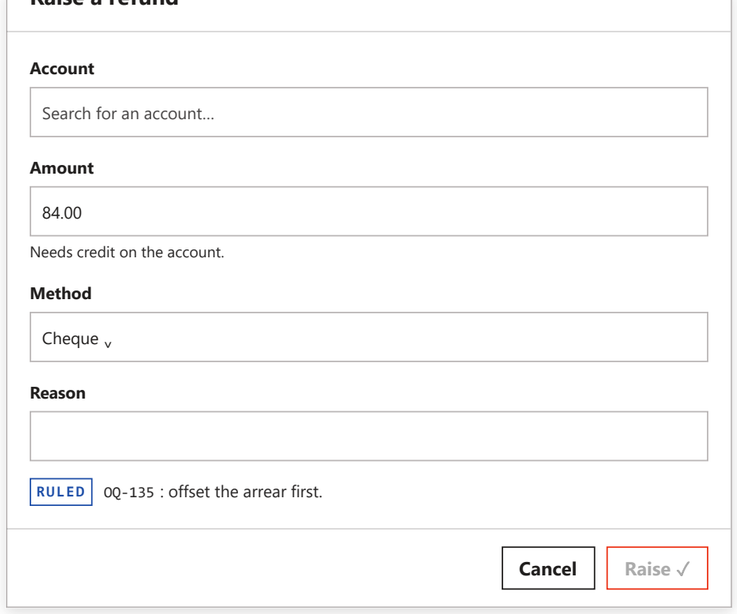
`modal` KEEP ③ WHAT YOU CAN DO

Opens from · `+ Refund` in the page head of **2·5 Refunds** · **and the Payments bulk action**, which calls the same `requestRefund` (`paymentsBulk.tsx:623`) — **two doors, one endpoint**

OWNED BY PAYMENTS · REUSED HERE, NOT REBUILT

TODAY **This is not a Finance screen.** It is `07-frontend/src/features/payments/RefundsPage.tsx` , rendered at a Finance address. The file's own header records why: "*THIS WAS AN EIGHTH FINANCE TAB AND IT IS NOT ONE NOW*" — PO ruling 2026-09-03, "*C25 says six tabs. - this is correct.*" **The page is RE-HOMED, not withdrawn**; `/finance/refunds` stays routed for links already in circulation, and the door is `RefundsQueuePanel` inside Revenue & AR.

Nothing here is to be built. It exists, it is tested, and it belongs to Payments. This document draws it so the Finance flow reads end to end — **not as work**. Any change to it is a **Payments** change.



AS BUILT

FOR	A billing clerk starting a refund from credit the account actually holds.
DOORS	`Raise a refund` on 2·5
PRIMARY ACTION	`Raise ✓` — `AdminOrClerk`
FIELDS	Account · Amount · Method · Reason
TABLE CONTROLS	Not a list
WRITE SURFACES	**Add** `POST /refunds` → `payments/refunds.py`
TODAY	TODAY Only from real credit (`US-PAY-011`). RULED `OQ-132` an account credit **may** go negative
TASK	None
WALKED	**NO**
RULES	Obeys B14 4 · hard rule 4 · `OQ-132` · `OQ-135`

2·5c Issue · clear · void

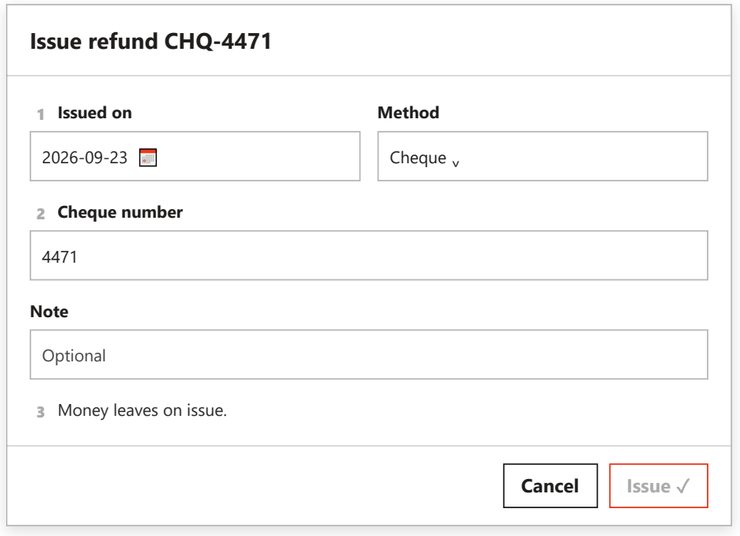
`modal` KEEP ③ WHAT YOU CAN DO

> **Opens from** · the row actions on **2·5 Refunds**, gated by status — `Issue` on approved, `Clear` on issued, `Void` on issued or cleared. Each renders disabled with its reason when the status is wrong

OWNED BY PAYMENTS · REUSED HERE, NOT REBUILT

TODAY **This is not a Finance screen.** It is `07-frontend/src/features/payments/RefundsPage.tsx` , rendered at a Finance address. The file's own header records why: "*THIS WAS AN EIGHTH FINANCE TAB AND IT IS NOT ONE NOW*" — PO ruling 2026-09-03, "*C25 says six tabs. - this is correct.*" **The page is RE-HOMED, not withdrawn**; `/finance/refunds` stays routed for links already in circulation, and the door is `RefundsQueuePanel` inside Revenue & AR.

Nothing here is to be built. It exists, it is tested, and it belongs to Payments. This document draws it so the Finance flow reads end to end — **not as work**. Any change to it is a **Payments** change.



AS BUILT · **ONE MODAL, THREE VERBS** — THE TITLE AND THE DATE LABEL CHANGE WITH THE ACT

1 **The date** · *Issued on* · *Cleared on* · *Voided on*. **Never taken from the clock** — `BR-TEN-043` · EXISTS

2 **Cheque number** · on issue only. It becomes the refund's reference, and it fills the **Cheque** column on 2·5 Refunds · EXISTS

3 **Consequence** · four words. Void swaps it for "*This cannot be undone.*" and a required reason · EXISTS

CORRECTION TO THE PREVIOUS VERSION OF THIS DOCUMENT

This screen had no drawing at all — a spec table and a commentary box, nothing to build from. **B14 rule 42 makes a drawing the specification and extracted text only an index**, so a screen listed without one is a screen this document does not actually specify. Drawn now from `RefundsPage.tsx:204-300` . **5·3b had the same hole and is fixed in the same pass**; those were the only two of 29.

ACT	ROLE GATE	ENDPOINT	WHAT IT MEANS
Approve	`SystemAdminOnly`	`POST /refunds/{id}/approve`	The money is agreed but has not left
Issue	`AdminOrClerk`	`POST /refunds/{id}/issue`	A cheque is cut or an ACH is sent
Clear	`AdminOrExecutive`	`POST /refunds/{id}/clear`	The bank confirms it left
Void	`SystemAdminOnly`	`POST /refunds/{id}/void`	Cancelled before it left

FIVE WRITES, THREE DIFFERENT GATES, ONE QUEUE

TODAY Raise and issue are `AdminOrClerk` ; approve and void are `SystemAdminOnly` ; clear is `AdminOrExecutive` . That is a deliberate separation of duties and it is correct — **recorded here because it is invisible on the screen**, where all four look like row actions of the same kind.

FOR	Three different roles, each owning one step of the lifecycle.
DOORS	Row actions on 2·5
PRIMARY ACTION	Varies by state; each is a `Verb ✓` commit
FIELDS	Reason on void; cheque number on issue; clearing date on clear
TABLE CONTROLS	Not a list

WRITE SURFACES	**Update** ×4, as above. **No delete, ever**
TODAY	TODAY All four live in `refunds_api.py`
TASK	None
WALKED	**NO**
RULES	Obeys hard rules 4 · 7

2·5d Refund detail

`/finance/refunds/:id` NEW ② ONE ROW OPENED

SMARTBillHub

Finance › Revenue & AR › Refunds › **RFD-2609-0004**

⌕ Search ? ◔

BB Brazos Bend Utilities

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

v2.340.0

DW Dana Whitfield
System Admin

REFUND RFD-2609-0004 · CHEQUE

$84.00 to R. Alvarez

Approve ✓

Requested | Cheque | Overpayment

General ledger | Revenue & AR | Deposits & Bank | Taxes & Remittance | Costs | Close & export

1 WHOSE MONEY

Account	**AC-2026-00003 · R. Alvarez**
Premises	**105 Cedar Ridge Way · Unit 1**
Credit on the account	**$84.00**
Other accounts in arrear	**None**

2 WHERE IT CAME FROM

Overpayment	**$84.00**
2026-09-12 · on PY-2609-0041	

3 HISTORY

WHEN	WHAT	WHO
2026-09-19	Raised	J. Blake

FIN-03

PROPOSED · THE WHOLE SCREEN IS NEW · FIGURES ILLUSTRATIVE

1 **Whose money** · `GET /accounts/{id}/credit-sources` — **built, zero callers** · NEW · P3.3
2 **Credit path** · `GET /accounts/{id}/credit-path` — **built, zero callers** · NEW
3 **History** · `GET /refunds/{id}` — **built, zero callers** · NEW

> **THE REPO ID IS NOT INVENTED**
>
> GAP This screen is drawn as **`S-NEW`** . Allocating a real number is a governance act through `10-scripts/allocate.py` against the remote, not a drawing decision.

FOR	An approver who needs to see whose money is going back out before agreeing to it.
DOORS	**NOTHING today.** After P3.3: the refund number on **2·5 Refunds**
PRIMARY ACTION	`Approve ✓` — danger tier
FIELDS	Refund no. · Amount · State · Account · Premises · Credit held · Other arrears · credit path · history
TABLE CONTROLS	Two short tables bounded by one refund — no pager, and the exemption is stated
WRITE SURFACES	**Detail** this screen · **Update** the four lifecycle acts
TODAY	GAP Four reads exist and nothing calls them
TASK	TASK P3.3 · P3.5 door asserted by address
WALKED	**NO** — does not exist
RULES	Will obey B14 2 · 6 · 31 · 40

④ What each act changes, and where — Tab 2

ACT	WRITES	THE LEDGER	ELSEWHERE
Approve a write-off 2·4a	`write_offs` status · `audit_log`	**Posts at approval** — DR bad debt / CR receivables (`BR-LED-008` , direct method)	1·2 trial balance moves · 2·3 Tie-out recomputes · the account balance drops · **a live payment plan is closed** (`OQ-171`) · Outstandings loses the row
Approve a refund 2·5a	`refunds` status · `audit_log`	**Nothing yet** — money is agreed, not moved	2·5 state changes; nothing else moves until Issue
Raise a refund 2·5b	`refunds` · `audit_log`	Nothing	2·5 Refunds gains a row · offsets first against another account's arrear (`OQ-135`)
Issue a refund 2·5c	`refunds` · cheque number	**DR customer credit / CR cash**	3·5 reconciliation statement gains an **outstanding cheque** (`CS-14`) · the customer receives a cheque
Clear a refund 2·5c	`refunds` · clearing date	Nothing new — the pair posted at Issue	3·5 the outstanding cheque clears · the bank tie-out closes

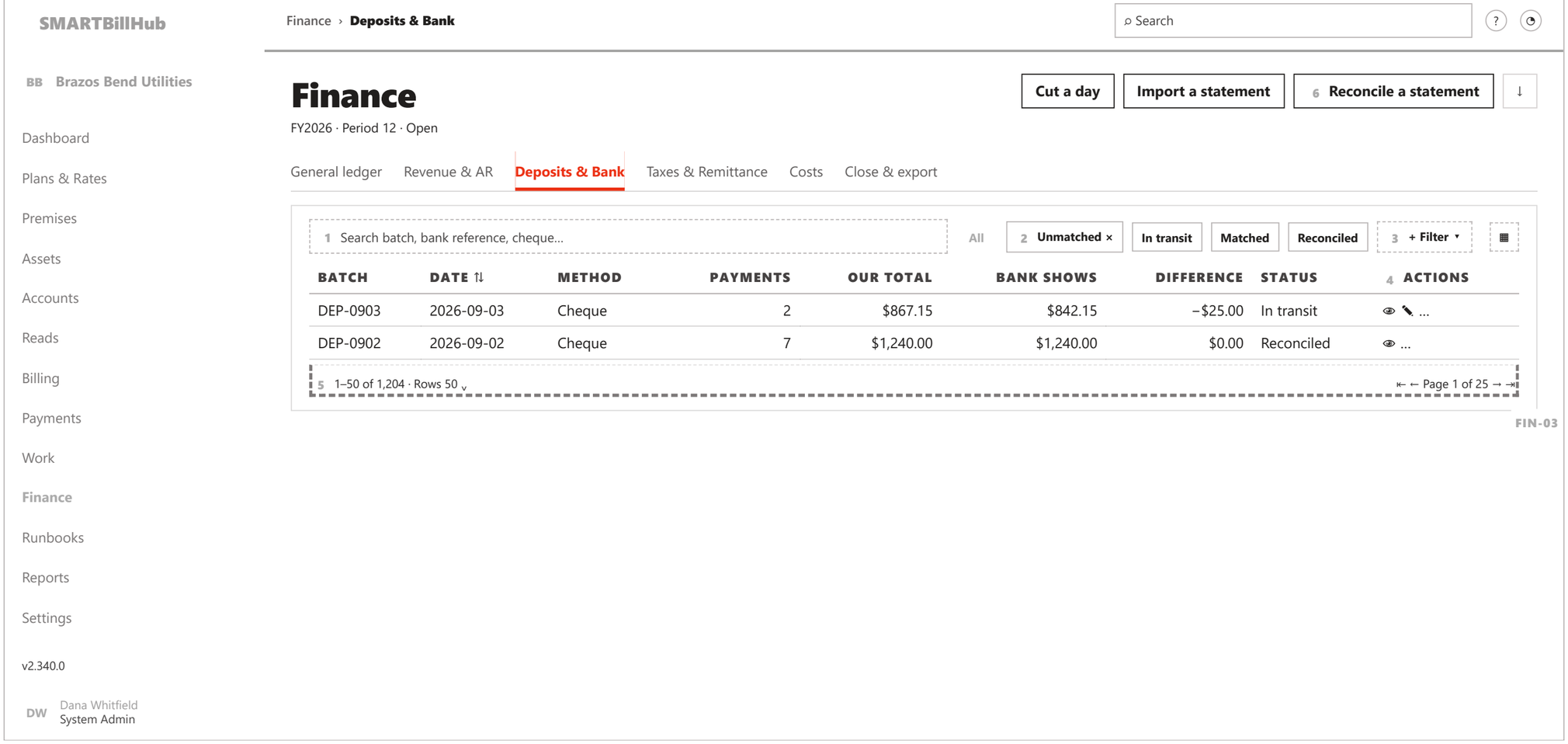
Tab 3 · Deposits & Bank

Six screens, **2,940 lines of front end over ~2,700 lines of tested reconciliation service**, all hanging off one list. This is where the door audit matters most: two of the six are reached through exactly one control each.

3·1 Deposits `/finance/deposits` UPDATE ① THE LIST

OWNED BY PAYMENTS · REUSED HERE, NOT REBUILT

TODAY **This is not a Finance screen.** It is `07-frontend/src/features/payments/DepositsPage.tsx` , and its endpoints are in `06-backend/apps/payments/` . **Every screen in tab 3 is Payments code** shown at a Finance address. **Nothing here is Finance's to build**, and any change is a Payments change — front end, backend and tables alike.



END STATE · DASHED = NEW · FIGURES ILLUSTRATIVE BEYOND THE WALKED EMPTY STATE

1 **Search** · **NEW READ** — the endpoint takes no `q`. Copy the cost register's shape: minimum-length guard, term in the URL · UI-RULES 12 · P1.7

2 **Status chips** · wired server-side but **local** `useState` today — unlinkable, lost on refresh · CHANGED · P1.4 · P1.9

3 **+ Filter** · exposes `f.method`, already accepted by the endpoint and the typed client, **with no control** · NEW

4 **Row actions as icons** · text links today · UI-RULES 15 *X* · CHANGED

5 **Pager** · `bankrec_read.py:316` materialises every batch and its docstring still cites the **3,000-connection ceiling that moved to 40,000** · UI-RULES 16 · NFR-3 · NEW

6 **Reconcile a statement** · **the only door to 3·5 in the entire product** · EXISTS

Filter criteria

CRITERION	PARAMETER	VALUES	SOURCE	SERVED?
State	`f.status`	Unmatched · In transit · Matched · Reconciled	chip values are a literal array — `DepositsPage.tsx:103-106` — P1.4	SERVED
Method	`f.method`	the tenant's tenders	`GET /deposit-batches`	SERVED
Search	`q`	batch code, bank reference, cheque	— · **P1.7**	NOT SERVED
Day	`from · to`	a date range	—	NOT SERVED

Components to use

COMPONENT	WHERE	WHY THIS ONE

`DataTable · ListPager`	batches	Server paging with P1.8
`FilterBar · ChipMultiSelect`	state · method	Chips from the server once P1.4 lands
`KPIStrip`	the tiles	*In transit* reads *pending*, never $0.00 — the one thing this screen must not get wrong
`DownloadButton`	deposit slip	Disabled with its reason until P4.6
`MoneyCell · DateCell · StatusPill`	cells	The shared kit

FOR	A clerk banking the day's takings and matching them to what the bank actually received.
DOORS	**Eleven** — the tab strip twice, plus every back link and denied/error state from its three children. The best-connected sub-route in Finance
PRIMARY ACTION	`Cut a day` — creates the batches; everything else reads
FIELDS	Batch · Date · Method · Payments · Our total · Bank shows · Difference · Status
TABLE CONTROLS	Search **none** · status wired but local · method unwired · chooser **none** · sort SQL `-deposited_on` (**passes R14**) · **pagination none**
WRITE SURFACES	**Detail** row → 3·2 · **Add** `Cut a day` · **Edit / Update** NOTHING here — reconciliation happens on the batch
TODAY	`TODAY` Walked: **0 tables** on the seeded fixture — the empty state reads *"No deposit batches yet. Cut a day's takings once payments have cleared."*
TASK	`TASK` P1.2 · P1.4 · P1.7 · P1.8 · P1.9
WALKED	**YES**
RULES	Obeys R14 · UI-RULES 17. Breaks ✗ UI-RULES 12·13·14·**15**·16 · ✗ NFR-3

3·2 Batch `/finance/deposits/:batchId` `UPDATE` ② ONE ROW OPENED

OWNED BY PAYMENTS · REUSED HERE, NOT REBUILT

`TODAY` **This is not a Finance screen.** It is `07-frontend/src/features/payments/DepositBatchPage.tsx`, and its endpoints are in `06-backend/apps/payments/`. **Every screen in tab 3 is Payments code** shown at a Finance address. **Nothing here is Finance's to build**, and any change is a Payments change — front end, backend and tables alike.

THE FINDING: A DETAIL SCREEN WITH NOTHING YOU CAN DO ON A LINE

`GAP` **Three `PlainTable`s and one page-level button.** `DepositBatchPage` renders the batch's payments, its bank line and its resolutions through `PlainTable` (`:163`, `:250`, `:343`) — the bare table with **no toolbar, no pager, no row actions and no empty state**. The only control on the screen is in the page head, and which one you get depends on state: `Unreconcile` when locked, `Resolve difference` when there is a bank line, and **a sentence instead of a button** when there is not.

Some of that is correct and must stay. A payment inside a cut batch cannot be removed — the day's total would change after the fact — and hard rule 4 forbids editing it. **So "edit this line" is rightly absent.**

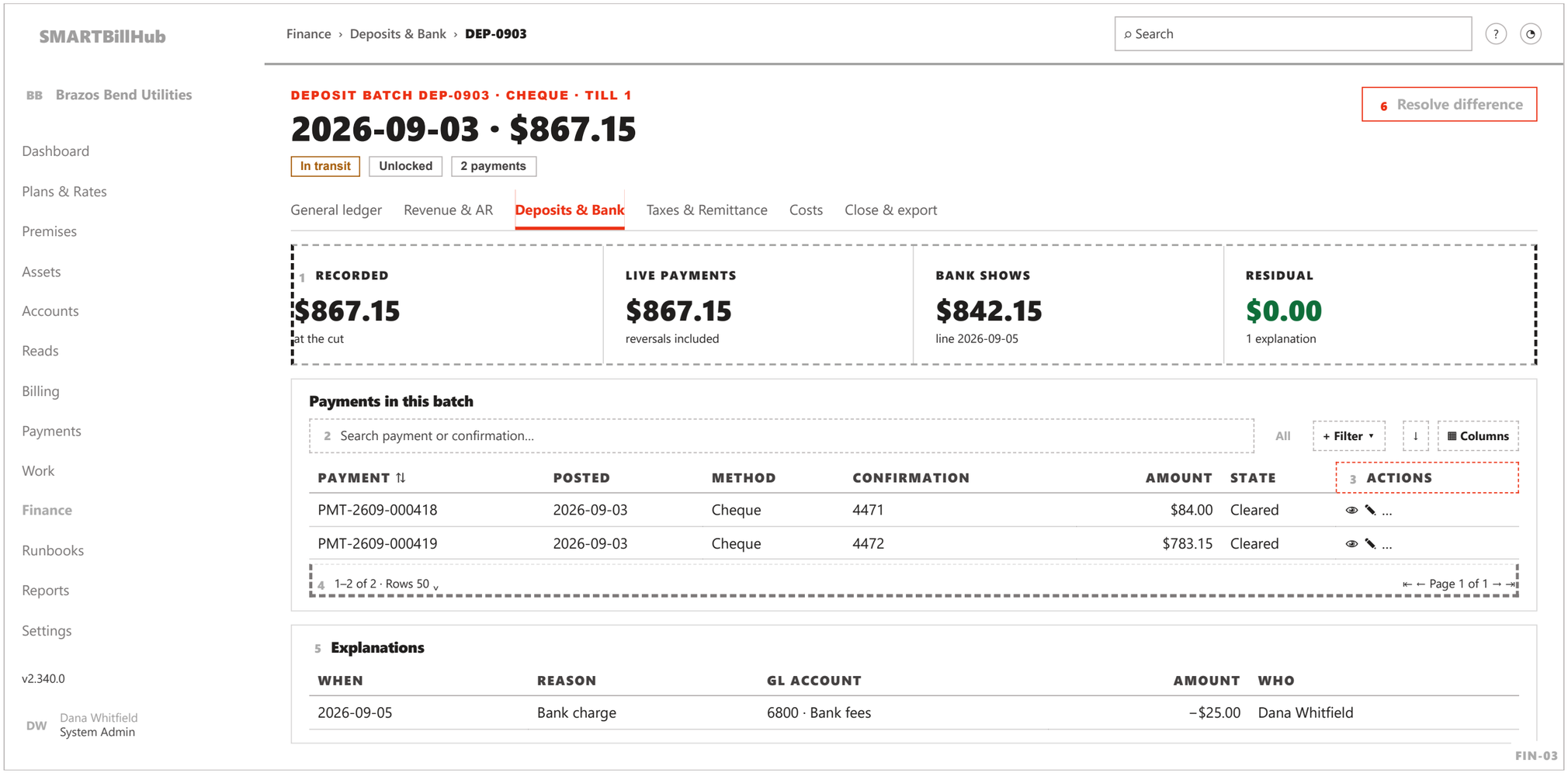
But three lawful acts are missing, and two already have endpoints: (a) 👁 open the payment — you cannot navigate from a batch line to the payment record it names, which is the first thing a clerk wants when a figure looks wrong. **(b) ✎ correct the reference** — `POST /payments/{id}/correct-reference` permits `confirmation` and `memo` **in any status, banked included** (hard rule 4 as narrowed 2026-09-02); only the tender is refused once banked. **This screen is exactly where a mistyped cheque number is discovered**, because it is where you sit matching against the bank. **(c) ↺ reverse** — `_refuse_if_batch_locked` deliberately permits a reversal inside a locked batch.

P3.10. Row actions on the payments table: 👁 ✎ then … carrying ↺. Two visible plus overflow, UI-RULES 15.

AND ONE CONSEQUENCE NOTHING SURFACES

A reversal inside a **locked** batch is lawful, so a locked batch can end up differing from the sum of its live payments — and **no screen says so**. The batch shows its frozen `recorded_total`. **P3.11** adds a

stated difference on this screen; it is a read, not a rule change.



END STATE · DASHED = NEW · FIGURES ILLUSTRATIVE (B14 RULE 37)

1. **Batch facts as a KPI strip** · **Live payments** is the new tile: it equals *Recorded* until a payment inside a **locked** batch is reversed, and then it says so instead of hiding it · NEW · P3.11
2. **Search, + Filter, Columns, download** · in the browser — the batch detail takes no parameters · NEW
3. **Row actions** · 👁 open the payment · ✎ correct the cheque or confirmation number (allowed even once banked) · … carries ↺ reverse, which is allowed inside a locked batch · NEW · P3.10
4. **Pager** · a busy till-day can hold hundreds of payments; batch members page on the server with P1.8 · NEW
5. **Explanations** · the reasons and GL accounts recorded against the difference · EXISTS
6. **Resolve difference** · opens 3·4. Shown only while the batch is unlocked and has a bank line; locked, the head offers *Unreconcile* instead · EXISTS

A REVERSAL INSIDE A LOCKED BATCH IS ALLOWED, AND THE REASONING IS RIGHT

TODAY `_refuse_if_batch_locked` permits it: the batch total stays because *"the bank statement the batch was agreed against did not change."* New rows are written instead. Same shape as `OQ-134` 's bounced cheque — RULED *the lock HOLDS*. **The consequence nobody surfaces:** the batch then differs from the sum of its live payments, which is correct and invisible.

Filter criteria

CRITERION	PARAMETER	VALUES	SOURCE	SERVED?
Search	`q`	payment number or confirmation	the batch detail reads no parameters	NOT SERVED in the browser

State	`f.status`	Cleared · Reversed	DERIVED from the members	NOT SERVED in the browser

Components to use

COMPONENT	WHERE	WHY THIS ONE
`KPIStrip`	batch facts	Four figures, and the fourth tells the truth when a locked batch drifts — P3.11
`DataTable` · `ListPager`	payments	The shared table; members page with P1.8
`RowActions`	Actions	👁 ✎ then … with ↻. ⚠ It builds no overflow today — P1.17
`FormModal`	✎ correct reference	`POST /payments/{id}/correct-reference` — cheque number and memo only; the tender is refused once banked
chips · kicker · `Panel`	the head	The detail-page pattern from the Accounts blueprint
`MoneyCell` · `DateCell` · `StatusPill`	cells	The shared kit

FOR	A clerk making one day's deposit tie to the bank.
DOORS	**Four**, all row-level and correct for a `:param` route — two on 3·1's table, two on 3·3b's matched rows
PRIMARY ACTION	`Reconcile ✓` — refused by the server until the resolutions explain the whole difference, then the batch **LOCKS**
FIELDS	Payment · Posted · Method · Confirmation · Amount · then per resolution: When · Reason · GL account · Amount · Who
TABLE CONTROLS	Two lists, **neither paginated**. Members sort `posted_at asc` — oldest first, which fails R14 but is right for a deposit slip. **Flagged, not condemned**
WRITE SURFACES	**Detail** this screen · **Add** resolve · **Update** reconcile / unreconcile · **Edit** NOTHING
TODAY	TODAY Unreconciling **posts nothing** (`BR-LED-024`) — the cash stays in the books
TASK	TASK P1.8. The reversed-member view is a **gap with no task**
WALKED	**NO** — no batch existed on the fixture
RULES	Obeys hard rule 4 · `BR-LED-024` · `BR-PAY-004` . Breaks ✗ UI-RULES 16 · ✗ NFR-3

3·3a Cut a day

`modal` KEEP ③ WHAT YOU CAN DO

Opens from · `Cut a day` in the page head of **3·1 Deposits**. The day list is `cuttable_days` — **only days that actually hold unbatched cleared money**

OWNED BY PAYMENTS · REUSED HERE, NOT REBUILT

TODAY **This is not a Finance screen.** It is `07-frontend/src/features/payments/DepositsPage.tsx` , and its endpoints are in `06-backend/apps/payments/` . **Every screen in tab 3 is Payments code** shown at a Finance address. **Nothing here is Finance's to build**, and any change is a Payments change — front end, backend and tables alike.

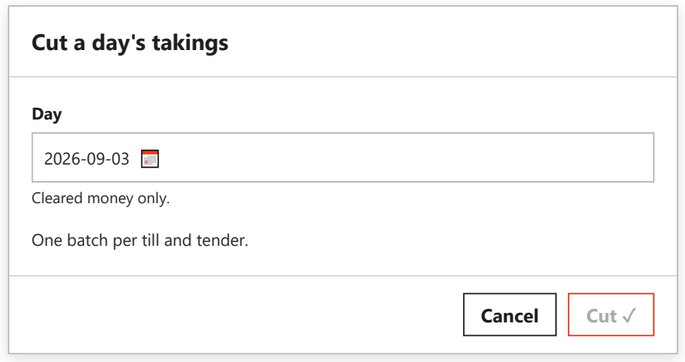
WHY THIS EXISTS AT ALL — THE QUESTION PUT DIRECTLY

Because the bank credits deposit slips, not payments. A teller takes the day's cash and cheques to the bank as a physical deposit; the statement then shows **one line per slip**. Reconciliation matches your side against the bank's side, and with **1,204 payments against roughly 30 bank lines** there is nothing to match unless the payments are first grouped the way the money physically moved. **Cutting is that grouping** — `cut_deposit_batches` puts one drawer day's `cleared` money into one batch per **till × tender** (`BR-PAY-025`), which is exactly how the slips are made up: a till's cash in one bag, its cheques in another, two bank lines.

Why a person presses it rather than a timer. `BR-TEN-043` forbids the server reading a clock, so there is no "today" to run on. The control offers **the days that actually hold unbatched cleared money** and the clerk picks one — and **an empty list is the honest "everything is banked"**, which a *Cut today* button could never say. The cut is also a declaration: after it, that day is closed and a late

payment joins the next one. A timer firing at 5pm would silently strand a payment taken at 4:59.

What it does not do: it moves no money and posts nothing to the books. It is a grouping, and the deposit itself was already recorded when each payment cleared.



AS BUILT

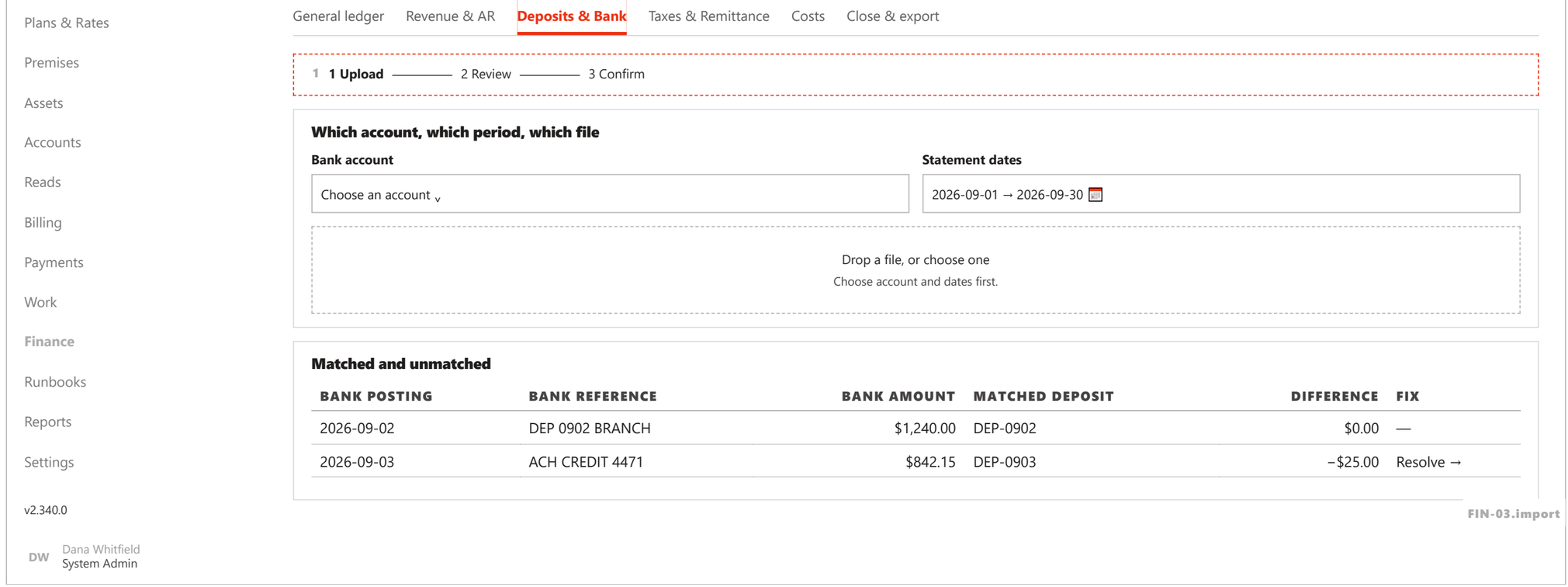
FOR	A clerk banking a day.
DOORS	`Cut a day` on 3·1
PRIMARY ACTION	`Cut ✓ — AdminOrExecutive`
FIELDS	Day
TABLE CONTROLS	Not a list
WRITE SURFACES	**Add** `POST /deposit-batches/cut → reconciliation.cut_deposit_batches:735`
TODAY	TODAY Takes **cleared payments only** (`:743`) — which is why a cancelled payment can never be in a batch: cancel works on `processing` alone. **The two states are disjoint by construction**
TASK	None
WALKED	**NO**
RULES	Obeys `BR-PAY-025` · hard rule 6 idempotency

3·3b Import a statement

`/finance/deposits/import` **UPDATE** ③ **WHAT YOU CAN DO**

OWNED BY PAYMENTS · REUSED HERE, NOT REBUILT

TODAY **This is not a Finance screen.** It is `07-frontend/src/features/payments/BankImportWizard.tsx` , and its endpoints are in `06-backend/apps/payments/` . **Every screen in tab 3 is Payments code** shown at a Finance address. **Nothing here is Finance's to build**, and any change is a Payments change — front end, backend and tables alike.



END STATE · THE RAIL AT ① IS NEW; TODAY THERE IS NO RAIL AT ALL

1 **The shared wizard rail** · numbered, clickable when complete, with a forward fence, a working state and one transactional commit · `BR-SHL-018` · `BR-SHL-019` · NEW

★ **Title** · *Import a statement.* The product's title is a ten-word sentence — *"Import a bank statement and match it to your deposits"* — and B14 rule 33 caps a heading at three words. `payments.deposits.wizardTitle` , a Payments string · COPY CUT · P1.19

> **CORRECTION — AND THE ONE REAL WIZARD IN FINANCE IS HAND-ROLLED**
>
> GAP The previous version of this document drew this as **two screens with a step strip**. It is **one page**: step 1 always renders, step 2 is a conditional panel *below* it (`{statementId !== null && …}`). There is no rail, no fence and no shared frame.
>
> **No Finance or Payments file imports `WizardFrame` .** Twenty files across tenancy, assets, metering, rating and customers use it; this one — named `BankImportWizard.tsx` — builds its steps as two `<h3>` headings. Drawing a rail over it would have said "this is the shared frame" while showing something else.

Filter criteria

CRITERION	PARAMETER	VALUES	SOURCE	SERVED?
Bank account	`account`	the tenant's bank accounts	`GET /bank-accounts`	SERVED
Match	`f.match`	Matched · Suggested · Unmatched	DERIVED — one statement at a time	NOT SERVED in the browser
Search	`q`	reference or amount	`GET /bank-statements/{id}` reads no parameters	NOT SERVED in the browser

Components to use

COMPONENT	WHERE	WHY THIS ONE
`WizardFrame`	the two steps	P1.14 — today the steps are two headings
`FileDrop`	the upload	Bank CSV or OFX
`DataTable`	the match table	Batch code as a column, so both sides read together — P1.7

`StatusPill · MoneyCell · DateCell`	cells	The shared kit	

FOR	A clerk bringing the bank's file in and seeing what matched.
DOORS	**Two, both on 3·1** — the header action (unconditional) and an inline link that appears only when there are unmatched lines
PRIMARY ACTION	`Upload ↑` , then the frame's single commit
FIELDS	Bank account · statement dates · file · then per line: posting · reference · amount · matched deposit · our amount · difference
TABLE CONTROLS	The review table is a **wizard preview — UI-RULES 16 exempts it**. Stated so nobody logs it
WRITE SURFACES	**Add** `POST /bank-statements` → auto-match on amount + date ±3d · **Update** the match set
TODAY	`TODAY` Walked and renders. With no bank account configured the whole wizard refuses by name
TASK	`TASK` **P1.14 — move onto the shared** `WizardFrame`
WALKED	**YES**
RULES	Breaks ✗ B14 rule 3 (a 2+ stage flow uses the numbered step rail) · ✗ `BR-SHL-018` · ✗ `BR-SHL-019`

3·4 Resolve

`modal` `KEEP` ③ WHAT YOU CAN DO

Opens from · `Resolve difference` , the primary action in the page head of **3·2 Batch**. It renders only when the batch is **unlocked** and **has a bank line**; with no bank line the head shows the reason instead

OWNED BY PAYMENTS · REUSED HERE, NOT REBUILT

`TODAY` **This is not a Finance screen.** It is `07-frontend/src/features/payments/DepositBatchPage.tsx` , and its endpoints are in `06-backend/apps/payments/` . **Every screen in tab 3 is Payments code** shown at a Finance address. **Nothing here is Finance's to build**, and any change is a Payments change — front end, backend and tables alike.

FINDING · THE SERVICE TAKES A LIST, THE POP-UP DRAWS ONE LINE

`GAP` `reconcile(… resolutions: tuple[ResolutionInput, ...])` — the service accepts **as many explanations as the difference needs**, each with its own reason, GL account and **signed** amount, and it **checks that they sum to the difference** (`reconciliation.py:2301` ; `api.py:1135` reads a list off the payload). **The pop-up builds exactly one.**

Why that bites. A card day short by **$172.50** is routinely a **$147.50 processor fee + a $25.00 chargeback** — two reasons, two GL accounts. Today the clerk picks one of each for the whole amount, which files $25 of chargeback as a fee — **and defeats the reason the reason list exists**: it was introduced so a finance user could ask how much of a quarter was fees and how much was error.

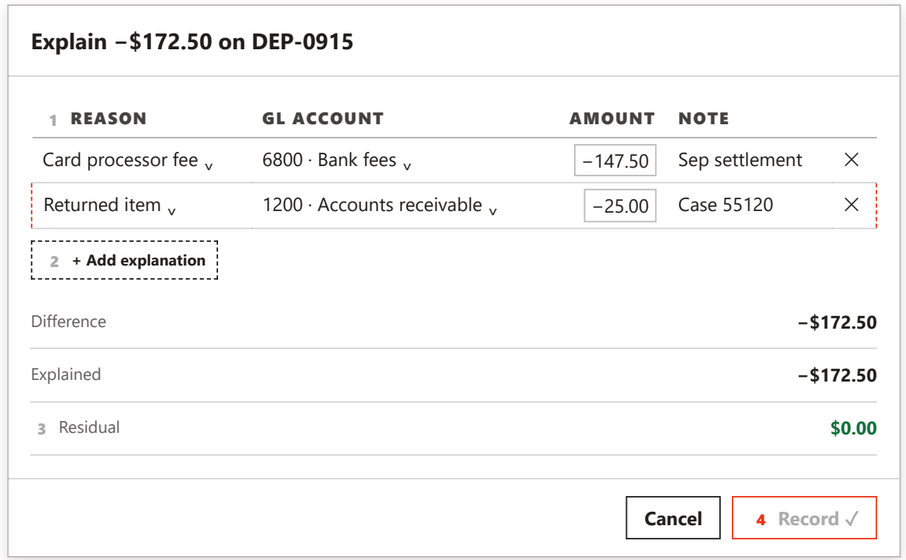
`TASK` **P3.12 — a line-adding pop-up.** A repeating row (reason · GL · amount · note), `+ Add explanation` , and **a running residual that must reach $0.00 before Record ✓ enables**. The service already enforces that sum; the pop-up only has to show it. **No endpoint changes.**

THREE DIFFERENT "MULTIPLES", AND ONLY ONE OF THEM IS A GAP

THE CASE	WHERE IT IS HANDLED
Many payments in one deposit	**Always true, and needs nothing.** A batch *is* a group of payments; `recorded_total` is their sum. This pop-up explains the **gap** between that sum and the bank
Many causes of one difference	**The gap above** — service yes, pop-up no. P3.12
Many batches against one bank line	`MatchSetResult` carries `batch_ids` and `batch_codes` **plural**. But a set **must foot exactly** — "*across a set there is no ruled way to apportion it between the members. Tie the members that do foot, and explain the rest.*" So a multi-member set **cannot carry a difference at all**

THE BATCH CODE — A SCREEN PROBLEM, NOT A POP-UP PROBLEM

By the time this pop-up is open the clerk has already found the batch; matching and filtering happen one step earlier. There, `GET /deposit-batches` **serves only** `f.status` **and** `f.method` (`api.py:1001-1002`) — **no search and no code filter**, so a clerk holding a code off a deposit slip or the Payments screen has nowhere to type it. The code itself is handled right everywhere else — it is the column 3·1 renders, and `MatchSetResult` carries `batch_codes` beside the ids, so B14 rule 31 is obeyed. TASK **P1.7** adds the search, and the code as a column on 3·3b's match table.



END STATE · DASHED = NEW · A CARD DAY SHORT BY $172.50: A FEE AND A CHARGEBACK

1. **One row per cause** · its own reason, GL account and signed amount. The service already takes a list (`reconciliation.py:2301`); the pop-up drew one row · NEW · P3.12
2. **+ Add explanation** · as many rows as the difference has causes · NEW
3. **Residual** · difference minus explained. **The service already refuses a sum that does not match**; the pop-up now shows it before you press · NEW
4. **Record ✓** · enabled only at a residual of $0.00 · EXISTS

FOR	A clerk saying *why* the bank and the books differ, in a way the books can carry.
DOORS	`Resolve a difference` on 3·2, and `Resolve →` on 3·3b's rows
PRIMARY ACTION	`Record ✓`
FIELDS	Reason · GL account · Amount
TABLE CONTROLS	Not a list
WRITE SURFACES	**Add** a row in `bank_rec_resolutions` ; repeatable until the difference is fully explained
TODAY	TODAY The server refuses the reconcile until the resolutions sum to the difference — **a service rule, not a convention**
TASK	None
WALKED	**NO**
RULES	Obeys `US-PAY-022` · hard rules 4 · 7

OWNED BY PAYMENTS · REUSED HERE, NOT REBUILT

TODAY **This is not a Finance screen.** It is `07-frontend/src/features/payments/BankImportWizard.tsx` , and its endpoints are in `06-backend/apps/payments/` . **Every screen in tab 3 is Payments code** shown at a Finance address. **Nothing here is Finance's to build**, and any change is a Payments change — front end, backend and tables alike.

SMARTBillHub

Finance › Deposits & Bank › **Statement**

⌕ Search ? ◔

BB Brazos Bend Utilities

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

v2.340.0

DW Dana Whitfield
System Admin

Reconcile a bank statement

Tie to deposits

← Deposits & Bank

General ledger | Revenue & AR | **Deposits & Bank** | Taxes & Remittance | Costs | Close & export

Statement · September 2026 ᵥ (in ?statement= — addressable)

LINE	BANK SHOWS	OUR TOTAL	DIFFERENCE	REASON
Closing balance	$19,151.60	$18,911.60	$240.00	Deposits in transit
Outstanding cheques	—	$84.00	−$84.00	Refund RFD-2609-0004

A figure is missing.

Unexplained lines

Not in the books. Recorded here, not posted.ur books.

FIN-03.statement

AS BUILT · FIGURES ILLUSTRATIVE

ONE DOOR. DELETE ONE BUTTON AND 1,122 LINES BECOME TYPED-URL-ONLY

GAP Reached from exactly one place: `DepositsPage.tsx:348` . Not a tab, not on the rail, not in the report library. The comment directly above that button says the quiet part — *"the four reconciliation backends have been live and unreachable since 2026-09-01, and a screen no nav item reaches is the same defect one layer up."* **Somebody fixed an unreachable backend with a single button and recreated the problem one layer up.**

ITS REFUSAL TO POST IS THE RULING, NOT A HOLD

RULED `OQ-1310` closed **by neither option put**. PO verbatim: *"we need to go on factual basis… if clark wants to balance the book they can always make journal entries on their own."* **Do not "fix" this.**

Filter criteria

CRITERION	PARAMETER	VALUES	SOURCE	SERVED?
Period	`from · to`	the reconciliation window	`GET /bank-rec/differences`	SERVED
Reason	`f.reason`	the tenant's difference reasons	Settings → reference data	NOT SERVED
State	`f.state`	Matched · Explained · Open	DERIVED	NOT SERVED

Components to use

COMPONENT	WHERE	WHY THIS ONE
`KPIStrip`	statement totals	Bank · books · difference
`DataTable`	reconciling items	Server paging with P1.8
`DownloadButton`	the statement	For the auditor
`MoneyCell · StatusPill`	cells	The shared kit

FOR	A finance officer proving the bank's closing balance against the books.
DOORS	**ONE** — `DepositsPage.tsx:348`
PRIMARY ACTION	`Tie to deposits` — disabled until at least one line and one deposit are ticked
FIELDS	Statement picker · per line: bank shows · our total · difference · reason · explanation
TABLE CONTROLS	Three lists, **none paginated**. The statement picker **is** in `?statement=` — the only addressable filter in the tab
WRITE SURFACES	**Add** an explanation (recorded, **not posted**) · **Update** the many-to-one match set
TODAY	TODAY Four finished backends behind one route: the statement, a per-line explanation, the match set, the differences report
TASK	TASK P1.8. **A second door is a gap with no task** — adding one is a nav change, which is the PO's
WALKED	**YES** — 0 tables on the empty fixture
RULES	Obeys `OQ-1310` . Breaks ✗ UI-RULES 12·13·14·**15**·16 · ✗ NFR-3

④ What each act changes, and where — Tab 3

ACT	WRITES	THE LEDGER	ELSEWHERE
Cut a day 3·3a	`deposit_batches · deposit_batch_events · a DEP-number · audit_log`	**Nothing** — grouping is not a posting	3·1 gains rows · 6·1 "*deposit batches reconciled*" can now be answered · **cleared payments only; a processing payment is not in the drawer**
Import a statement 3·3b	`bank_statement_imports · bank_lines · bank_rec_match_sets/members`	Nothing	3·1 statuses move to matched / unmatched · 3·5 gains a statement to reconcile
Resolve a difference 3·4	`bank_rec_resolutions · audit_log`	**Nothing yet** — the reason and GL are recorded; the posting happens at Reconcile	3·2's remaining difference shrinks; the reconcile stays refused until it reaches zero
Reconcile 3·2	`deposit_batches` status → reconciled, **LOCKED** · `deposit_batch_events`	**DR 1000 Cash / CR 1010 Undeposited funds**, plus each resolution to its own GL. RULED `OQ-143` option (b)	1·1 cash and undeposited move · the Revenue & AR *Collected* figure moves · 6·1 checklist item flips · **the batch can no longer be edited**
Unreconcile 3·2	status back · `audit_log`	**Posts nothing** — `BR-LED-024` : the cash stays in the books	3·1 status reverts; **the trial balance does not move**, which is the whole point of the rule

Tab 4 · Taxes & Remittance

Two screens, and the best refusal in the module.

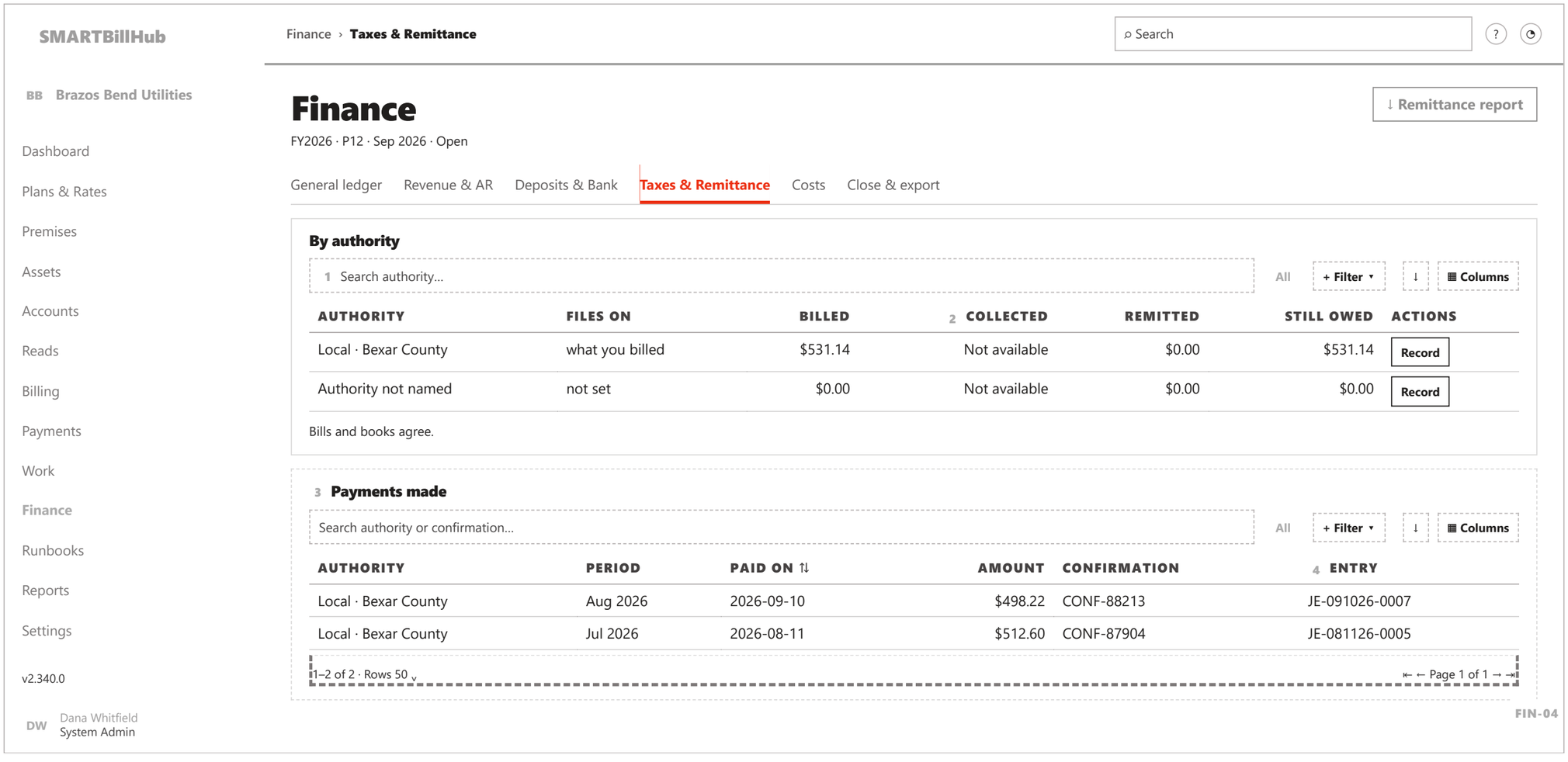
4·1 By authority `/finance/taxes` UPDATE ① THE LIST

FINDING · YOU CAN RECORD A TAX PAYMENT AND NEVER SEE IT AGAIN

GAP **`POST /finance/taxes/remittances` exists. There is no GET, and no table.** `record_remittance` (`finance_api.py:1110`) posts a balanced journal entry and **stores nothing else** — there is **no remittance, filing or tax-payment model anywhere in the backend**. The screen renders what you owe by authority, a tie-out, and the record pop-up. **Nothing lists what you have paid.**

What that costs. Asked *"did we pay Bexar County in August?"*, an officer leaves Finance, opens the journal and filters by source `remittance` — which works, because the entry is stamped `SOURCE_REMITTANCE` , but only if you know. **And owed and paid never appear on the same screen**, which is the one comparison this tab exists to make.

TASK **P4.11 — a second panel, *Payments made***: authority · period · paid on · amount · confirmation · the entry it posted. Servable now from the journal by source type; a table is needed only if a filing reference or a document must be kept. **The read comes first.**



END STATE · DASHED = NEW · FIGURES ILLUSTRATIVE (B14 RULE 37)

1 **Toolbar** on both tables — search, filters, columns, download · NEW

2 **Collected** · *Not available* by design: a payment does not record which charges it met, so this basis is correctly refused · EXISTS

3 **Payments made** · **what you have already paid each authority** — the list that did not exist. Servable now from the journal by source type · NEW · P4.11

4 **Entry** · the journal entry each payment posted, in the ruled `JE-MMDDYY-nnnn` format once P0.3 lands. A filing reference column follows with P4.4 · NEW

Filter criteria

CRITERION	PARAMETER	VALUES	SOURCE	SERVED?
Period	`from · to`	the accounting period	`GET /finance/taxes` — the period window	SERVED
Authority	`f.authority`	the tenant's tax authorities	—	NOT SERVED in the browser
Search	`q`	authority name	—	NOT SERVED in the browser
Payments made — period	`from · to`	a date range	no read exists yet — P4.11	NOT SERVED

Components to use

COMPONENT	WHERE	WHY THIS ONE
`DataTable ×2`	owed · paid	The same table twice — **owed and paid finally on one screen**
`RowActions`	Record	Opens 4·2
`AbsentFigure`	Collected	The refusal travels with the empty cell — P2.14
`DownloadButton`	Remittance report	Disabled with its reason until the export resource exists
`MoneyCell · DateCell`	cells	The shared kit

FOR	A finance officer working out what to file and what to pay each authority.
DOORS	**Three** — the tab strip twice, plus a report-library card that renders as a link only when the server marks it available
PRIMARY ACTION	`Record a payment` per row — a text link today, which UI-RULES 15 wants as an icon
FIELDS	Authority · Files on · Billed · Billed on bills · Collected · Remitted · Still owed · Action
TABLE CONTROLS	**All four absent.** Sorted by account `code asc` **in Python** (`reporting.py:2470`) — fails R14. Unpaginated, but **defensibly**: bounded by the authority list, not by transaction volume
WRITE SURFACES	**Detail** NOTHING · **Add / Edit** NOTHING · **Update** `POST /finance/taxes/remittances`
TODAY	TODAY Walked: 1 table, 0 pagers. Where nobody has chosen a basis the row says so — *"nothing is assumed, because the right answer differs per authority"*
TASK	TASK P0.3 stamp the authority · P4.4 the register · P4.1 the download
WALKED	**YES**
RULES	Obeys BR-REP-001 · BR-REP-002 · `BR-LED-002` . Breaks ✗ UI-RULES 12·13·14·**15**·16 · ✗ R14

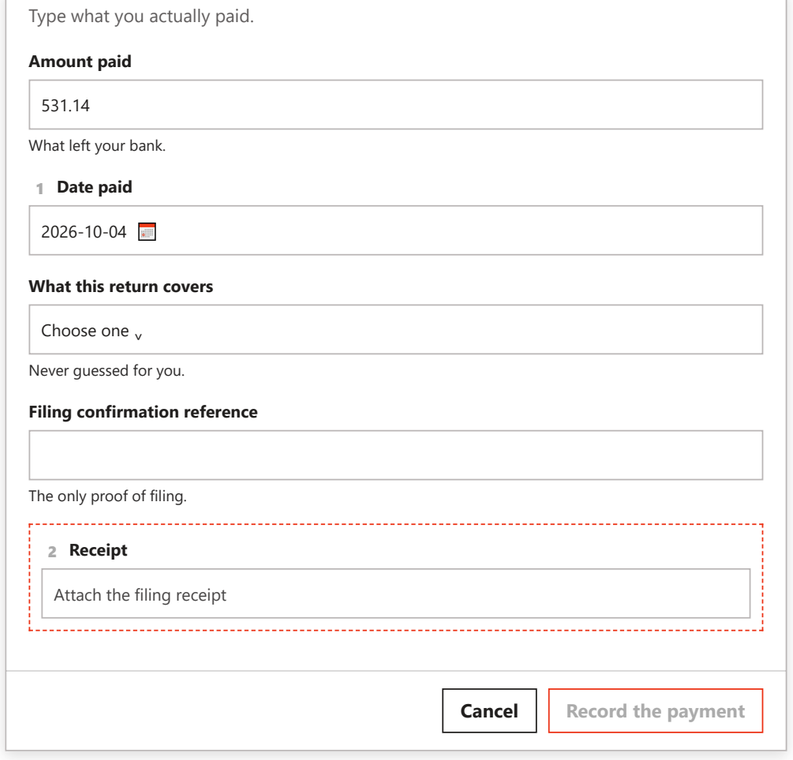
4·2 Record a payment

`modal` UPDATE ③ WHAT YOU CAN DO

Opens from · the `Record` row action on **4·1 By authority**, one per authority (`RemittancePage.tsx:316`)

LIABILITY ACCOUNT 2200-01

Record a payment · Bexar County



END STATE · DASHED = NEW

1 **Date paid** · a **bare text input** today with placeholder `YYYY-MM-DD` ; B14 rule 7 requires the date input with a calendar, keyed entry and a Today shortcut · CHANGED

2 **Receipt** · `BR-LED-019` **requires** the attachment; the form refuses it by name — *"a receipt file cannot be kept yet, so it is refused rather than accepted and lost"* · NEW · P4.3

A CLOSED QUESTION STILL CITED AS OPEN

RULED `OQ-153` — a typed posting date is midnight **in the utility's own timezone** — status column **CLOSED**. And `RemittancePage.tsx:437` still calls it *"(raised by the payments lane, unanswered)"*. This is the field where it matters.

FOR	A finance officer who has filed and paid, draining the liability without touching the books by hand.
DOORS	`Record a payment` on 4·1's rows
PRIMARY ACTION	`Record the payment — AdminOrClerk`
FIELDS	Amount paid · Date paid · Basis · Confirmation reference · NEW Receipt
TABLE CONTROLS	Not a list
WRITE SURFACES	**Add** `POST /finance/taxes/remittances → journal.record_tax_remittance`
TODAY	TODAY Posts one balanced entry, DR 2200-nn / CR cash. The amount is **never** pre-filled from the report
TASK	TASK P4.3 the receipt. **The date input is a gap with no task**
WALKED	**NO**
RULES	Obeys `BR-LED-013` · hard rules 2 · 4 · 7. Breaks ✗ B14 rule 7 · ✗ `BR-LED-019`

④ What each act changes, and where — Tab 4

ACT	WRITES	THE LEDGER	ELSEWHERE
Record a remittance 4·2	One `journal_entry` + two lines · `audit_log` · after P4.4, a row in `tax_remittances`	**DR 2200-nn Tax payable / CR 1000 Cash** — one balanced entry that drains the authority	4·1 *Still owed* drops and *Remitted* rises · 1·2 trial balance moves · 6·1 "*tax remittance recorded*" flips to done · **the filing confirmation is the only permanent proof** until P4.3

Tab 5 · Costs

RULED OQ-188 : the product is the system of record for AP and AR both, and **processes neither** — *capture, hold, reconcile, export; no approvals, no payment runs, no vendor master.* This tab produces the one figure the PO said an accounting package cannot: **what fraction of the water you bought did you bill?**

5·1 Register `/finance/cost-of-service` UPDATE ① THE LIST

SMARTBillHub

Finance › Costs › **Register**

⌕ Search ? ◔

BB Brazos Bend Utilities

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports
Settings

v2.340.0

DW Dana Whitfield
System Admin

Finance

FY2026 · Period 12 · Open

+ Capture invoice

General ledger | Revenue & AR | Deposits & Bank | Taxes & Remittance | **Costs** | Close & export

Register | **Bought vs billed**

1 Search supplier, reference, memo…

All | **Volume captured** | **Not captured** | 2 + Filter ▾ | ▦

INVOICE DATE ⇅	SUPPLIER	SERVICE	AMOUNT	3 VOLUME	GL ACCOUNT	STATUS	PAID ON	4 ACTIONS
2026-09-12	Canyon Lake Water	Water	$4,180.00	1,000,000 gal	5110 · Purchased water	Captured	In the books 2026-09-15	5 👁 ✎ …
2026-09-08	Pedernales Co-op	Electric	$11,240.55	▸ No volume	5100 · Purchased power	Captured	▸ Recorded	✎ …
2026-08-29	Aqua Texas	Water	$950.00	225,000 gal	5110 · Purchased water	Voided	▸ Recorded	

6 1–50 of 214 · Rows 50 ⌄ ⇤ ← Page 1 of 5 → ⇥

Proposed, not signed.

FIN-01.costOfService

END STATE · DASHED = NEW · FIGURES ILLUSTRATIVE

1 **Search · the best in Finance and the pattern to copy** — `q` reaches the server (`cost_capture.py:2090`), has a minimum-length guard, and **lives in the URL** · EXISTS

2 **+ Filter** · the endpoint accepts `f.service` , `f.status` , `f.from` , `f.to` , `f.gl_account_id` — **and not one has a control** · NEW

3 **Volume** · optional, RULED `OQ-190` *"make a provision and make it optional"* · EXISTS

4 **Row actions** · ✎ correct; … holds Record payment and Void (danger tier). Today three sit inline; the cap is two plus overflow · UI-RULES 15 ✗ · CHANGED

5 **View document** · 👁 opens the invoice attached on 5·3a — rule 15's own *view* icon, never a text link. No document, no eye · NEW · P5.11

6 **Pager + empty state** · the register's own docstring says "*Pagination is deliberately absent*"; a utility capturing monthly for years only grows · UI-RULES 16·17 ✗ · NEW

Filter criteria

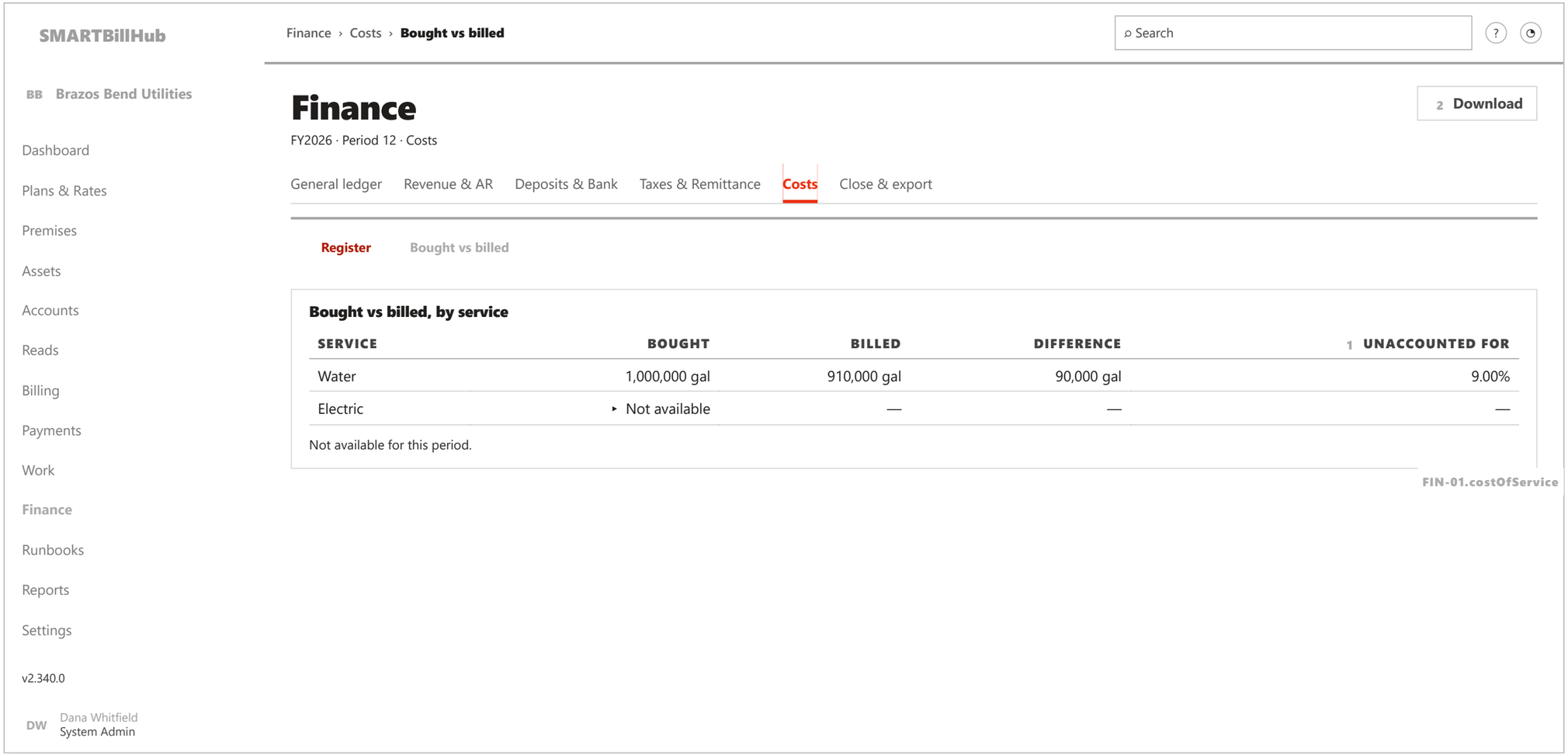
CRITERION	PARAMETER	VALUES	SOURCE	SERVED?
Period	`f.from · f.to`	the period each invoice lands in	`cost_capture.py:2070`	SERVED
Service	`f.service`	**eleven**, incl. **Fire**	`GET /reference/services`	SERVED
State	`f.status`	Captured · Voided — no paid status, by rule	`BR-LED-028 · cost_capture.py:2077`	SERVED
Volume captured	`f.volume_captured`	yes · no — the *All* · *Volume captured* · *Not captured* chips	`cost_api.py:375`	SERVED
GL account	`f.gl_account_id`	the cost accounts	`GET /ledger/accounts`	SERVED
Search	`q`	supplier, reference or memo	`cost_capture.py:2090`	SERVED
Pages	`page`	—	"*Pagination is deliberately absent*" — `cost_capture.py:2062` · **P1.8**	NOT SERVED

Components to use

COMPONENT	WHERE	WHY THIS ONE
`DataTable · ListPager`	the register	Five filters and search are served and **none has a control today**; paging with P1.8
`FilterBar · ChipMultiSelect`	state · service	The shared kit
`RowActions`	Actions	👁 view document · ✎ correct · … with Record payment and **Void** — danger tier, B14 rule 6. Two visible, rule 15
`AbsentFigure`	volume · paid on	*No volume*, and *Recorded* until the payment is in the books — stated, never blank
`MoneyCell · DateCell · StatusPill`	cells	The shared kit

FOR	A finance officer recording what the utility bought, so the bought-vs-billed figure can exist.
DOORS	**Two**, both the tab strip. **Not in SHELL_NAV at all** — not even as breadcrumb data, so a clerk here reads a one-segment crumb. `ChartOfAccountsPanel.tsx:662` claims it is "*one click from this panel*" and **no such link exists** — a comment claiming a guarantee
PRIMARY ACTION	`+ Capture invoice`
FIELDS	Invoice date · Supplier · Service · Amount · Volume · GL account · Status · Paid on
TABLE CONTROLS	Search **server-side and in the URL** · one filter wired, five unreachable · no chooser · sort SQL `-invoice_date` (**passes R14**) but the FE never sends it and draws no sortable headers · **no pager, no empty state**
WRITE SURFACES	**Detail** NOTHING · **Add** `cost_api.py:307` · **Edit** `:467` · **Update** settle / void `:628`
TODAY	TODAY Correcting a fact a **posted settlement** copied raises `NotImplementedError("OQ-1350")` → 409. Deliberate
TASK	TASK P1.8 pager · P1 kit for the filters and empty state
WALKED	**YES** — 1 table, 0 pagers
RULES	Obeys R14 · `OQ-188` · `OQ-190` . Breaks ✗ UI-RULES 13·14·**15**·16·**17** · ✗ NFR-3

5·2 Bought vs billed ?view=reconciliation UPDATE ① THE LIST



AS BUILT · WALKED · FIGURES FROM THE WORKED EXAMPLE

1 **Unaccounted for** · leak, under-registration or theft. **All-or-nothing per service per period** — reported absent rather than derived from a partial denominator · EXISTS

2 **Download** · disabled with a **server-supplied reason that is no longer true** · CHANGED · P2.5

> **A REFUSAL NAMING A QUESTION THAT HAS BEEN ANSWERED**
>
> GAP The reason served from `cost_api.py:742-751` reads "*...not built, because WHERE this view lives has not been ruled.*" **It was ruled and shipped:** `COST_OF_SERVICE_ROUTE` is mounted at `App.tsx:1488` and is a live Finance tab. OPEN `OQ-204` 's status column reads **PARTIALLY CLOSED 2026-09-03 — the ids are ratified; the download contract is the remaining half.** Two front-end files still assert it is wholesale OPEN. **The button is right to be disabled; the sentence is wrong**, and it tells a finance officer the product does not know where its own screen lives.

Filter criteria

CRITERION	PARAMETER	VALUES	SOURCE	SERVED?
Basis	`basis`	Service period · Issued	`billing_seam.PERIOD_BASES` — which one billing uses is `OQ-202`	SERVED
Period	`from · to`	the accounting period	`/finance/cost-of-service/reconciliation`	TO VERIFY
Service	`f.service`	**eleven**, incl. **Fire**	`GET /reference/services`	NOT SERVED in the browser

Components to use

COMPONENT	WHERE	WHY THIS ONE
`DataTable`	bought vs billed	One row per service — bounded
`SegmentedControl`	basis	Two values, both served
`AbsentFigure`	the % column	All-or-nothing per service and period — *Not available* with its reason, never a misleading figure
`DownloadButton`	the reconciliation	Disabled with its reason — `NotImplementedError("OQ-204")`
`MoneyCell`	cells	The shared kit

FOR	A finance officer measuring loss — the figure an accounting package cannot produce.
DOORS	**Two**, both the sub-view strip. Rendered as plain `<Button>` s with **no `aria-current` and no tab semantics** — a strip in appearance only
PRIMARY ACTION	`Download` — disabled
FIELDS	Service · Bought · Billed · Difference · Unaccounted-for %
TABLE CONTROLS	Bounded by the enabled service set — **no pager needed**, exemption stated. **No empty state** *X*
WRITE SURFACES	**All four NOTHING** — a read-only reconciliation
TODAY	TODAY Walked: 1 table, Download observed disabled
TASK	TASK P2.5 the sentence. **The download itself waits on `OQ-204` 's second half — open, not scheduled**
WALKED	**YES**
RULES	Obeys BR-REP-002. Breaks *X* UI-RULES 17

5·3a Capture

`modal` UPDATE ③ WHAT YOU CAN DO

Opens from · `+ Capture invoice` in the page head of **5·1 Register**

THE FINDING: NOWHERE TO PUT THE SUPPLIER'S INVOICE

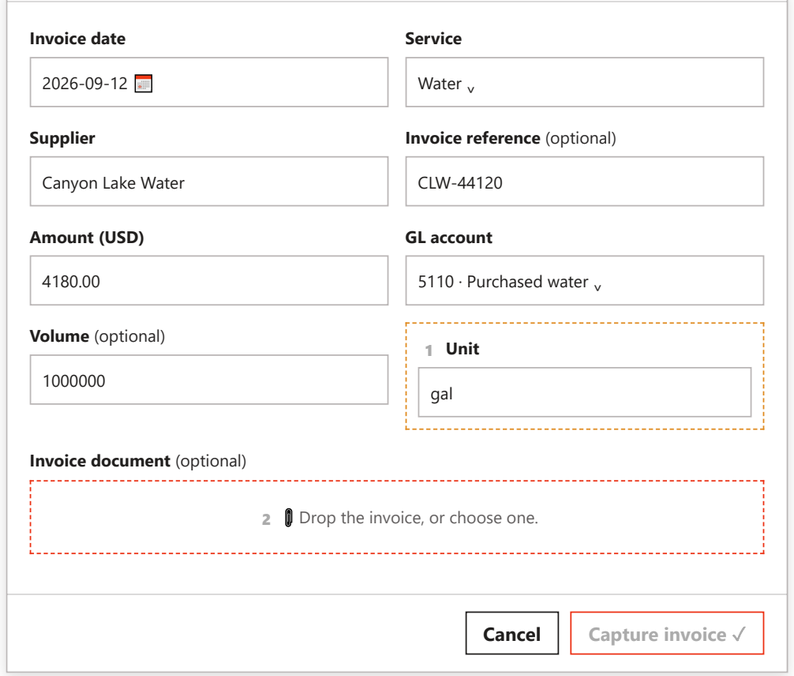
GAP **`supplier_invoices` has 21 columns and not one of them holds a document.** No `FileField` , no attachment FK, no `upload_to` anywhere in the module. The only document model in the whole backend is `ServiceRequestDocument` in customers — a **checklist of eight designed types** for service requests (`BR-CUS-095`), not a general store, and nothing in Finance points at it. **So a captured purchase carries the supplier's name and a free-text `reference` , and the PDF that proves it lives in somebody's inbox.**

Where it hurts, concretely. 5·2 bought-vs-billed exists to argue about a supplier's figure — and the document that settles the argument cannot be opened from the row. An auditor asking *"show me the invoice behind this $2,400"* gets a supplier name and a reference string. And **5·3b's correction flow requires a reason for a change** whose evidence the system cannot hold.

P5.11 — attach a document to a captured purchase. Drag-or-choose on this modal, a link on the 5·1 row and on 5·2's comparison. ⚠ **This is bigger than it looks and is sized as such**: it is the platform's **first general attachment**, so it brings storage, tenant-scoped access, a size and type limit, virus posture and a retention answer with it. **It is filed as a gap with a task, and the task is explicitly a spike first** — nobody should start it by adding a column.

OPEN **`OQ-LED-1512`** PROPOSED **— does an attached purchase document fall under the same retention schedule as `audit_log` ?** The PO ruled on 2026-09-23 that audit rows are pruned with no exemption; **an invoice a utility is required to keep for seven years is a different question and has not been put to them.** It must be answered before storage is chosen, not after.

Capture invoice



END STATE · DASHED = NEW · FIGURES ILLUSTRATIVE

1 **Unit** · free text, **never a picker** — a picker would be the vocabulary OPEN `OQ-201` asks for (`CostOfServicePage.tsx:469`). Never resolve it by choosing a value · EXISTS

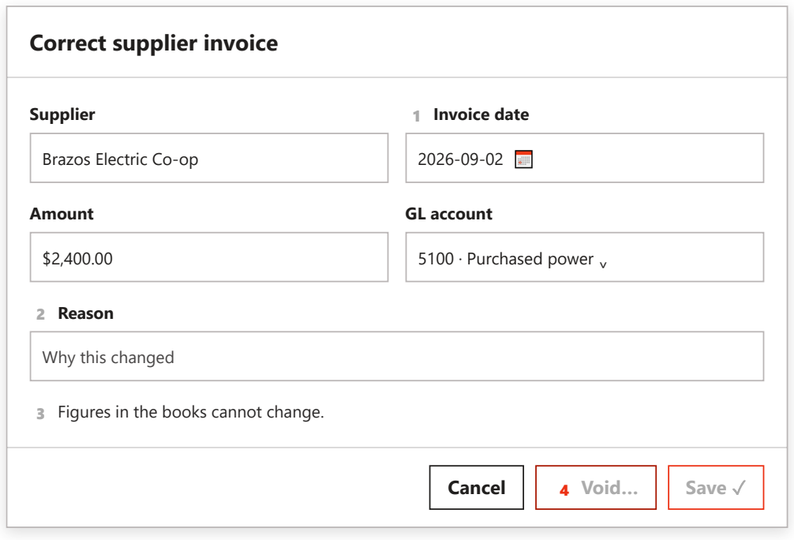
2 **Attach the invoice** · drag or choose the supplier's PDF. **Spike first** — it is the platform's first general attachment, so storage, access, limits and retention come with it; retention is `OQ-LED-1512` and is not assumed. Optional, so capture still works without one · NEW · P5.11

FOR	Whoever opens the supplier's bill.
DOORS	`+ Capture invoice` on 5·1
PRIMARY ACTION	`Capture ✓`
FIELDS	Invoice date · Service · Supplier (free text) · Invoice reference · Amount · GL account · Volume · Unit (free text) · **Invoice document — new**. The period comes from the invoice date (`cost_capture.py:1107`)
TABLE CONTROLS	Not a list
WRITE SURFACES	**Add** `POST /finance/cost-of-service → supplier_invoices`
TODAY	TODAY **No vendor master, by ruling.** Capital categories are offered **disabled** with "*record it on the asset instead*" — a key **missing from all eleven catalogues**, so ten locales show English
TASK	TASK P2.6 adds the missing key. **The unit picker waits on** `OQ-201`
WALKED	**NO**
RULES	Obeys `OQ-188` · `OQ-190` · `BR-LED-028` · the prime directive

5·3b Correct · void

modal KEEP ③ WHAT YOU CAN DO

Opens from · ✎ or the … menu on a **5·1 Register** row. Void sits in the menu (B14 rule 15) and commits in the danger tier (rule 6)



AS BUILT · **DRAWN FOR THE FIRST TIME — SEE 2·5C'S NOTE**

1 **Invoice date** · **the invoice stays in its own period** — `OQ-227` , a correction is never a move · EXISTS

2 **Reason** · required on both verbs · EXISTS

3 **The refusal** · *In the books* is rule 34's word · changing amount, currency, GL or premises on a **posted settlement** raises `NotImplementedError("OQ-1350")` and surfaces as a 409 · EXISTS

4 **Void…** · danger tier, B14 rule 6, and **in the footer rather than beside Save** · EXISTS

> **A CORRECTION AFTER THE PERIOD CLOSES IS ALLOWED, AND THAT IS THE RULING**
>
> RULED `OQ-203` — PO verbatim "*Yes it can be corrected*". And RULED `OQ-227` : **the invoice stays in its own period. A correction is a correction, never a move.** The consequence the PO accepted is drawn on **6·2 Export** as the drift panel — "*a captured purchase was corrected after this month was handed over. The handover cannot be repeated, so the difference has to be posted by hand in your accounting system.*"

FOR	Correcting a captured invoice, or voiding one that should not have been captured.
DOORS	The ✎ and overflow row actions on 5·1
PRIMARY ACTION	`Save / Void ✓` — void is danger tier
FIELDS	Every capture field, plus a reason on void
TABLE CONTROLS	Not a list
WRITE SURFACES	**Edit** `cost_api.py:467` · **Update** void `:628` . **Never delete**
TODAY	TODAY Changing one of the four facts a **posted settlement** copied — amount, currency, GL account, premises — raises `NotImplementedError("OQ-1350")` and surfaces as a 409
TASK	None
WALKED	**NO**
RULES	Obeys `OQ-203` · `OQ-227` · hard rule 4

④ What each act changes, and where — Tab 5

ACT	WRITES	THE LEDGER	ELSEWHERE

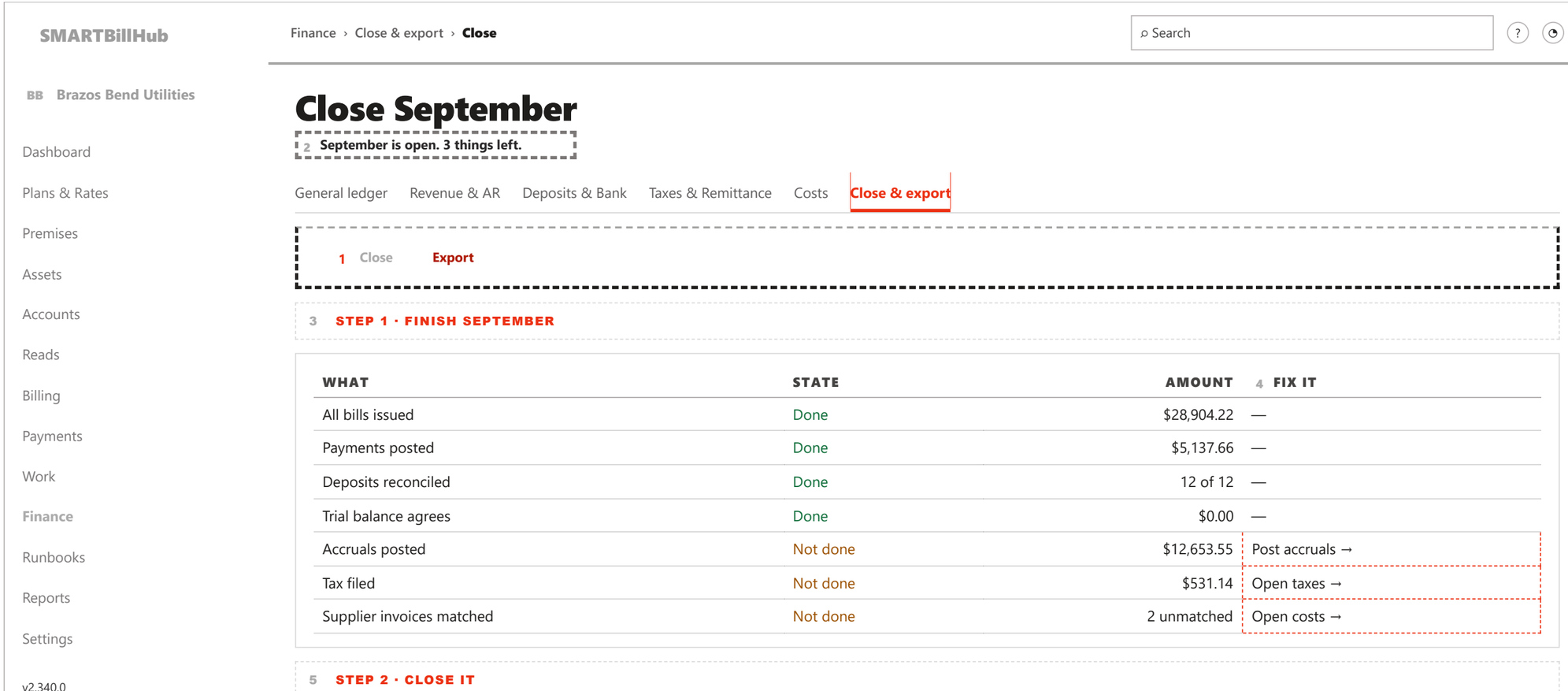
Capture invoice 5·3a	`supplier_invoices` · `audit_log`	`RULED` **DR cost / CR accrued expenses** — `BR-LED-028` as amended by `CR-060` C1	5·2 recomputes, or reports absent if any invoice for that service lacks a volume · 6·2a the cost side joins the same export batch · 1·2 trial balance moves
Correct an invoice 5·3b	`supplier_invoices` · `audit_log`	A correcting entry in **the invoice's own period** — never a move (`OQ-227`)	**If that month was already handed over, 6·2 Export raises the drift panel** and the handover is not repeated — the difference is posted by hand in the accounting package
Void an invoice 5·3b	status → voided, with a reason. **Never deleted**	A reversing entry	5·2's denominator changes · the drift rule applies as above

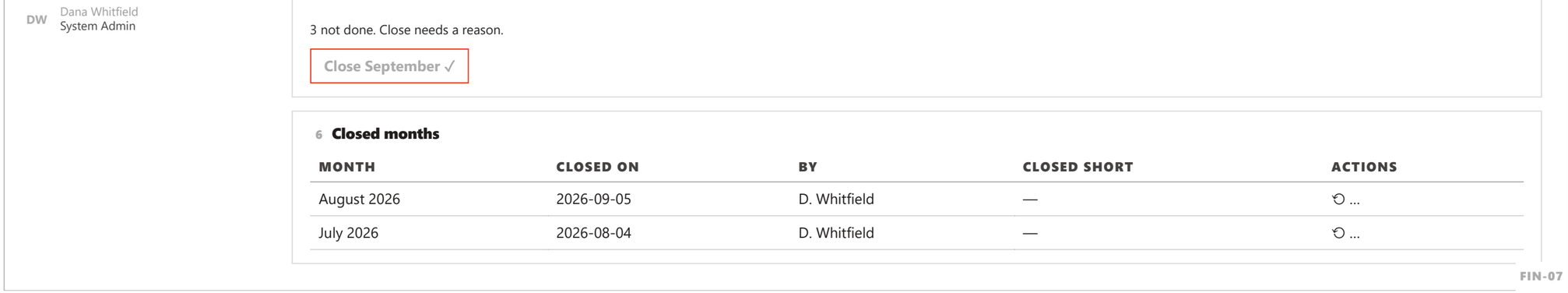
Tab 6 · Close & export

Where the month is put to bed. Two independent gates guard the handover and each states itself plainly; the button that starts it all is gated on a ruling about something else.

6·1 Close `/:tenant/finance/export?view=close` `UPDATE` ① THE LIST

> **A CLOSE, NOT A BOARD — AND SENDING NOW HAS ITS OWN VIEW**
>
> `TODAY` The screen is headed **Finance** and stacks three system-side panels — "*Before you close*", "*What the accounting package receives*", "*Handed over so far*" — so a person finishing September is never told that is what they are doing. **Redrawn as the close:** the page says which month and what is left; the work is two numbered steps; every unfinished item links to the screen that clears it.
>
> **Sending moved to 6·2 Export** (PO 2026-09-24: "*this tab is for close and export — there is no functionality to export*"). Closing and sending are two jobs with two audiences — the close is the finance officer's, the export is the accountant's — so each has a view.





END STATE · DASHED = NEW · FIGURES ILLUSTRATIVE (B14 RULE 37)

1 **Close · Export** · two views, one word each, the same pattern as tabs 1 and 2 · NEW · P4.12
2 **The one line that was missing** · which month, and how much is left — two sentences of four words · NEW
3 **Step 1 · Finish** · the checklist, with the amount each item stands for · NEW · P5.10
4 **Fix it** · every unfinished item links to the screen that clears it — **the change that makes this usable** · NEW · P5.9
5 **Step 2 · Close it** · a short list may still close, with a typed reason (`BR-LED-031`) · EXISTS
6 **Closed months** · each with ↺ reopen for a System Admin — opens 6·1b · NEW · P3.2

Filter criteria

CRITERION	PARAMETER	VALUES	SOURCE	SERVED?
Month	`period`	the open accounting period	`GET /shell/display`	SERVED
Closed months — year	`f.year`	the fiscal years with a closed month	DERIVED	NOT SERVED

Components to use

COMPONENT	WHERE	WHY THIS ONE
`SegmentedControl`	Close · Export	The same view switch as tabs 1 and 2
`BandHeading`	the two steps	A component, never inline markup — B14 rule 43 · P2.15
`DataTable` ×2	checklist · closed months	Both bounded — seven items, twelve months a year
`InlineLink`	Fix it	One door per unfinished item
`ConfirmDialog`	Close September ✓	Irreversible — rule 33 caps the consequence at two short sentences
`RowActions`	↺ reopen	System Admin only — a reason is required and audited

FOR	A finance officer finishing the month.
DOORS	**Four** — the **Close & export** tab, which lands here · 1·2's `Open close & export →` · the Dashboard month-end to-do · Settings → Fiscal periods
PRIMARY ACTION	`Close September ✓` , in step 2
FIELDS	Checklist (what · state · amount · fix it) · closed months (month · closed on · by · closed short)
TABLE CONTROLS	None needed — both lists are bounded
WRITE SURFACES	**Close** 6·1a · **Reopen** 6·1b

TODAY	TODAY Three system-side panels on one screen, headed *Finance*; no doors on any checklist row; no amounts
TASK	TASK P5.9 the steps and the doors · P5.10 the amounts · P3.2 reopen
WALKED	**YES** · 4 of 7 done, two months listed
RULES	Obeys B14 2 · 32 · 33 · 40 · 43 · `BR-LED-031`

6·2 Export

`/:tenant/finance/export?view=export` NEW ① THE LIST

ONE FILTER, ONE TABLE, TWO WAYS OUT — RULED 2026-09-24

RULED PO: "*this filter should be expanded — I need it by period start date and end date; based on the filter, show the table below it … then allow to send or download it in different file formats. We need a column selector — all selected columns go in the downloaded file.*" On the extracts: "*why do we need this — can't you merge it with the filter itself*". On the history: "*this is good*".

The server already takes any two dates (`from` · `to`). It sends only when **every month in the range is closed** and **no earlier hand-over overlaps it** (`close.window_is_closed` · `exports.overlapping_batch`). Today's screen offers the current month only.

⚠ **Download and Send stay apart.** ↓ is the table as shown — ticked columns, any dates, nothing recorded. ↗ Send hands the range over once, in the accounting format. **A QuickBooks file taken without a record could be imported twice** — the double booking `BR-LED-004` exists to stop — so the four accounting formats leave only through Send, and come back from the history.

SMARTBillHub

Finance › Close & export › **Export**

⌕ Search ? ◔

BB Brazos Bend Utilities

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- **Finance**
- Runbooks
- Reports
- Settings

v2.340.0

DW Dana Whitfield
System Admin

Export

Closed through August 2026.

General ledger | Revenue & AR | Deposits & Bank | Taxes & Remittance | Costs | **Close & export**

Close 1 Export

2 PERIOD START: 2026-08-01 | PERIOD END: 2026-08-31 | 3 GROUP BY: Account | SEND IT TO: QuickBooks Online | DETAIL: Summary | Apply

4 **Preview**

Search account… All + Filter 6 ↓ 5 ▦ Columns

Download CSV
Download PDF

GL ACCOUNT ⇅	WHAT IT IS	CODE IN YOUR PACKAGE	DEBIT	CREDIT
1200	Accounts receivable	12	$15,066.00	—
2200-01	Tax payable, State of Texas	33	—	$1,842.30
4100	Electric usage	79	—	$7,948.30
4105	Electric base	80	—	$1,250.00
4110	Water usage	84	—	$2,310.40
4115	Water base	85	—	$640.00
4120	Sewer	86	—	$440.00
4130	Trash	87	—	$255.00

COLUMNS · SHOW / HIDE · ↑↓ REORDER

- ☑ GL account ↑ ↓
- ☑ What it is ↑ ↓
- ☑ Code in your package ↑ ↓
- ☑ Debit ↑ ↓
- ☑ Credit ↑ ↓

4200	Late-fee revenue	81	—	$350.00
4210	NSF & other fee revenue	82	—	$30.00
Balanced			**$15,066.00**	**$15,066.00**

1–10 of 10 · Rows 50 ⌄ ⇤ ← Page 1 of 1 → ⇥

☐ Week ↑ ↓

☐ Type ↑ ↓

Reset table

✓ Months closed · ✓ Not sent before · ✓ Balances · ✓ Codes set

7 ↗ Send

8 **Hand-over history**

Search month or batch… All + Filter ▾ ↓ ▦ Columns

PERIOD	SENT TO	DETAIL	SENT ON ⇅	LINES	FILE	ACKNOWLEDGED	ACTIONS
July 2026	QuickBooks Online	Summary	2026-08-04	7	▸ Not written	▸ Not available	👁 …
June 2026	QuickBooks Online	Summary	2026-07-06	7	▸ Not written	▸ Not available	👁 …

1–2 of 2 · Rows 50 ⌄ ⇤ ← Page 1 of 1 → ⇥

9 **DRIFT**

July changed after it was sent. Post the difference by hand.

FIN-07

PROPOSED · THE WHOLE VIEW IS NEW · ↓ AND ▦ DRAWN OPEN · FIGURES ILLUSTRATIVE (B14 RULE 37)

1 **Export** · its own view on Close & export · NEW · P4.12

2 **Period start · Period end** · any two dates. Send needs **every month in the range closed** and **no earlier hand-over overlapping it** (`BR-LED-017` · `BR-LED-004`) · RULED · NEW

3 **Group by** · Account · Service · Premises · Tax authority — **replaces the three extract buttons**. Only *Account* is handed over; the other three are the extracts, for download (`GET /finance/export/extracts/{kind}` , called by no screen today) · RULED · NEW

4 **Preview** · the table for the chosen dates and grouping, *Balanced* and both totals at the foot (`GET /finance/export/preview`) · EXISTS for one month

5 ▦ **Columns** · the kit's panel, drawn open — show, hide, reorder, *Reset table*. **The download carries the ticked columns, in their order** · RULED · NEW · P4.13

6 ↓ **Download** · *Download CSV* or *Download PDF* of the table as shown — any dates, nothing recorded. **No Excel**: IS-11 · NEW · P4.1

7 ↗ **Send** · hands the range over once, in the *Send it to* format — QuickBooks Online · Xero journal file · QuickBooks Desktop file · Plain spreadsheet file. **Its files keep the layout each importer needs — the ticked columns shape the download, never the hand-over.** The four checks show before you press it; a *X* disables Send and says why, and so does another grouping: "*Send hands over accounts only.*" Opens 6·2a · EXISTS as *Hand this month over*

8 **Hand-over history** · every hand-over, 👁 drilling back to its lines (`exports.py:646`). *File* and *Acknowledged* are stated absences until P4.2 and P5.4; *Download again* sits in … · RULED kept

9 **Drift** · a sent month whose cost section no longer matches — the condition the PO attached to `OQ-203` (`export_drift`) · NEW

Filter criteria

CRITERION	PARAMETER	VALUES	SOURCE	SERVED?
Period start	`from`	any date	`finance_api.py:771` — required on preview and send	SERVED
Period end	`to`	on or after the start	`finance_api.py:771`	SERVED
Group by	the endpoint	Account → `/preview` · Service · Premises · Tax authority → `/extracts/{kind}`	`finance_api.py:1230-1233`	SERVED
Send it to	`target`	QuickBooks Online · Xero journal file · QuickBooks Desktop file (IIF) · Plain spreadsheet file (CSV)	`EXPORT_TARGETS` — typed a second time in `endpoints.ts:17276` · P1.6	SIGNED
Detail	`granularity`	Summary · Weekly — Account only	`EXPORT_GRANULARITIES` · `BR-LED-017`	SIGNED
Columns in the file	`columns`	the ticked columns, in order	IS-9 says so; `POST /exports` carries no column state — `apps/tenancy/exports.py` · P4.13	NOT SERVED

file			P4.13	
Preview — search	`q`	account code or name	every row comes back; fine by account, not by premises · P1.8	NOT SERVED
Preview — pages	`page`	—	by premises returns every premises at once (`reporting.by_premises_summary`) — up to 40,000 at T5 · P1.8	NOT SERVED
History — search	`q`	month or batch number	`GET /finance/export/batches` reads only `target`	NOT SERVED
History — sent to	`target`	the four formats	`finance_api.py:830`	SERVED

Components to use

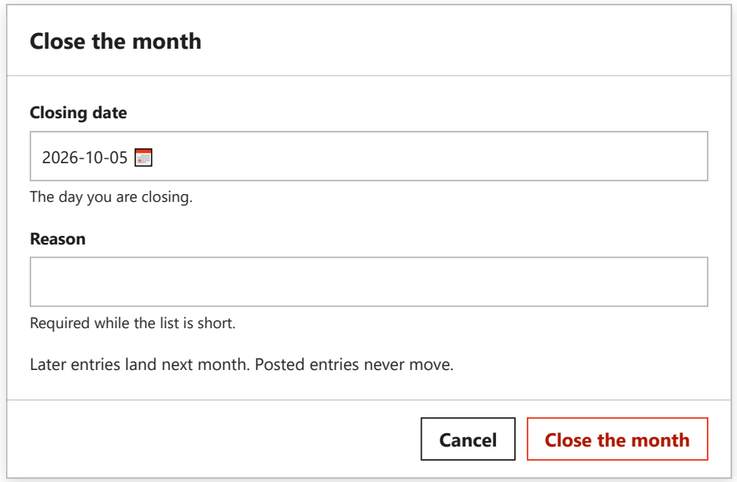
COMPONENT	WHERE	WHY THIS ONE
`SegmentedControl`	Close · Export	The same view switch as tabs 1 and 2
`DatePickerField` ×2	period start · end	B14 rule 7 — calendar, keyed entry
`SelectField` ×3	group by · send it to · detail	In a params band — `.fin-params` , as on 1·2
`DataTable` ×2	preview · history	The toolbar in the product's order: search · filters · ↓ · ▦ (CR-045)
`ColumnChooser`	▦	The kit's one panel; its ticks go to the download — P4.13
`DownloadButton`	↓	CSV always, PDF because this is a formatted statement — IS-11
`StatusPill · Chip` ×4	Balanced · the four checks	The shared kit
`RowActions`	history	👁 drill back, then … with *Download again* once P4.2 writes a file
`AbsentFigure`	File · Acknowledged	An absence with its reason, never a blank — P2.14
amber callout	Drift	The detail-page pattern from the Accounts blueprint
`ConfirmDialog`	↗ Send	A hand-over cannot be repeated — 6·2a

FOR	A finance officer handing closed months to the accountant's package, and pulling the same figures as a file.
DOORS	**Two** — the **Export** view on 6·1 · a `/finance/export?view=export` bookmark
PRIMARY ACTION	`↗ Send` — opens 6·2a
FIELDS	Period start · Period end · Group by · Send it to · Detail · the preview · the history (period · sent to · detail · sent on · lines · file · acknowledged)
TABLE CONTROLS	Preview — search, ↓, ▦; pages for Premises with P1.8. History — all six, in the browser today
WRITE SURFACES	**Send** 6·2a — one `export_batches` row with its `EXP-` number and content hash, and nothing else. Download and the three groupings are reads
TODAY	TODAY Step 3 inside the close screen, for the current month only — a preview and a Send button. **The server already takes any dates, and the three extracts are built and called by no screen**
TASK	TASK **P4.12 the view** · P4.13 columns in the file · P4.1 each grouping downloads · P4.2 the file · P5.4 the acknowledgement · P1.6 formats served once · P1.8 pages for Premises
WALKED	**NO** — the view is new
RULES	Obeys B14 2 · 4 · 7 · 12 · 13 · 14 · 16 · 31 · 33 · 40 · IS-9 · IS-11 · `BR-LED-017` · `BR-LED-004`

6·1a Close the month

modal UPDATE ③ WHAT YOU CAN DO

Opens from · `Close September ✓` , step 2 of **6·1 Close**



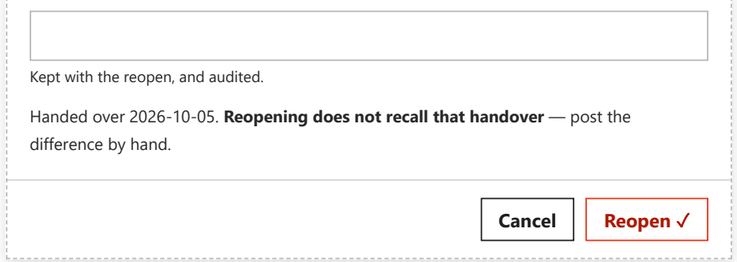
AS BUILT

FOR	A System Admin putting the month to bed.
DOORS	`Close this month` on 6·1
PRIMARY ACTION	`Close the month` — danger tier; a period lock is destructive in effect
FIELDS	Closing date · Reason (required only when the list is incomplete)
TABLE CONTROLS	Not a list
WRITE SURFACES	**Add** `POST /finance/periods/close → close.close_period → fiscal_period_closes`
TODAY	`TODAY` **WARN + CONFIRM** (`BR-LED-018`): without a reason over an incomplete list the server answers **422** `BIZ-APPROVAL` naming every incomplete item; the same request with a reason succeeds, and the **incomplete list is snapshotted as it stood**
TASK	`TASK` P0.11 asserts numbers survive the close
WALKED	**NO**
RULES	Obeys `BR-LED-006` · `BR-LED-018` · `BR-TEN-038` · hard rule 7

6·1b Reopen `modal` NEW ③ WHAT YOU CAN DO

Opens from · the ↺ row action on **6·1**'s *Closed months* — System Admin only





PROPOSED · THE WHOLE SCREEN IS NEW

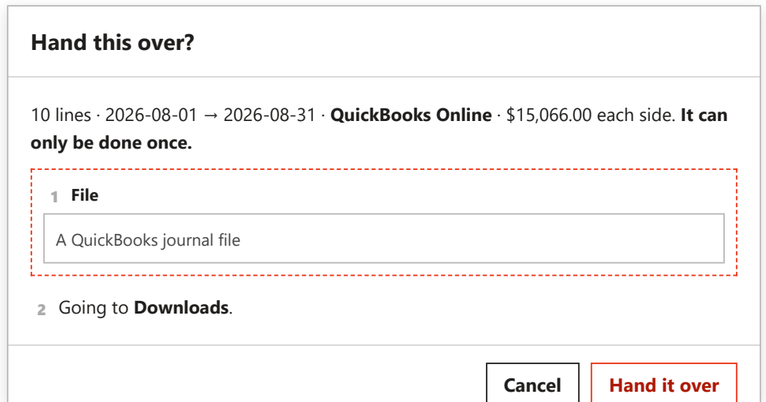
BUILT, WRAPPED, AND CALLABLE FROM NOWHERE

GAP `POST /finance/periods/reopen` exists and a typed client wrapper exists at `endpoints.ts:17263` . **No component calls either.** A period closed in error is reopened from a shell today.

FOR	A System Admin undoing a close made in error.
DOORS	**NOTHING today.** After P3.2: the closed-period row on 6·1
PRIMARY ACTION	`Reopen ✓` — danger tier
FIELDS	Period · Reason · a consequence line about any handover already sent
TABLE CONTROLS	Not a list
WRITE SURFACES	**Update** `POST /finance/periods/reopen` , System Admin only
TODAY	GAP Endpoint + wrapper, no caller
TASK	TASK P3.2 · P3.7 the journey asserts the trial balance is byte-identical either side
WALKED	**NO** — does not exist
RULES	Will obey `BR-LED-006` · B14 6 · hard rule 7

6·2a Hand over `dialog` UPDATE ③ WHAT YOU CAN DO

Opens from · ↗ `Send` on **6·2 Export**. Disabled with its reason while any of the four checks fails



END STATE · DASHED = NEW

1 **A file** · written from the **stored payload** so it is hash-stable and a later ref edit cannot rewrite a sent batch. ⚠ Direct OAuth sync is explicitly **v2** under `BR-LED-020` · NEW · P4.2

2 **Going to** · the saved destination — the browser's downloads, or a folder the utility named (the accounting-file connection: download or drop). That is the product's *send or download* · NEW · P4.9

FOR	The person who hands the month to the accountant.
DOORS	↗ `Send` on 6·2
PRIMARY ACTION	`Hand it over` — danger tier, and an IS-6 confirm naming entries, both totals, the destination and the window
FIELDS	Destination · line count · window · both totals
TABLE CONTROLS	Not a list
WRITE SURFACES	**Add** `POST /finance/export/batches` → `exports.py:462` , an `EXP-` number, the payload and a content hash stored
TODAY	TODAY A second attempt returns **409** `BIZ-IDEMPOTENT` with the existing batch on `params` so the screen can point at it. **Nothing reports back whether the package accepted it**
TASK	TASK P4.2 · P5.4 the acknowledgement
WALKED	**NO** — the period was open
RULES	Obeys `BR-LED-004` · `BR-LED-017` · B14 6

④ What each act changes, and where — Tab 6

ACT	WRITES	THE LEDGER	ELSEWHERE
Close the month 6·1a	`fiscal_period_closes` with the closing date, the reason and an **incomplete-item snapshot** · `audit_log`	**Posts nothing.** The lock is the effect	**Every module changes behaviour:** an event dated into the closed month now lands in the next open one with a note. ⚠ **Only the manual JE calls** `resolve_entry_date` — a late-dated bill or payment still lands where it was dated · 6·2a's first gate opens
Hand over 6·2a	`export_batches` — who, when, the payload, a content hash, an `EXP-` number	**Posts nothing.** It is a record of what left	6·2's hand-over history gains a row · a second attempt is refused forever · **after P4.2 a file exists**, at the saved destination · if a captured cost is later corrected, the drift panel raises and the handover is **not** repeated
Reopen 6·1b, after P3.2	`fiscal_period_closes` · `audit_log`	**Posts nothing.** The trial balance must be byte-identical either side	Late-dated events stop forwarding · **a handover already sent is not recalled** — the screen says so before you press it

SECTION 04

Journeys

Everything above is a read. **This is the only section whose findings cannot be reconstructed from source**, which is exactly why an invented one would be undetectable.

> HOW THE APP WAS DRIVEN
>
> PostgreSQL 16 started · **405 migrations** applied · `seed_demo` run (tenant `brazosbend`) · Django on :8000 · `vite build` served by `server.mjs` on :8080 — **the deployed same-origin join, not the dev server**. Signed in as **Dana Whitfield, System Admin**. All 17 addressable screens opened and were screenshotted at 1440×1000, and four rule checks were measured with `getComputedStyle` on the rendered page.

TWO THINGS THIS PASS GOT WRONG FIRST

(1) The first walk was unauthenticated. The selector `input[type="email"]` matched nothing — the field is `#signin-email` — so the run proceeded signed out and every Finance screen rendered "*We couldn't load your regional settings.*" with no rail. That looked exactly like a defect and is not one. Re-run authenticated.

(2) A clipping probe that only measured width. W1 was missed on the first sweep because the probe compared `scrollWidth` on leaf text nodes and the clipping element is an ancestor `span.v`. The finding is real and measured; the first probe could not have found it.

Measured on rendered output — all four pass

RULE	REQUIRED	MEASURED	
B14 rule 40	page-head title 32px	`32px`, Archivo, weight 800	PASS
B14 rule 20	table type one size	td `13.5px` — a single value across nine screens; th `12px`	PASS
tokens.css	every corner square	`0px` on every button, panel and table	PASS
R8 · B14 rule 8	left rail 13 items · no native selects	13 items in order; **0** native selects	PASS

What broke

#	SCREEN	DID	GOT	TASK
W1	Revenue & AR · KPI strip	Read the KPI strip	`$23,766…` — `span.v` needs `scrollWidth 159px`, has `clientWidth 146px`, `overflow:hidden; text-overflow:ellipsis` at 26px. **DOM text correct, so every string assertion passes**	P2.1 · P2.2
W2	Revenue & AR · KPI strip	Same strip, the label	"Collected · Posting date" needs 149px into 146px and clips	P2.1
W3	all nine list screens	Counted rendered pagers	**1 of 17 lists has one** — and it pages a client-side slice	P1.8
W4	Revenue & AR · page head	Looked for the primary action	`↓ Download period pack` **disabled**, so the screen has none. Reason false	P2.5
W5	1·3	Opened `?view=accruals`	**0 tables**, one disabled button — over eight live endpoints and four engines	P3.1
W6	5·2	Read the Download reason	Disabled; the reason names a question **ruled on 2026-09-03**	P2.5
W8	3·1 · 2·5	Clicked a status chip, reloaded	Local `useState` — the filter is lost and cannot be shared	P1.9
W9	1·1	Switched sub-view with a filter applied	`setSearchParams({view})` **wipes `q`, `sort`, `page` and every `f.*`**	P1.9
W10	2·5	Arrived from the Payments tab strip	The page draws the **Finance** strip with **Revenue & AR** lit. The tab pressed is not on screen	GAP no task

NOT WALKED, AND WHY

Every write path. `seed_demo` states in its own output that it seeds no finance data — "*customers/accounts, bills, payments and finance: their lanes have not landed — nothing here fakes them*" — so there was no batch to reconcile, no refund to approve and no closable period with unfinished items. **Posting, cutting, importing, reconciling, remitting, closing and handing over were all read from source and none was driven.** Nine of the ten findings above are about reading and reaching screens. **The write paths are UNVERIFIED, not healthy.** The fixture that would close this exists: `06-backend/tests/ledger_le_fixtures.py`, `post_le1_month`, the LE-1 book that foots 598.69 = 598.69.

SECTION 05

Cross-module

Work that is **not a screen in this area**. It sits in its own section because a screen cannot own it, and because the cheapest way to lose a dependency is to bury it in a drawing of something else.

A · Settings

WHAT	WHERE EXACTLY	STATUS	WHY — AND WHAT WAS WRONG WITH THE FIRST ANSWER
Company currency	Settings → Company	EXISTS, WEAK	A free-text `CharField(3)`. **My first answer was that Finance needed almost no settings work — that was wrong.** A reader asked why currency was not a foreign key; there are **27 free-text currency columns** across 7 modules and **no lookup table anywhere**. P0.6 · P0.8
Tax remitting basis, per authority	Settings → Configure	EXISTS	Landed 2026-08-15 as `tax_remitting_basis_by_authority`. Correct: *"the right answer differs per authority"*
GL maps	Settings → Configure *and* Settings → Services	SPLIT	`BR-LED-014` puts them with ledger settings; a service's revenue account lives on Services. **The screen tells the clerk to go to two places** because `PUT /ledger/gl-mappings` is unwired. P3.4
Fiscal period	Settings → Company	EXISTS	The accounting period is **computed** from fiscal settings and never stored (`BR-TEN-022`), served by `GET /shell/display`
Number series	Settings → Numbering	EXISTS	**The `journal_entry` row exists and has never been consumed** — `next_number` is still 1 after 168 entries. This is **not** a settings gap; the allocator is never called. P0.3
Export destination + retention	Settings → Integrations	NEW	P4.9, once a file exists to send
Currency lookup	Reference data	NEW	P0.6. `OQ-031` already ruled the shape: **enums → lookup tables, seeded defaults, tenant-extensible**
Who may close a period	—	NOT SETTINGS	A role gate is a **ruling**, not a tenant setting. It is OPEN and borrowed from `OQ-175`

B · Every other module

MODULE	WHAT	STATUS	WHY, AND WHICH SCREEN NEEDS IT
billing	`gl_account_id` becomes a real FK	DEFERRED	P0.4. Nothing today stops a stale GL pointer on an invoice line
billing	Call `resolve_entry_date`	MISSING	A late-dated bill still lands where it was dated. 6·1a's lock is therefore partial
payments	Three of the ten lists	THEIRS	P1.13 — deposits, refunds and the write-off queue are payments-owned; Finance draws them
payments	Refund detail reads	BUILT, UNWIRED	P3.3 · P3.9 — four routes with zero callers; 2·5a and 2·5d need them
payments	Call `resolve_entry_date`	MISSING	As billing, for a late-dated payment
customers	`is_primary` on the account link	MISSING	P0.2 — **prerequisite for stamping a customer on a journal line**; nothing today says which of two joint holders is billed
customers	`premises_id` becomes a real FK	DEFERRED	P0.5
customers	Write the deposit register	MISSING	**Nothing in the product writes it**, so the customer-deposit row on 2·3 Tie-out can never be checked
rating	Three deferred FKs	DEFERRED	P0.4 · P0.5
tenancy	Two GL pointers; the currency FK	DEFERRED	P0.4 · P0.6
reports	Register five Finance export resources	NEW	P4.1 · P4.10 — the kernel exists and already serves four others

integrations	Two **LEAKY** seams	LEAKY	`newworld.py:624` queries `JournalEntry.objects` directly and `mappings.py:310` imports `LedgerAccount` — **while that module declares its own ledger seam**. A read today; the queryset could be written through tomorrow
metering	Nothing	CLEAR	Finance reads premises and units through stamped FKs, never a live join

> **THE DEPENDENCY THAT GATES THE MOST, AND IT IS NOT OURS**
>
> **Billing and payments must publish their printed document numbers.** Without it, 7·1's Reference column, the partner ledger feed (P5.3) and every reconciliation a partner attempts all carry an **internal identifier** instead of the number on the paper the customer holds. It blocks the whole of Phase 5, it cannot be done inside Finance, and it is the one thing on this page that another team has to want.

SECTION 06

Register

29 screens across seven tabs: **3 new · 18 update · 10 keep · 0 remove**. Counts reconcile with the manifest at the top of this file. **"S-NEW" means a number must be allocated**, which is a governance act through `10-scripts/allocate.py` against the remote — not a drawing decision.

TAB	SCREEN	ADDRESS	CLASS	REPO ID	DOORS	WHAT IT NEEDS
1 · General ledger	1·1 Journal	`/finance`	UPDATE	S56	9	Journal toolbar and pager; premises chip; the false panel deleted
	1·2 Balance	`?view=chart`	UPDATE	S56	1	Toolbar, pager, the GL map save wired
	1·4 Accruals	`?view=accruals`	NEW	S56	1	**The whole screen** — 8 endpoints, 0 callers
	1·4 Accruals	/finance?view=accruals	UPDATE	S56	3	P3.1 · P3.2 · P3.3
	1·5a New entries	modal	KEEP	S57	1	Nothing
	1·5b Reverse	modal	KEEP	S57	1	Nothing
2 · Revenue & AR	2·1 Services	`/finance/revenue?view=services`	UPDATE	S36	3	Toolbar on the shared table · **the landing view**
	2·2 Aging	`?view=aging`	NEW	S-NEW	2	The whole view, and the report's paging and filters — **Reports**
	2·3 Tie-out	`?view=tieout`	UPDATE	S36	2	Toolbar · *Not checked* as an absent figure
	2·4 Write-offs	`?view=writeoffs`	UPDATE	S36	3	Paging, search and the pending count — **Payments**
	2·5 Refunds	`?view=refunds`	UPDATE	S58	4	Paging, search, the count — **Payments** · `/finance/refunds` redirects
	2·4a Approve write-off	modal	KEEP	S36	1	Nothing
	2·5a Approve refund	modal	KEEP	S58	1	The account name, via 2·5d
	2·5b Raise a refund	modal	KEEP	S58	2	Nothing
	2·5c Issue · clear · void	modal	KEEP	S58	1	Nothing
	2·5d Refund detail	`/finance/refunds/:id`	NEW	S-NEW	1	The whole screen — 4 reads, 0 callers
3 · Deposits & Bank	3·1 Deposits	`/finance/deposits`	UPDATE	S37	11	Search, pager, URL state, icon row actions
	3·2 Batch	`/deposits/:batchId`	UPDATE	S39 · S40	4	Pagers; a reversed-member view
	3·3a Cut a day	modal	KEEP	S37	1	Nothing
	3·3b Import	`/deposits/import`	UPDATE	S38	2	**The shared wizard frame**

	3·4 Resolve	modal	KEEP	S40	2	Nothing
	3·5 Statement	`/deposits/statement`	UPDATE	**S-NEW**	1	Pagers; **a second door**
4 · Taxes	4·1 By authority	`/finance/taxes`	UPDATE	S41	3	The authority stamp; the filing register
	4·2 Record a payment	modal	UPDATE	S41	1	The receipt attachment; a real date input
5 · Costs	5·1 Register	`/cost-of-service`	UPDATE	S73	2	Five filters, a pager, an empty state
	5·2 Bought vs billed	`?view=reconciliation`	UPDATE	S74	2	The download sentence; tab semantics
	5·3a Capture	modal	UPDATE	S73	1	The missing catalogue key; the unit picker waits on `OQ-201`
	5·3b Correct · void	modal	KEEP	S73	2	Nothing
6 · Close & export	6·1 Close	`/finance/export?view=close`	UPDATE	S42	4	Two steps with doors and amounts; closed months with reopen
	6·2 Export	`?view=export`	NEW	S43	2	The whole view — dates, grouping, columns, send, history, drift
	6·1a Close the month	modal	UPDATE	S42	1	The consequence in two short sentences
	6·1b Reopen	modal	NEW	S-NEW	1	The whole pop-up — endpoint and wrapper exist, no component
	6·2a Hand over	dialog	UPDATE	S43	1	States that no file is written until P4.2

> **EVERY SCREEN HAS ITS DRAWING**
>
> 35 screens · 35 drawings. **Until 2026-09-23 two did not** — 2·5c *Issue · clear · void* (then 2·4b) and 5·3b *Correct · void* carried a spec table and no wireframe. **Both were drawn**, and a structural check now fails any screen that has none, so this count is measured rather than stated.

> **TWO ADDRESSES THAT ARE NOT SCREENS, AND ONE THAT NAMES NO DOOR**
>
> `/finance/gl` renders 1·1 and stays mounted for bookmarks and tickets. `/finance/:tab` is the unknown-tab fallback — it lands on the General Ledger and **never 404s** (`BR-SHL-011`). It is the **one** route in Finance whose door verdict is **NOTHING**, and that is deliberate, not an orphan.

Capability map — every capability, its screen, and the task that closes it

The PO's map of 2026-09-23, carried over with today's screen numbers. 109 capabilities: **36 built**, **24 partial**, **49 missing**. Two statuses moved on evidence found since — aging (a Reports report exists) and collection % (a KPI tile exists). **9 rows marked NEW** were found on 23–24 September.

> **16 CAPABILITIES HAVE NO TASK, ON PURPOSE**
>
> **A gap with no task beside it is reported, not scheduled** — the six-marker rule in §00. These rows are here so nobody can say they were not seen; scheduling any of them is a decision, not an oversight to be tidied.

CAPABILITY	TODAY	SCREEN	BACKEND	DATABASE	TASK
1 · The books — tab 1					
A1 balanced pair, single writer	✅	1·2	`ledger/posting.py`	journal_entries · journal_lines	—
D1 trial balance read	✅	1·2	`api.py:349 → reporting.trial_balance`	journal_lines (partitioned) · ledger_accounts	—
G1 balanced / out-of-balance verdict	✅	1·2	server flag; the screen computes nothing	—	—
P4 pagination · G4 search · sort	◐	1·2	in the browser — bounded by the chart, so lawful	—	P1.15

G2 premises filter control	⛔	1·2	built: `api.py:482 → reporting.py:730`	journal_lines.premises_id	P2.3 · P0.14
G3 open an entry to its lines and dimensions	⛔	1·1	no read — `journal_api.py` is DELETE-only	—	no task — reported, not scheduled
L1 close checklist strip	◐	1·2 · 6·1	`finance_api.py:493 → close.py`	fiscal_period_closes	P5.9 · P5.10
A3 reverse + repost	✅	1·5b	`journal_api.py:440` (AdminOrClerk)	journal_entries.reversal_of	—
G4 five server filters, no controls	⛔	1·1	`api.py:478-485` accepts all five	—	P1.1
C1 JE- number	⛔	1·1	`PostRequest` has no number field	number_series — format ruled `JE-MMDDYY-nnnn`	P0.3 · P0.13
P4 pagination footer	⛔	1·1	pages in Python, `api.py:487`	—	P1.1 · P1.8
E1 seeded, customisable chart	✅	1·3	`chart_api.py:224 · chart.py`	ledger_accounts	—
E2 audit-lock once posted against	✅	1·3	`services.py`	ledger_accounts.deleted_at	—
E3 designations	✅	1·3	CHECK constraint, 18 values	ledger_accounts.designation	—
E4 export codes save	⛔	1·3	`PUT /ledger/gl-mappings` unwired	ledger_accounts.qbo_ref · xero_ref	P3.4
R2 designations tenant-extensible	◐	1·3	needs a migration to extend	—	no task — reported, not scheduled
F2 accrual auto-reverse	✅	1·4	`manual_je.py` day-1 sweep	journal_entries.auto_reverse_entry	—
F4 four period-end engines	⛔	1·4	8 routes, 0 callers — **two reverse, two stand**	journal_entries	P3.1
F1 draft → post, must balance	✅	1·5a	`manual_je.py · posting.post_draft`	journal_entries.status	—
F5 late-dated auto-forward	◐	1·5a	`close.resolve_entry_date` — manual entries only	fiscal_period_closes	no task — reported, not scheduled
O4 who may reverse	✅	1·5b	AdminOrClerk	—	—
NEW five dimensions on a manual line	⛔	1·5a	`ManualJournalLine` carries account · side · amount only	journal_lines	P1.16
NEW several entries in one sitting	⛔	1·5a	the service posts one entry per call	journal_entries	P1.21 · P1.22
2 · Revenue & AR — tab 2					
H1 KPI strip, basis-labelled	◐	above every view	`api.py:310 → reporting.py`	journal_lines	P2.1
P5 money displays two decimals	◐	KPI strip	`$23,766…` clips — `KPIStrip.css:40,91`	—	P2.1 · P2.2
H2 revenue by service	✅	2·1	`reporting.py:1367` (sorts in Python)	journal_lines.service_id	P2.20
H6 subledger tie-out	✅	2·3	`finance_api.py:548 → tie_out.py`	journal_lines · accounts	P2.20
H3 aging buckets	◐ was ⛔	2·2	the `ar_aging` report and a Dashboard panel exist; **no Finance screen**	—	P2.21
H4 collection %	◐ was ⛔	KPI strip	a collected ÷ billed tile exists; a per-bill rate needs payments linked to bills	—	no task — reported, not scheduled
H5 days sales outstanding	⛔	—	needs twelve months of history	—	no task — reported, not scheduled

H7 customer deposit register	⛔	2·3 · *Not checked*	nothing writes it	(no table)	no task — reported, not scheduled
H8 cash position	✅	KPI strip	`reporting.py`	journal_lines	—
M3 write-off approval queue	✅	2·4 · 2·4a	`payments/writeoffs.py`	write_offs	P2.19
L7 Download period pack	⛔	page head	disabled; its reason is false	export_batches	P2.5 · P4.5
3 · Cash & bank — tab 3					
I1 cut batches, day × tender	✅	3·3a	`reconciliation.cut_deposit_batches:735`	deposit_batches · number_series (DEP-)	—
I2 status filter in the URL	◐	3·1	wired server-side; the front end keeps it in local state	—	P1.9
I2 search · columns · pager	⛔	3·1	no `q` ; unpaginated	—	P1.2 · P1.7 · P1.8
S2 status chips hardcoded	◐	3·1	`DepositsPage.tsx:103-106`	—	P1.4
I8 deposit slip	⛔	3·1	no export resource	—	P4.6
I3 upload → auto-match ±3 days	✅	3·3b	`reconciliation.py`	bank_statement_imports · bank_lines · bank_accounts	P1.14
I7 period tie-out + differences	◐	3·5	four backends, one inbound link	bank_rec_match_sets · members · resolutions	no task — reported, not scheduled
refusal to post an explanation	✅	3·4	correct — `OQ-1310` closed that way	—	—
I4 reconcile with reason + GL	✅	3·4	`reconciliation.py`	bank_rec_resolutions · difference_reasons	—
NEW several causes in one difference	◐	3·4	the service takes a list; the pop-up drew one row	bank_rec_resolutions	P3.12
I5 the reconciled lock	✅	3·2	`_refuse_if_batch_locked`	deposit_batches · deposit_batch_events	—
I6 unreconcile posts nothing	✅	3·2	`BR-LED-024`	—	—
I9 cancel a processing payment	✅	Payments	`payments/cancel.py` — disjoint from the cut	payments	—
I10 reverse inside a locked batch	✅	3·2	`reversal.py:770`	payments	P3.10
I11 locked batch with reversed members	⛔	3·2	nothing surfaces it	—	P3.11
NEW act on a batch line	⛔	3·2	two of three acts already have endpoints	payments	P3.10
4 · Tax — tab 4					
J1 billed basis per authority	✅	4·1	`api.py:506 → reporting.py:2470`	journal_lines · taxes · tax_jurisdictions	—
J1 collected basis	⛔ by design	4·1	correctly refused — a payment does not record which charges it met	—	—
B6 tax authority on the line	⛔	4·1	never stamped — 0 of 663	journal_lines.tax_authority_id	P0.3
J2 remitting basis per authority	✅	Settings	Settings → Configure	tenants	—
J4 record the remittance entry	✅	4·2	`finance_api.py:1110 → journal.record_tax_remittance`	journal_entries	—
J5 filing confirmation reference	✅	4·2	—	journal_entries.memo	—
J6 receipt file attachment	⛔	4·2	`BR-LED-019` requires it	(no column)	P4.3
J7 payments made · filing history	⛔	4·1	POST with no GET, and no model	tax_remittances does not exist	P4.11 · P4.4

B14-7 date field is a real date input	◐	4·2	a bare text input, YYYY-MM-DD	—	no task — reported, not scheduled
5 · Cost of service — tab 5					
K2 capture a supplier invoice	✅	5·3a	`cost_api.py:307 → cost_capture.py`	supplier_invoices	—
K3 optional volume + unit	◐	5·3a	no unit picker — `OQ-201` open	supplier_invoices	no task — reported, not scheduled
NEW upload and view the supplier's invoice	⛔	5·3a · 5·1	no document column anywhere in the platform	supplier_invoices	P5.11
K4 server search in the URL	✅	5·1	`cost_api.py:390 → cost_capture.py:2090`	supplier_invoices	—
K4 five accepted filters, no controls	⛔	5·1	`cost_api.py:385-389`	—	no task — reported, not scheduled
K4 pagination · empty state	⛔	5·1	"*Pagination is deliberately absent*" — `cost_capture.py:2062`	—	P1.8
K5 bought vs billed %	✅	5·2	`cost_reconciliation.py`	supplier_invoices · journal_lines	—
K1 cost chart section signed by finance	⛔	1·3	unsigned	ledger_accounts (5100–5300)	a finance sign-off, not a build task
L3 download the reconciliation	⛔	5·2	`NotImplementedError("OQ-204")`	—	no task — reported, not scheduled
6 · Close & export — tab 6					
L1 checklist computed live	◐	6·1	`close.py` — 3 of 7 items report unknown	fiscal_period_closes	P5.10
NEW close as steps, every item a door	◐	6·1	—	—	P5.9
NEW export as its own view — dates, grouping, columns, history, drift	⛔	6·2	preview, batches, drift and drill-back reads exist	export_batches	P4.12
NEW any start and end date	◐	6·2	`from · to` on preview and send — the screen offers one month	export_batches.period_start · period_end	P4.12
NEW the download carries the chosen columns	⛔	6·2	IS-9 says so; `POST /exports` carries no column state	—	P4.13
L2 warn + confirm with a reason	✅	6·1a	`finance_api.py:627 → close.close_period`	fiscal_period_closes	—
L3 period lock	✅	6·1a	—	fiscal_period_closes	—
L4 reopen	⛔	6·1b	endpoint + client wrapper, no component	fiscal_period_closes	P3.2
L5 idempotent export, four gates	✅	6·2 · 6·2a	`finance_api.py:809 → exports.py:462`	export_batches · number_series (EXP-)	—
L6 drill-back to dimensioned lines	✅	6·2	`exports.py:679`	journal_lines	—
E4 export codes on every row	✅	6·2a	gate 4 refuses without them	ledger_accounts.qbo_ref · xero_ref	—
L7 a file is written or sent	⛔	6·2 · 6·2a	"*No file is written and nothing is emailed*"	—	P4.2
N8 acknowledgement back	⛔	6·2	—	—	P5.4

O5 who may close a period	⛔	6·1a	gate borrowed from `OQ-175`	—	no task — reported, not scheduled
7 · Refunds — tab 2, Payments code					
M1 raise · approve · issue · clear · void	✅	2·5 · 2·5a–c	`refunds_api.py` · `refunds.py` — three role gates	refunds	—
M2 name the account	⛔	2·5 · 2·5d	four reads built, zero callers	refunds · accounts	P3.3
P4 pagination	⛔	2·5	`selectors.py:1569` , deliberately	—	P1.8 · P2.19
S3 status chips hardcoded	◐	2·5	`RefundsPage.tsx:198`	—	P1.5
tab highlight	◐	2·5	lights Revenue & AR — correct once it is a view	—	P2.18
8 · Report viewers — kept in Reports by the PO, 2026-09-24					
G5 account ledger, running balance	✅	`/reports/account-ledger`	`apps/reports`	journal_lines · accounts	not in this blueprint
G6 by premises: billed · collected · owed	◐	`/reports/by-premises`	answers *available: false* correctly	journal_lines.premises_id	not in this blueprint
customer statement	◐	`/reports/customer-statement`	its own endpoint	journal_lines · customers	not in this blueprint
P3 PDF / CSV download	⛔	Reports	disabled; its reason is stale	—	not in this blueprint
9 · Capabilities that had no screen					
F4 four period-end engines	⛔	**now 1·4**	8 routes, ~145 KB, 0 callers	journal_entries	P3.1
E4 GL map bulk save	⛔	**now 1·3**	`PUT /ledger/gl-mappings → gl_mapping_save.py`	ledger_accounts	P3.4
L4 reopen a period	⛔	**now 6·1b**	`POST /finance/periods/reopen`	fiscal_period_closes	P3.2
M2 refund detail + credit sources	⛔	**now 2·5d**	four routes	refunds · accounts	P3.3
cheque register · void	⛔	—	`GET /cheques` , `POST /payments/{id}/cheque/void` — zero callers	payments	no task — reported, not scheduled
payment imports	⛔	—	six routes; nothing in `07-frontend`	—	no task — reported, not scheduled
accounting periods list	⛔	—	`GET /finance/periods`	fiscal_period_closes	no task — reported, not scheduled
export extracts	⛔	**now 6·2's Group by**	`GET /finance/export/extracts/{kind}` — service · premises · authority	export_batches	P4.12
10 · Cross-cutting — belongs to no screen, affects every one					
B7 customer stamped on the line	⛔	—	`posting.PostRequest`	journal_lines (no column)	P0.1 · P0.3
B8 primary holder on the customer link	⛔	—	prerequisite for B7	customer_accounts	P0.2
C1 JE- number allocated at post	⛔	1·1	`posting.PostRequest`	number_series	P0.3 · P0.13

D2 currency default on the model	◐	—	`models.py` still says `default="USD"`	journal_entries.currency	P0.7
D3 currency lookup table	⛔	—	—	27 free-text columns, no table	P0.6 · P0.8
Q1 six GL pointers as bare UUIDs	⛔	—	billing · rating ×3 · tenancy ×2	→ ledger_accounts	P0.4 · P0.12
Q2–Q5 four more deferred FKs	⛔	—	customers · rating ×3	→ metering, customers, identity	P0.5
S1 export targets typed out twice	◐	6·2	`endpoints.ts:17276` copies the backend	—	P1.6
O6 two leaky seams	◐	—	`integrations/newworld.py` , `mappings.py`	→ ledger_accounts	no task — reported, not scheduled
N5–N7 partner feed inherits B6 and C1	⛔	—	`partner_api.partner_ledger`	journal_lines	P5.1 · P5.2 · P5.3

Who owns each screen — and what that means for the plan

Finance owns 19 of its 33 screens. Payments owns 12, one view (2·2 Aging) is built over a Reports report, and one — 2·5d — does not exist yet and would be Payments. They are Payments code, rendered at Finance addresses. **This is the single largest duplication risk in the plan**: a Finance lane that "builds" one of them rebuilds something that already exists and is already tested, in a module it does not own — and hard rule *cross-module writes are forbidden* makes that a boundary violation, not just wasted effort.

TAB	SCREENS	FRONT END	BACKEND
1 · General ledger	1·1 · 1·2 · 1·3 · 1·4 · 1·5a · 1·5b	`features/ledger/`	`apps/ledger/`
2 · Revenue & AR	2·1 Services · 2·3 Tie-out	`features/ledger/`	`apps/ledger/`
	2·4 Write-offs · 2·4a · 2·5 Refunds · 2·5a · 2·5b · 2·5c	**`features/payments/`**	**`apps/payments/`**
	2·5d Refund detail	**does not exist** — no route	`apps/payments/`
	2·2 Aging	**no page yet** — built over the `ar_aging` report	**`apps/reports/`**
3 · Deposits & Bank	**all six** — 3·1 · 3·2 · 3·3a · 3·3b · 3·4 · 3·5	**`features/payments/`**	**`apps/payments/`**
4 · Taxes	4·1 · 4·2	`features/ledger/`	`apps/ledger/`
5 · Costs	5·1 · 5·2 · 5·3a · 5·3b	`features/ledger/`	`apps/ledger/`
6 · Close & export	6·1 · 6·2 · 6·1a · 6·1b · 6·2a	`features/ledger/`	`apps/ledger/`

WHICH TASKS THIS RE-ATTRIBUTES — ELEVEN OF THEM

Every task below touches **another module's** code. **They stay in the plan**, but they are raised, scheduled and reviewed with the module that owns them.

TASK	WHAT	TOUCHES	OWNER
P1.2	Deposits list onto DataTable	`payments/DepositsPage.tsx`	PAYMENTS
P1.3	Refunds queue onto DataTable	`payments/RefundsPage.tsx`	PAYMENTS
P1.4	Deposit chips from the server	`payments/DepositsPage.tsx`	PAYMENTS

P1.5	Refund chips from the server	`payments/RefundsPage.tsx`	PAYMENTS
P1.7	Search deposits, incl. by batch code	`apps/payments/api.py`	PAYMENTS
P1.14	Bank import onto the wizard frame	`payments/BankImportWizard.tsx`	PAYMENTS
P2.19	Paging, search and the count on both queues	`apps/payments/` + two views	PAYMENTS
P3.10	Row actions on the batch lines	`payments/DepositBatchPage.tsx`	PAYMENTS
P3.11	Locked-batch difference	`payments/DepositBatchPage.tsx`	PAYMENTS
P3.12	Multi-line resolve	`payments/DepositBatchPage.tsx`	PAYMENTS
P2.21	The Aging view and the report's paging	`apps/reports/`	REPORTS

The eighteen that remain are Finance's own, and tabs 1, 4, 5 and 6 are clean — every screen in them is `features/ledger/` over `apps/ledger/`.

WHY THE SPLIT EXISTS, IN THE REPO'S OWN WORDS

`BR-PAY-020`: "*Payments owns the transaction record, ledger posts, FINANCE REPORTS.*" A refund, a write-off and a deposit are **payment-lifecycle facts**; Finance is where they are **read, approved and reconciled**. `CR-060` C25 folded the queues into Revenue & AR "*where the receivable it gives up already lives*" — **the door moved, the code did not**. That is the intended design and this register exists so nobody undoes it by accident.